

15 Peripherals interconnect matrix

15.1 Interconnect matrix introduction

Several peripherals have direct connections between them, enabling autonomous communication and/or synchronization: this approach saves CPU resources, and power supply consumption. In addition, these hardware connections remove software latency and help the design of predictable system.

Depending on peripherals, these interconnections can operate in Run, Sleep, and Stop modes.

15.2 Connection summary

Table 133. Peripherals interconnect matrix⁽¹⁾⁽²⁾⁽³⁾

Source	Destination																				
	TIM1	TIM8	TIM2	TIM3	TIM4	TIM5	TIM6	TIM7	TIM12	TIM15	TIM16	TIM17	LPTIM1/2/3	LPTIM4	LPTIM5/6	ADC1/2	DAC1/2	GPDMA1/2	TAMP	RTC	AES/SAES
TIM1	-	1	1	1	1	1	-	-	1	1	-	-	-	-	-	2	4	-	-	-	-
TIM8	1	-	1	1	1	1	-	-	1	1	-	-	-	-	-	2	4	-	-	-	-
TIM2	1	1	-	1	1	1	-	-	1	1	-	-	-	-	-	2	4	10	-	-	-
TIM3	1	1	1	-	1	1	-	-	1	1	-	-	-	-	-	2	-	-	-	-	-
TIM4	1	1	1	1	-	1	-	-	1	1	-	-	-	-	-	2	4	-	-	-	-
TIM5	1	1	1	1	1	-	-	-	1	1	-	-	-	-	-	-	4	-	-	-	-
TIM6	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	2	4	-	-	-	-
TIM7	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	4	-	-	-	-
TIM12	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-	4	10	-	-	-
TIM13	1	1	1	1	1	1	-	-	1	1	-	-	-	-	-	-	-	-	-	-	-
TIM14	1	1	1	1	1	1	-	-	1	1	-	-	-	-	-	-	-	-	-	-	-
TIM15	1	1	1	1	1	-	-	-	1	-	-	-	-	-	-	2	4	10	-	-	-
TIM16	1	1	1	1	1	-	-	-	1	-	-	-	-	-	-	-	-	-	-	-	-
TIM17	1	1	1	1	1	-	-	-	1	-	-	-	-	-	-	-	-	-	-	-	-
LPTIM1/2	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	2	4	10	-	-	-
LPTIM3/4/5/6	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	10	-	-	-
ADC1	3	3	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
ADC2	3	3	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	13	-	-
GPDMA1/2	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	10	-	-	-
EXTI	-	-	-	-	-	-	-	-	-	-	-	-	6	6	6	2	4	10	-	-	-
RTC wake-up	-	-	-	-	-	7	-	-	-	-	7	-	-	-	-	-	-	10	-	-	-
RTC alarm	-	-	-	-	-	-	-	-	-	-	-	-	6	6	6	-	-	10	-	-	-
TAMP	-	-	-	-	-	-	-	-	-	-	-	-	6	6	6	-	-	10	-	14	15
HSE	-	-	-	-	-	-	-	-	-	-	-	5	-	-	-	-	-	-	-	-	-
LSE	-	-	5	-	-	-	-	-	-	5	-	5	-	-	-	-	12	-	-	-	-
CSS in LSE	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	13	-	-
CSI	-	-	-	-	-	-	-	-	5	5	-	-	-	-	-	-	-	-	-	-	-
HSI	-	-	-	-	-	-	-	-	5	-	-	-	-	-	-	-	-	-	-	-	-
LSI	-	-	-	-	-	-	-	-	-	-	5	-	-	-	-	-	-	-	-	-	-

Table 133. Peripherals interconnect matrix⁽¹⁾⁽²⁾⁽³⁾ (continued)

Source	Destination																				
	TIM1	TIM8	TIM2	TIM3	TIM4	TIM5	TIM6	TIM7	TIM12	TIM15	TIM16	TIM17	LPTIM1/2/3	LPTIM4	LPTIM5/6	ADC1/2	DAC1/2	GPDMA1/2	TAMP	RTC	AES/SAES
MCO1	-	-	-	-	-	-	-	-	-	-	-	5	-	-	-	-	-	-	-	-	-
MCO2	-	-	-	-	-	-	-	-	-	5	-	-	-	-	-	-	-	-	-	-	-
V _{CORE}	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	11	-	-	-	-	-
V _{REFINT}	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	11	-	-	-	-	-
V _{sensor}	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	11	-	-	-	-	-
V _{BAT8}	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	11	-	-	-	-	-
V _{BAT} / Temp. monitor	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	13	-	-
System errors	8	8	-	-	-	-	-	-	-	8	8	8	-	-	-	-	-	-	-	-	-
System flash memory	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	15
AES/SAES	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	15
Ethernet	-	-	9	9	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-

1. Numbers in this table are links to corresponding subsections of [Section 15.3](#).

2. "-" means no interconnect.

3. Refer to the product datasheet for the availability of peripherals. If the peripheral is not present, consider the associated interconnection as unavailable.

15.3 Interconnection details

15.3.1 Master to slave interconnection for timers

From timer (TIM1/2/3/4/5/8/12/13/14/15/16/17) to timer (TIM1/2/3/4/5/8/12/15).

Purpose

Some timers are linked together internally for synchronization or chaining.

When one timer is configured in master mode, it can reset, start, stop, or clock the counter of another timer configured in slave mode.

A description of the feature is provided in [Section 39.4.23: Timer synchronization](#).

The synchronization modes are detailed in:

- [Section 38.3.30](#) for advanced-control timers TIM1/8
- [Section 39.4.22](#) for general-purpose timers TIM2/3/4/5
- [Section 41](#) for the general-purpose timers TIM12/13/14
- [Section 42.4.23](#) for the general-purpose timer TIM15

Triggering signals

The output (from master) is on signal TIMx_TRGO (and TIMx_TRGO2 for TIM1/8) following a configurable timer event. It can be also from signals tim16_oc1 and tim17_oc1 in case of TIM16/17. The input (to slave) is on signals TIMx_ITR0/1/2/3.

The possible master/slave connections are given in:

- [Table 398](#) for advanced-control timers TIM1/8
- [Table 421](#) for general-purpose timers TIM2/3/4/5
- [Table 442](#) for the general-purpose timers TIM12/13/14
- [Table 454](#) for the general-purpose timer TIM15

Active power mode

Timers are optionally active in Run and Sleep modes. The effects of low-power modes on TIMx are given in:

- [Table 410: Effect of low-power modes on TIM1/TIM8](#)
- [Table 428: Effect of low-power modes on TIM2/TIM3/TIM4/TIM5](#)
- [Table 444: Effect of low-power modes on TIM12/TIM13/TIM14](#)
- [Table 460: Effect of low-power modes on TIM15/TIM16/TIM17](#)

15.3.2 Triggers to ADCs

From EXTI, timers (TIM1/2/3/4/6/8/15) and LP timers (LPTIM1/2) to ADC1/ADC2.

Purpose

A conversion (or a sequence of conversions) can be triggered either by software or by an external event (such as timer capture or input pins). For ADC12, if the EXTEN[1:0] (for a regular conversion) or JEXTEN[1:0] bits (for an injected conversion) are different from 0b00, external events can trigger a conversion with the selected polarity.

More details in:

- [Section 26.4.18: Conversion on external trigger and trigger polarity \(EXTSEL, EXTEN, JEXTSEL, JEXTEN\)](#)
- EXTEN[1:0] defined in [ADC configuration register \(ADC_CFGR\)](#)
- JEXTEN[1:0] defined in [ADC injected sequence register \(ADC_JSQR\)](#)

General-purpose timers (TIM2/3/4), basic timer (TIM6), advanced-control timers (TIM1/8), and general-purpose timer (TIM15) can be used to generate the ADC triggering event through the timer outputs tim_oc and tim_trgo.

Low-power timers (LPTIM1/2) can be used to generate the ADC triggering event through the LPTIM channels, in addition to the EXTI on channels 11 and 15.

Triggering signals

For ADC1/ADC2, the input triggering signals and the description of the interconnection between ADC1/ADC2 and timers are given in:

- adc_ext_trg: [Table 257: ADC interconnection](#)
- adc_jext_trg: [Table 257: ADC interconnection](#)
- [Section 26.4.18: Conversion on external trigger and trigger polarity \(EXTSEL, EXTEN, JEXTSEL, JEXTEN\)](#)
- [Section 26.4.25: Timing diagrams example \(single/continuous modes, hardware/software triggers\)](#)

Active power mode

This interconnection is active in Run and Sleep modes for all ADCs. The timers are active only in Run and Sleep modes. The effects of low-power modes are given in:

- [Table 410: Effect of low-power modes on TIM1/TIM8](#)
- [Table 428: Effect of low-power modes on TIM2/TIM3/TIM4/TIM5](#)
- [Table 460: Effect of low-power modes on TIM15/TIM16/TIM17](#)
- [Table 466: STM32H5 LPTIM features](#)
- [Table 481: Effect of low-power modes on the LPTIM](#)

15.3.3 ADC analog watchdogs as triggers to timers

From ADC1/ADC2 to TIM1/8.

Purpose

The internal analog watchdog output signals coming from ADC1/ADC2 are connected to on-chip timers. ADC1/ADC2 can provide trigger events through analog watchdog signals to advanced-control timers (TIM1/8) to reset, start, stop, or clock the counter.

Settings description of the ADC analog watchdog and timer trigger, are provided in:

- [Section 38.3.6: External trigger input](#) for TIM1/8
- [Table 399](#) for the internal ADC1/ADC2 sources connected to TIM1/8 (tim_etr) input multiplexer
- [Section 26.4.28](#) for the ADC1/ADC2/ADC_AWDy_OUT signal output generation

Triggering signals

The output (from ADC) is on signals ADCn_AWDx_OUT, with n being the ADC instance and x = 1, 2, 3 (three watchdogs per ADC). The input (to timer) is on signal TIMx_ETR (external trigger).

Active power mode

ADC1/ADC2 are active in Run and Sleep modes.

15.3.4 Triggers to DAC

From timer (TIM1/2/4/5/6/7/8/15), low-power timers (LPTIM1/2), and EXTI to DAC.

Purpose

General-purpose timers (TIM2/4/5/15), basic timers (TIM6/7), advanced-control timers (TIM1/8), LP timers (LPTIM1/2) outputs channels (lptim1_ch1 and lptim2_ch1), and EXTI can be used as triggering event to start a DAC conversion.

Triggering signals

The output (from timer) on the TIMx_TRGO signal and from LP timers are directly connected to corresponding DAC inputs.

The selection of input triggers on DAC is provided in:

- [Table 281: DAC interconnection](#)
- [Section 28.4.8: DAC trigger selection](#)

Active power mode

This interconnect is active in Run, Sleep, and Stop modes.

15.3.5 Clock sources to timers

From HSE, LSE, LSI, HSI, and MCO to timers (TIM2/12/15/16/17) and LP timers (LPTIM1/2).

Purpose

A timer input or counter can receive different clock sources, and can be used, for example, to calibrate the internal oscillator on a reference clock.

External clocks (HSE, LSE), internal clocks (LSI, CSI, HSI), and microcontroller output clock (MCO) can be used as input to timers:

- LSE, HSI, and CSI are assigned to general purpose timers TIM2/12 as external inputs signals. LSE can be selected as counter clock provided by an external clock source in mode1 (tim_ti1_in) and mode2 (external trigger input tim_etr_in). Inputs assignment and clock selection description are detailed in:
 - [Section 39.4.5: Clock selection](#) for TIM2
 - External clock mode1: [Table 440: Interconnect to the tim_ti1 input multiplexer](#) for TIM12, tim_ti1_in4 (HSI), and tim_ti1_in5 (CSI)
 - External clock mode2: [Table 422: Interconnect to the tim_etr input multiplexer](#) for tim_etr3 (LSE)
- LSE, LSI, CSI, and HSE are assigned to general purpose timers TIM15/16/17 as external inputs signals. LSE/LSI/CSI/HSE can be selected as counter clock provided by an external clock source in mode1 (tim_ti1 or tim_ti2 signals). Inputs assignment and clock selection description are detailed in:
 - [Section 42.4.6: Clock selection](#) for TIM15/16/17. External clock mode1: external input pin (tim_ti1 or tim_ti2, if available)
 - [Table 440: Interconnect to the tim_ti1 input multiplexer](#), tim_ti1_in1 (LSI-TIM16), tim_ti1_in2 (LSE-TIM16/HSE-TIM17), tim_ti1_in4 (LSE-TIM15), and tim_ti1_in5 (CSI-TIM15)
- Microcontroller output clock (MCO): MCO1 is connected as external input to general-purpose timer TIM17, MCO2 is connected as external input to general-purpose timer TIM15.
 - [Table 452: Interconnect to the tim_ti1 input multiplexer](#) for TIM15/TIM16/TIM17
- LSI and LSE can be selected as input capture 2 to LPTIM1 as described in [Table 475: LPTIM1 input capture 2 connections](#).
- HSI/1024, CSI/128, and HSI/8 can be selected as input capture 2 to LPTIM2 as described in [Table 476: LPTIM2 input capture 2 connections](#).

Triggering signals

lptim_ic2_mux1 LPTIM input capture selection can be set in the LPTIM configuration register 2 (LPTIM_CFGR2). For timers, the internal clock signal can be selected as counter clock provided by an external clock source in mode1 (tim_ti1_in) and mode2 (external trigger input tim_etr_in).

Active power mode

This feature is available under Run and Sleep modes.

15.3.6 Triggers to low-power timers

From EXTI, TAMP, and RTC alarm to LP timers (LPTIM1/2/3/4/5/6).

Purpose

LPTIM1/2/3/4/5/6 counters can be started either by software, or after the detection of an active edge on one of the eight trigger inputs (see [Section 43.4.7: Trigger multiplexer](#)).

GPIO can also be selected as LPTIM input capture selection or LPTIM input selection, according to the LPTIM configuration register 2 (LPTIM_CFGR2).

Triggering signals

This trigger feature is described in [Section 43.4.7: Trigger multiplexer](#) and the following sections. The input selection is described in [Table 471: LPTIM1/2/3/4/5/6 external trigger connections](#).

Active power mode

This interconnection is active in Run, Sleep, and Stop modes.

15.3.7 RTC wake-up as inputs to timers

From RTC to timer (TIM16).

Purpose

RTC wake-up interrupt can be used as input to general-purpose timer (TIM16) channel 1.

Triggering signals

RTC wake-up signal is connected to tim_ti1_in3 signal as described in [Table 452: Interconnect to the tim_ti1 input multiplexer](#) for TIM16.

Active power mode

This interconnection is active down to Stop mode. Timers are not active but the count is performed at wake-up.

15.3.8 System errors as break signals to timers

From system errors to timers (TIM1/8/15/16/17).

Purpose

CSS, CPU lockup, SRAM2/3 ECC double errors, SRAM1 parity errors, FLASH ECC double-error detection, and PVD can generate system errors in the form of timer break toward timers (TIM1/8/15/16/17).

The purpose of the break function is to protect power switches driven by PWM signals generated by the timers.

Triggering signals

The possible sources of break are described in:

- [Section 38.3.18: Using the break function](#) for TIM1/8
- [Section 42.4.15: Using the break function](#) for TIM15/16/17
- [Table 402: System break interconnect](#) for TIM1/8
- [Table 456: System break interconnect](#) for TIM15/16/17

Active power mode

Timers are optionally active in Run and Sleep modes. The effects of low-power modes on TIMx are given in:

- [Table 410: Effect of low-power modes on TIM1/TIM8](#)
- [Table 428: Effect of low-power modes on TIM2/TIM3/TIM4/TIM5](#)
- [Table 460: Effect of low-power modes on TIM15/TIM16/TIM17](#)

15.3.9 Triggers for communication peripherals

From Ethernet to timers (TIM2/TIM3).

Purpose

To synchronize system clock with network, internal connections are available between timers and PTP to check for clock drifts.

Triggering signals

- The outputs (from timer) are directly connected to Ethernet PTP triggers inputs. More details are given in the sections below:
- [Section 57.3: Ethernet pins and internal signals](#)
- [Table 417: Interconnect to the tim_ti1 input multiplexer](#)
- [Table 422: Interconnect to the tim_etr input multiplexer](#)

Active power mode

These interconnections remain active in Run and Sleep modes.

15.3.10 Triggers to GPDMA1/2

From EXTI, RTC (alarm/wake-up), TAMP, timers (TIM2/12/15), LP timers (LPTIM1/2/3/4/5/6), GPDMA1 transfer complete (gpdma1_chx_tc, gpdma2_chx_tcf) to GPDMA1/2.

Purpose

A GPDMA trigger can be assigned to GPDMA channel x. A programmed GPDMA transfer can be triggered by a rising/falling edge of a selected input trigger event. The trigger mode can also be programmed to condition the LLI link transfer. More details are given in the sections below:

- [Section 16.3.7: GPDMA triggers](#)
- [Section 16.4.12: GPDMA triggered transfer](#)

- [GPDMA channel x transfer register 2 \(GPDMA_CxTR2\)](#) for more details on:
 - Trigger selection TRIGSEL[5:0] field
 - Trigger mode (LLI) defined by TRIGM[1:0]
 - Trigger polarity as defined by TRIGPOL[1:0]

Triggering signals

GPDMA trigger mapping is specified in [Table 140: Programmed GPDMA1/2 trigger](#), according to GPDMA_CxTR2.TRIGSEL[5:0].

Active power mode

This interconnection remains functional in Sleep mode.

Refer to:

- [Section 16.6: GPDMA in low-power modes](#)

15.3.11 Internal analog signals to analog peripherals

From internal analog source to ADC (ADC1/2).

Purpose

The internal reference voltage (V_{REFINT}), the internal temperature sensor (V_{SENSE}), the internal digital core voltage (V_{DDCORE}), and the V_{BAT} monitoring channel are connected to ADC (ADC1/2) input channels.

This is according to:

- [Section 26.2: ADC main features](#)
- [Section 26.4.11: Channel selection \(ADC_SQRy, ADC_JSQR\)](#)

15.3.12 Clock source for the DAC sample and hold mode

From LSI/LSE to DAC1.

Purpose

DAC1 can run in Stop mode. The sample and hold block and its associated registers use the LSI or LSE clock source (dac_hold_ck) in Stop mode.

[Table 280: DAC internal input/output signals](#): dac_hold_ck, Input, DAC low-power clock used in sample and hold mode

Active power mode

This feature remains available in Run, Sleep and Stop modes.

15.3.13 Internal tamper sources

From internal peripherals, clocks, or monitoring, to tamper.

Purpose

To detect any abnormal activity or tentative to corrupt the device, embedded tampers alert the system of undesired events. Different actions can be taken as consequence.

The list of tamper sources can be found in [Table 507: TAMP interconnection](#).

Active power mode

This interconnection is active in all power modes if the tamper source is activated.

15.3.14 Output from tamper to RTC

From TAMP to RTC.

Purpose

The RTC can timestamp a tamper event to retrieve history of the detection. The RTC can also control GPIOs, and set a signal based on tamp or alarm status outside the MCU.

Refer to [Section 46.3.3: GPIOs controlled by the RTC and TAMP](#) for more details.

Active power mode

This interconnection remains active in all power modes.

15.3.15 Encryption keys to AES/SAES

From TAMP backup registers, system flash memory to and in between SAES and AES.

Purpose

The encryption mechanism requires an hardware key that must be stored in a protected non-volatile memory. Different approaches are implemented to load them in a non-readable way. Tamper backup registers or system flash memory can be used to store respectively BHK or RHUK, and to implement a dedicated bus to pass it to the SAES.

Refer to [Section 34.4.14: SAES operation with wrapped keys](#) for more details.

The AES encryption mechanism (faster than the SAES) can benefit from the sharing key of the SAES. Refer to [Section 34.4.15: SAES operation with shared keys](#) for more details.

Active power mode

AES and SAES are operating under Run and Sleep modes.

16 General purpose direct memory access controller (GPDMA)

16.1 GPDMA introduction

The general purpose direct memory access (GPDMA) controller is a bus master and system peripheral.

The GPDMA is used to perform programmable data transfers between memory-mapped peripherals and/or memories via linked-lists, upon the control of an off-loaded CPU.

16.2 GPDMA main features

- Dual bidirectional AHB master
- Memory-mapped data transfers from a source to a destination:
 - Peripheral-to-memory
 - Memory-to-peripheral
 - Memory-to-memory
 - Peripheral-to-peripheralTransfer arbitration based on a 4-grade programmed priority at channel level:
 - One high-priority traffic class, for time-sensitive channels (queue 3)
 - Three low-priority traffic classes, with a weighted round-robin allocation for non time-sensitive channels (queues 0, 1, 2)
- Per channel event generation, on any of the following events: transfer complete, half transfer complete, data transfer error, user setting error, link transfer error, completed suspension, and trigger overrun
- Per channel interrupt generation, with separately programmed interrupt enable per event
- 8 concurrent GPDMA channels:
 - Per channel FIFO for queuing source and destination transfers (see [Section 16.3.2](#))
 - Intra-channel GPDMA transfers chaining via programmable linked-list into memory, supporting two execution modes: run-to-completion and link step mode
 - Intra-channel and inter-channel GPDMA transfers chaining via programmable GPDMA input triggers connection to GPDMA task completion events
- Per linked-list item within a channel:
 - Separately programmed source and destination transfers
 - Programmable data handling between source and destination: byte-based reordering, packing or unpacking, padding or truncation, sign extension and left/right realignment
 - Programmable number of data bytes to be transferred from the source, defining the block level
 - Linear source and destination addressing: either fixed or contiguously incremented addressing, programmed at a block level, between successive burst transfers

- 2D source and destination addressing: programmable signed address offsets between successive burst transfers (non-contiguous addressing within a block, combined with programmable signed address offsets between successive blocks, at a second 2D/repeated block level, for a reduced set of channels (see [Section 16.3.2](#))
- Support for scatter-gather (multi-buffer transfers), data interleaving and deinterleaving via 2D addressing
- Programmable GPDMA request and trigger selection
- Programmable GPDMA half transfer and transfer complete events generation
- Pointer to the next linked-list item and its data structure in memory, with automatic update of the GPDMA linked-list control registers
- Channel abort and restart
- Debug:
 - Channel suspend and resume support
 - Channel status reporting, including FIFO level, and event flags
- TrustZone support:
 - Support for secure and nonsecure GPDMA transfers, independently at a first channel level, and independently at a source/destination and link sublevels
 - Secure and nonsecure interrupts reporting, resulting from any of the respectively secure and nonsecure channels
 - TrustZone-aware AHB slave port, protecting any GPDMA secure resource (register, register field) from a nonsecure access
- Privileged/unprivileged support:
 - Support for privileged and unprivileged GPDMA transfers, independently at a channel level
 - Privileged-aware AHB slave port

16.3 GPDMA implementation

16.3.1 GPDMA instances

There are two instances of the GPDMA in the devices, named as GPDMA1 and GPDMA2.

Each GPDMA instance has the same channel-based implementation and is connected to the same requests and triggers, as detailed in the following sub-sections.

16.3.2 GPDMA channels

A given GPDMA channel *x* is implemented with the following features and intended usage. To make the best use of the GPDMA performances, the table below lists some general recommendations, allowing the user to select and allocate a channel, given its implemented FIFO size and the requested GPDMA transfer.

Table 134. GPDMA1/2 channel implementation

Channel x	Hardware parameters		Features
	dma_fifo_size[x]	dma_addressing[x]	
x = 0 to 3	2	0	Channel x (x = 0 to 3) is implemented with: – a FIFO of 8 bytes, 2 words – fixed/contiguously incremented addressing These channels must be typically allocated for GPDMA transfers between an APB or AHB peripheral and SRAM.
x = 4 to 5	4	0	Channel x (x = 4 to 5) is implemented with: – a FIFO of 32 bytes, 8 words – fixed/contiguously incremented addressing These channels may be used for GPDMA transfers between a demanding AHB peripheral and SRAM, or for transfers from/to external memories.
x = 6 to 7	4	1	Channel x (x = 6 to 7) is implemented with: – a FIFO of 32 bytes, 8 words – 2D addressing These channels may be used for GPDMA transfers between a demanding AHB peripheral and SRAM, or for transfers from/to external memories.

16.3.3 GPDMA in low-power modes

The GPDMA wake-up feature is implemented in the device low-power modes as per the table below.

Table 135. GPDMA1/2 wake-up in low-power modes

Feature	Low-power modes
Wake-up	GPDMA1/2 in Sleep mode

16.3.4 GPDMA requests

A GPDMA request from a peripheral can be assigned to a GPDMA channel x, via REQSEL[7:0] in GPDMA_CxTR2, provided that SWREQ = 0.

The GPDMA requests mapping is specified in the table below.

Table 136. Programmed GPDMA1/2 request

GPDMA_CxTR2.REQSEL[7:0]	Selected GPDMA request
0	adc1_dma
1	adc2_dma
2	dac1_ch1_dma
3	dac1_ch2_dma
4	tim6_upd_dma

Table 136. Programmed GPDMA1/2 request (continued)

GPDMA_CxTR2.REQSEL[7:0]	Selected GPDMA request
5	tim7_upd_dma
6	spi1_rx_dma
7	spi1_tx_dma
8	spi2_rx_dma
9	spi2_tx_dma
10	spi3_rx_dma
11	spi3_tx_dma
12	i2c1_rx_dma
13	i2c1_tx_dma
14	Reserved
15	i2c2_rx_dma
16	i2c2_tx_dma
17	Reserved
18	i2c3_rx_dma
19	i2c3_tx_dma
20	Reserved
21	usart1_rx_dma
22	usart1_tx_dma
23	usart2_rx_dma
24	usart2_tx_dma
25	usart3_rx_dma
26	usart3_tx_dma
27	uart4_rx_dma
28	uart4_tx_dma
29	uart5_rx_dma
30	uart5_tx_dma
31	usart6_rx_dma
32	usart6_tx_dma
33	uart7_rx_dma ⁽¹⁾
34	uart7_tx_dma ⁽¹⁾
35	uart8_rx_dma ⁽¹⁾
36	uart8_tx_dma ⁽¹⁾
37	uart9_rx_dma ⁽¹⁾
38	uart9_tx_dma ⁽¹⁾
39	uart10_rx_dma ⁽¹⁾

Table 136. Programmed GPDMA1/2 request (continued)

GPDMA_CxTR2.REQSEL[7:0]	Selected GPDMA request
40	uart10_tx_dma ⁽¹⁾
41	uart11_rx_dma ⁽¹⁾
42	uart11_tx_dma ⁽¹⁾
43	uart12_rx_dma ⁽¹⁾
44	uart12_tx_dma ⁽¹⁾
45	lpuart1_rx_dma
46	lpuart1_tx_dma
47	spi4_rx_dma
48	spi4_tx_dma
49	spi5_rx_dma ⁽¹⁾
50	spi5_tx_dma ⁽¹⁾
51	spi6_rx_dma ⁽¹⁾
52	spi6_tx_dma ⁽¹⁾
53	sai1_a_dma ⁽¹⁾
54	sai1_b_dma ⁽¹⁾
55	sai2_a_dma ⁽¹⁾
56	sai2_b_dma ⁽¹⁾
57	ospi1_dma
58	tim1_cc1_dma
59	tim1_cc2_dma
60	tim1_cc3_dma
61	tim1_cc4_dma
62	tim1_upd_dma
63	tim1_trg_dma
64	tim1_com_dma
65	tim8_cc1_dma
66	tim8_cc2_dma
67	tim8_cc3_dma
68	tim8_cc4_dma
69	tim8_upd_dma
70	tim8_tig_dma
71	tim8_com_dma
72	tim2_cc1_dma
73	tim2_cc2_dma
74	tim2_cc3_dma

Table 136. Programmed GPDMA1/2 request (continued)

GPDMA_CxTR2.REQSEL[7:0]	Selected GPDMA request
75	tim2_cc4_dma
76	tim2_upd_dma
77	tim3_cc1_dma
78	tim3_cc2_dma
79	tim3_cc3_dma
80	tim3_cc4_dma
81	tim3_upd_dma
82	tim3_trg_dma
83	tim4_cc1_dma
84	tim4_cc2_dma
85	tim4_cc3_dma
86	tim4_cc4_dma
87	tim4_upd_dma
88	tim5_cc1_dma
89	tim5_cc2_dma
90	tim5_cc3_dma
91	tim5_cc4_dma
92	tim5_upd_dma
93	tim5_trg_dma
94	tim15_cc1_dma
95	tim15_upd_dma
96	tim15_trg_dma
97	tim15_com_dma
98	tim16_cc1_dma ⁽¹⁾
99	tim16_upd_dma ⁽¹⁾
100	tim17_cc1_dma ⁽¹⁾
101	tim17_upd_dma ⁽¹⁾
102	lptim1_ic1_dma
103	lptim1_ic2_dma
104	lptim1_ue_dma
105	lptim2_ic1_dma
106	lptim2_ic2_dma
107	lptim2_ue_dma
108	dcmi_dma or pssi_dma ⁽²⁾
109	aes_out_dma

Table 136. Programmed GPDMA1/2 request (continued)

GPDMA_CxTR2.REQSEL[7:0]	Selected GPDMA request
110	aes_in_dma
111	hash_in_dma
112	ucpd1_rx_dma
113	ucpd1_tx_dma
114	cordic_read_dma ⁽¹⁾
115	cordic_write_dma ⁽¹⁾
116	fmac_read_dma ⁽¹⁾
117	fmac_write_dma ⁽¹⁾
118	saes_out_dma
119	saes_in_dma
120	i3c1_rx_dma
121	i3c1_tx_dma
122	i3c1_tc_dma
123	i3c1_rs_dma
124	i2c4_rx_dma ⁽¹⁾
125	i2c4_tx_dma ⁽¹⁾
126	Reserved
127	lptim3_ic1_dma ⁽¹⁾
128	lptim3_ic2_dma ⁽¹⁾
129	lptim3_ue_dma ⁽¹⁾
130	lptim5_ic1_dma ⁽¹⁾
131	lptim5_ic2_dma ⁽¹⁾
132	lptim5_ue_dma ⁽¹⁾
133	lptim6_ic1_dma ⁽¹⁾
134	lptim6_ic2_dma ⁽¹⁾
135	lptim6_ue_dma ⁽¹⁾
136	i3c2_rx_dma ⁽³⁾
137	i3c2_tx_dma ⁽³⁾
138	i3c2_tc_dma ⁽³⁾
139	i3c2_rs_dma ⁽³⁾

1. Not available in STM32H523/533 devices.

2. Depends on which exclusive function is used.

3. Not available in STM32H562/563/573 devices.

16.3.5 GPDMA block requests

Some GPDMA requests must be programmed as a block request, and not as a burst request. Then BREQ in GPDMA_CxTR2 must be set for a correct GPDMA execution of the requested peripheral transfer at the hardware level.

Table 137. Programmed GPDMA1/2 request as a block request

GPDMA block requests
lptim1_ue_dma
lptim2_ue_dma
lptim3_ue_dma ⁽¹⁾
lptim5_ue_dma ⁽¹⁾
lptim6_ue_dma ⁽¹⁾

1. Not available in STM32H523/533 devices.

16.3.6 GPDMA channels with peripheral early termination

A GPDMA channel, if implemented with this feature, can support the early termination of the data transfer from the peripheral which does also support this feature.

Table 138. GPDMA1/2 channel with peripheral early termination

GPDMA channel x with peripheral early termination
x = 0 and x = 7

This GPDMA support is activated when the channel x is programmed with GPDMA_CxTR2.PFREQ = 1. Then, the peripheral itself can initiate and request a data transfer completion, before that the GPDMA has transferred the whole block (see [Section 16.4.14](#) for more details).

Table 139. Programmed GPDMA1/2 request with peripheral early termination

Programmed GPDMA channel x request with peripheral early termination
i3c2_rx_dma
i2c2_rx_dma ⁽¹⁾

1. Not available in STM32H562/563/573 devices.

16.3.7 GPDMA triggers

A GPDMA trigger can be assigned to a GPDMA channel x, via TRIGSEL[5:0] in GPDMA_CxTR2, provided that TRIGPOL[1:0] defines a rising or a falling edge of the selected trigger (TRIGPOL[1:0] = 01 or TRIGPOL[1:0] = 10).

Table 140. Programmed GPDMA1/2 trigger

GPDMA_CxTR2.TRIGSEL[5:0]	Selected GPDMA trigger
0	exti0
1	exti1
2	exti2
3	exti3
4	exti4
5	exti5
6	exti6
7	exti7
8	tamp_trg1
9	tamp_trg2
10	tamp_trg3
11	lptim1_ch1
12	lptim1_ch2
13	lptim2_ch1
14	lptim2_ch2
15	rtc_alra_trg
16	rtc_alrb_trg
17	rtc_wut_trg
18	gpdma1_ch0_tc
19	gpdma1_ch1_tc
20	gpdma1_ch2_tc
21	gpdma1_ch3_tc
22	gpdma1_ch4_tc
23	gpdma1_ch5_tc
24	gpdma1_ch6_tc
25	gpdma1_ch7_tc
26	gpdma2_ch0_tc
27	gpdma2_ch1_tc
28	gpdma2_ch2_tc
29	gpdma2_ch3_tc
30	gpdma2_ch4_tc
31	gpdma2_ch5_tc
32	gpdma2_ch6_tc
33	gpdma2_ch7_tc
34	tim2_trgo

Table 140. Programmed GPDMA1/2 trigger (continued)

GPDMA_CxTR2.TRIGSEL[5:0]	Selected GPDMA trigger
35	tim15_trgo
36	tim12_trgo
37	lptim3_ch1 ⁽¹⁾
38	lptim3_ch2 ⁽¹⁾
39	lptim4_ait ⁽¹⁾
40	lptim5_ch1 ⁽¹⁾
41	lptim5_ch2 ⁽¹⁾
42	lptim6_ch1 ⁽¹⁾
43	lptim6_ch2 ⁽¹⁾
44	Reserved
45	EVENTOUT ⁽²⁾

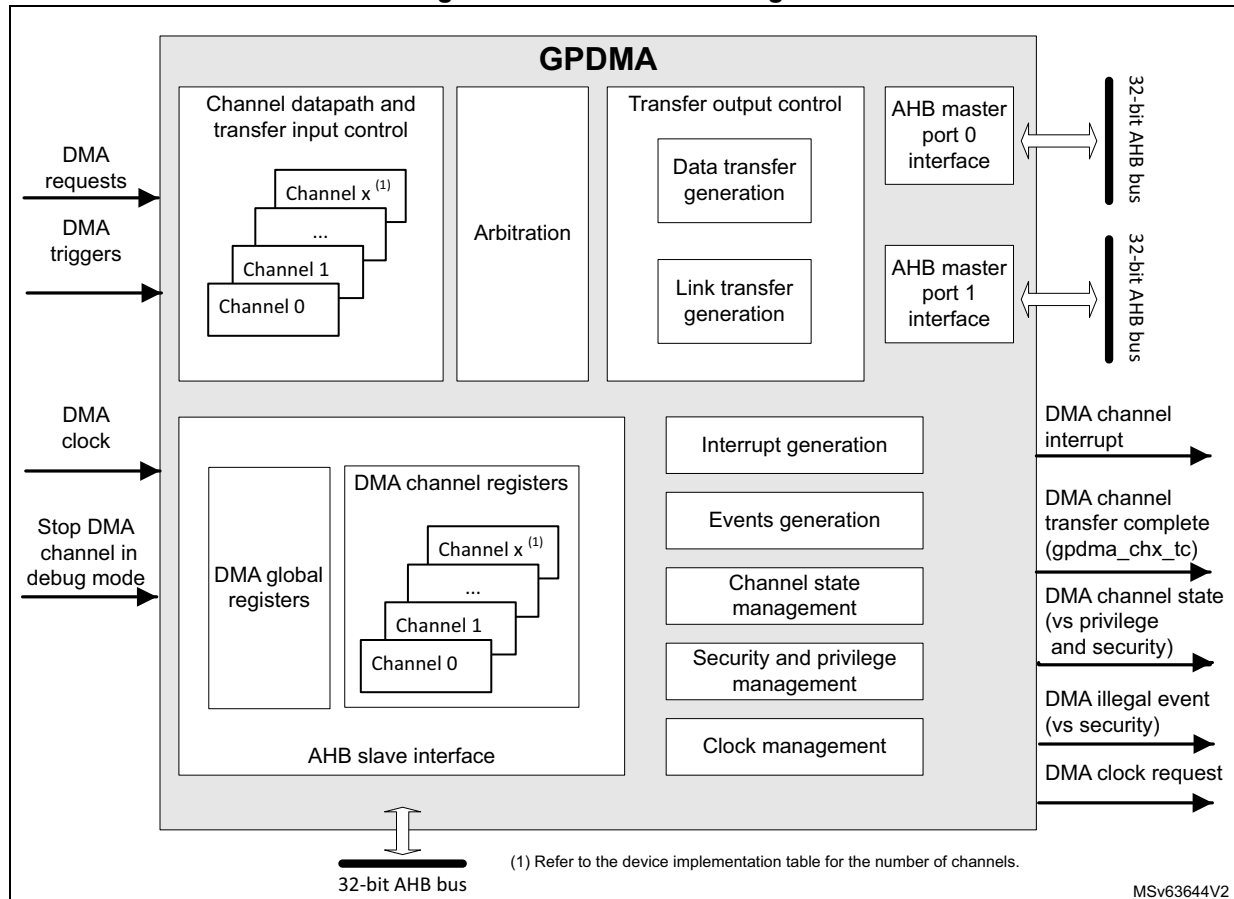
1. Not available in STM32H523/533 devices.

2. Not available in STM32H562/563/573 devices.

16.4 GPDMA functional description

16.4.1 GPDMA block diagram

Figure 70. GPDMA block diagram



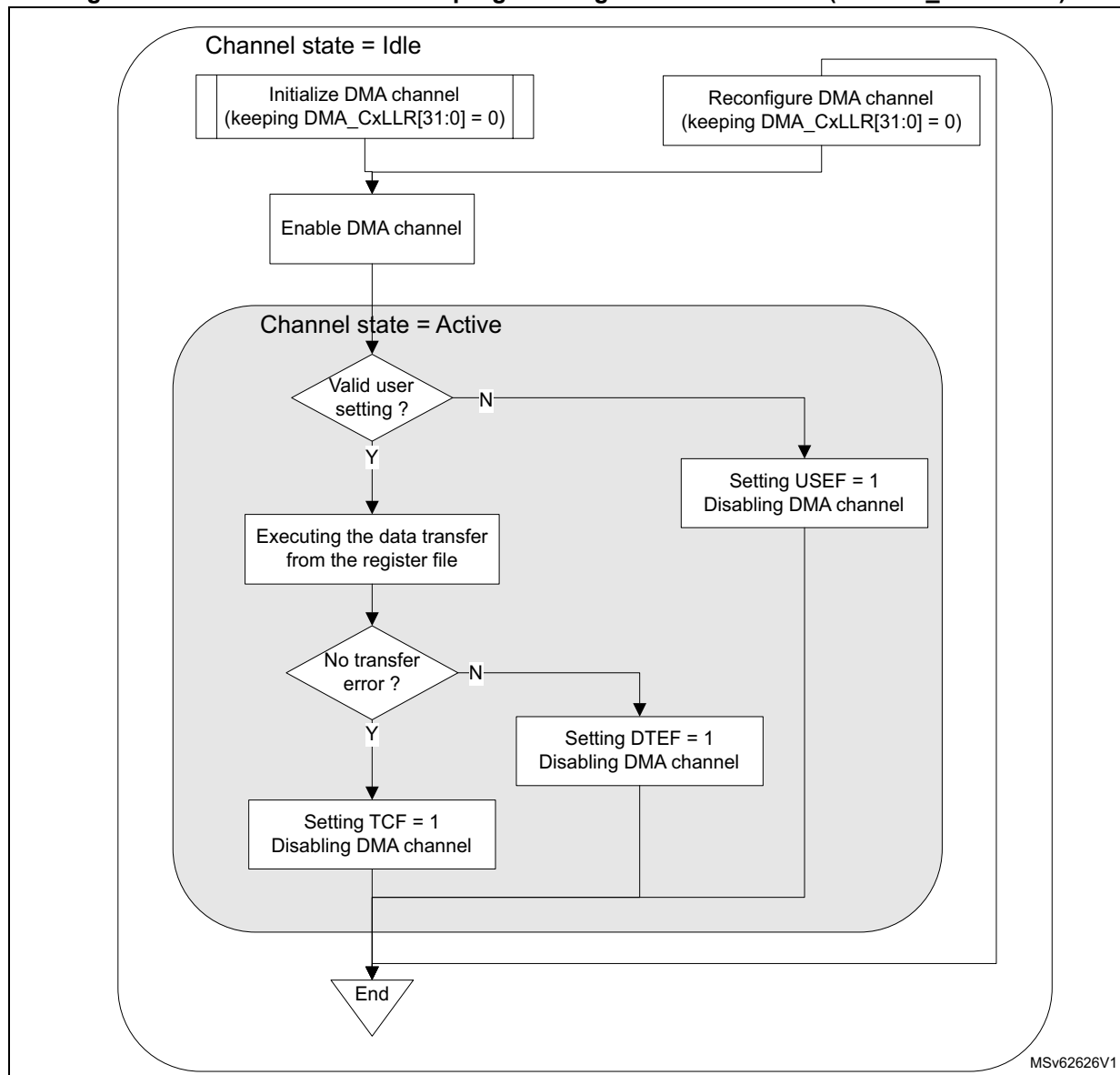
16.4.2 GPDMA channel state and direct programming without any linked-list

After a GPDMA reset, a GPDMA channel x is in idle state. When the software writes 1 in `GPDMA_CxCR.EN`, the channel takes into account the value of the different channel configuration registers (`GPDMA_CxXXX`), switches to the active/non-idle state and starts to execute the corresponding requested data transfers.

After enabling/starting a GPDMA channel transfer by writing 1 in `GPDMA_CxCR.EN`, a GPDMA channel interrupt on a complete transfer notifies the software that the GPDMA channel is back in idle state (`EN` is then deasserted by hardware) and that the channel is ready to be reconfigured then enabled again.

[Figure 71](#) illustrates this GPDMA direct programming without any linked-list (`GPDMA_CxLLR = 0`).

Figure 71. GPDMA channel direct programming without linked-list (GPDMA_CxLLR = 0)



16.4.3 GPDMA channel suspend and resume

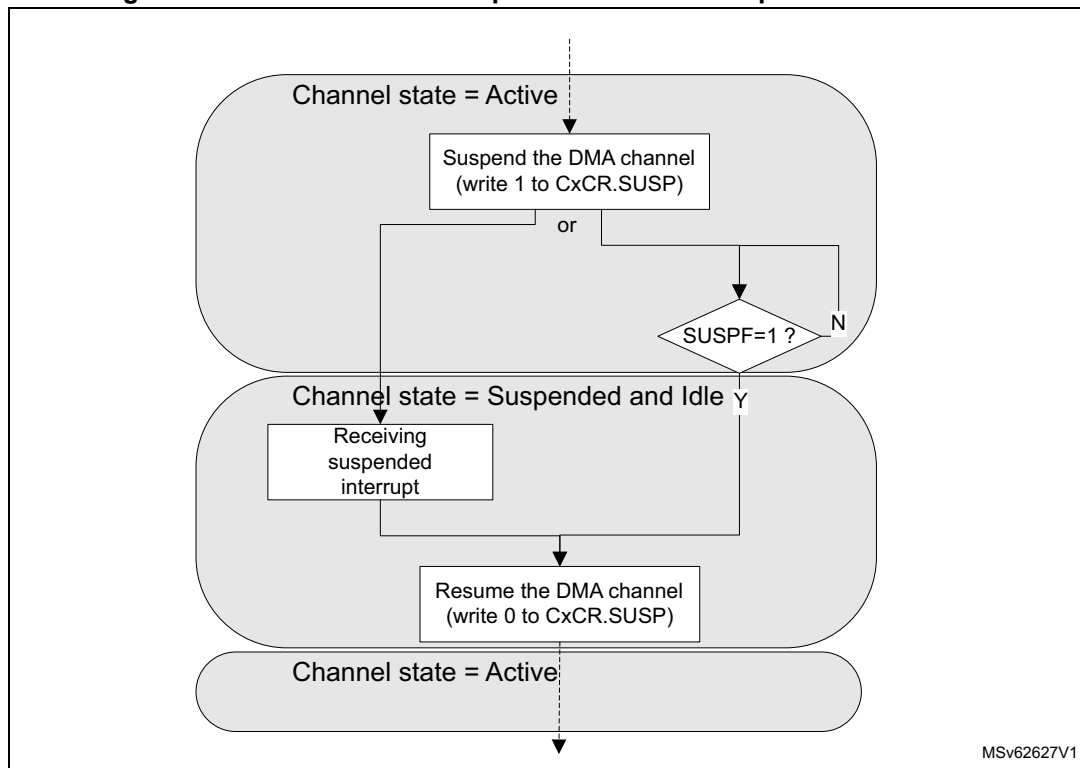
The software can suspend on its own a channel still active, with the following sequence:

1. The software writes 1 into the GPDMA_CxCR.SUSP bit.
2. The software polls the suspended flag GPDMA_CxSR.SUSPF until SUSPF = 1, or waits for an interrupt previously enabled by writing 1 to GPDMA_CxCR.SUSPIE. Wait for the channel to be effectively in suspended state means wait for the completion of any ongoing GPDMA transfer over its master ports. Then the software can observe, in a steady state, any read register or register field that is hardware modifiable.

Note: An ongoing GPDMA transfer can be a data transfer (a source/destination burst transfer) or a link transfer for the internal update of the linked-list register file from the next linked-list item.

3. The software safely resumes the suspended channel by writing 0 to GPDMA_CxCR.SUSP.

Figure 72. GPDMA channel suspend and resume sequence



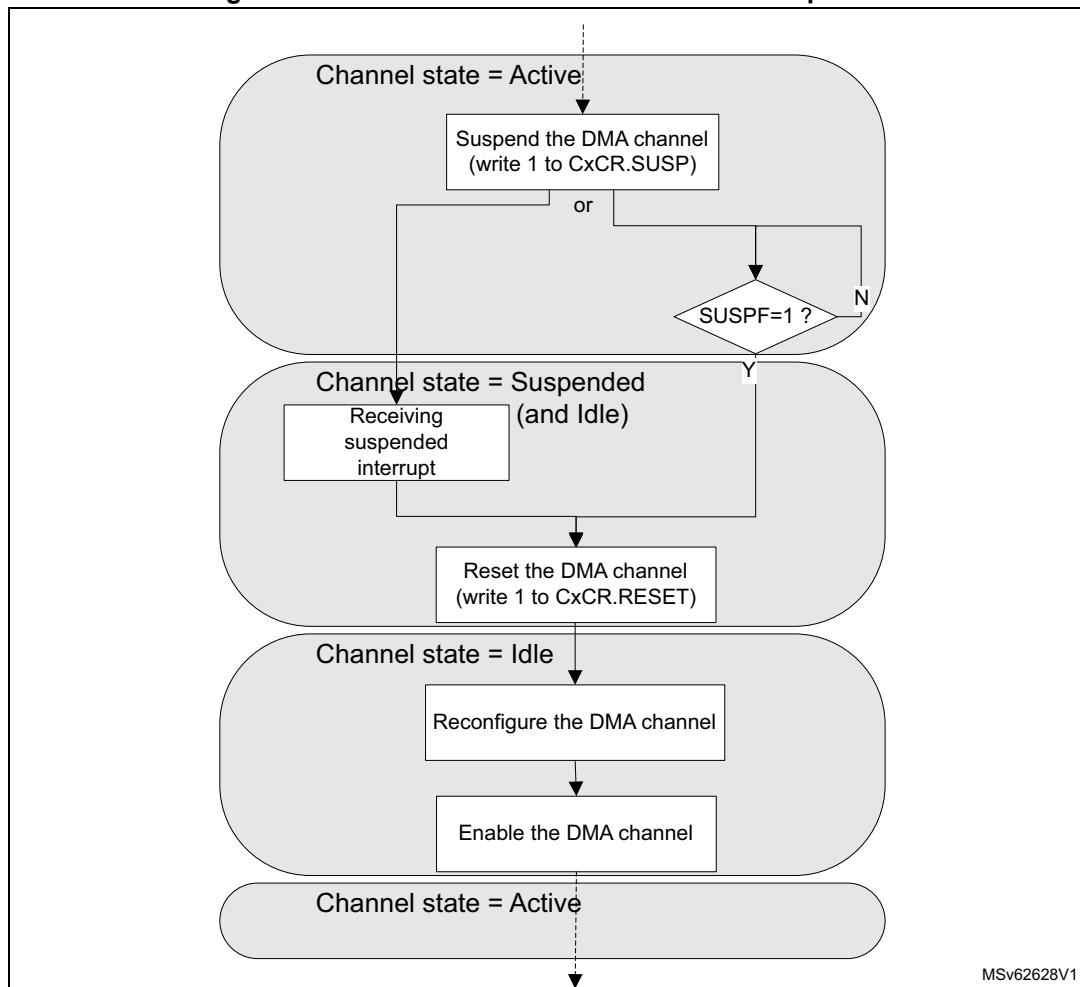
Note: A suspend and resume sequence does not impact the GPDMA_CxCR.EN bit. Suspending a channel (transfer) does not suspend a started trigger detection.

16.4.4 GPDMA channel abort and restart

Alternatively, like for aborting a continuous GPDMA transfer with a circular buffering or a double buffering, the software can abort, on its own, a still active channel with the following sequence:

1. The software writes 1 into the GPDMA_CxCR.SUSP bit.
2. The software polls suspended flag GPDMA_CxSR.SUSPF until SUSPF = 1, or waits for an interrupt previously enabled by writing 1 to GPDMA_CxCR.SUSPIE. Wait for the channel to be effectively in suspended state means wait for the completion of any ongoing GPDMA transfer over its master port.
3. The software resets the channel by writing 1 to GPDMA_CxCR.RESET. This causes the reset of the FIFO, the reset of the channel internal state, the reset of the GPDMA_CxCR.EN bit, and the reset of the GPDMA_CxCR.SUSP bit.
4. The software safely reconfigures the channel. The software must reprogram the hardware-modified GPDMA_CxBR1, GPDMA_CxSAR, and GPDMA_CxDAR registers.
5. In order to restart the aborted then reprogrammed channel, the software enables it again by writing 1 to the GPDMA_CxCR.EN bit.

Figure 73. GPDMA channel abort and restart sequence



16.4.5 GPDMA linked-list data structure

Alternatively to the direct programming mode, a channel can be programmed by a list of transfers, known as a list of linked-list items (LLI). Each LLI is defined by its data structure.

The base address in memory of the data structure of a next LLI_{n+1} of a channel x is the sum of the following:

- the link base address of the channel x (in GPDMA_CxLBAR)
- the link address offset (LA[15:2] field in GPDMA_CxLLR). The linked-list register GPDMA_CxLLR is the updated result from the data structure of the previous LLI of the channel x .

The data structure for each LLI may be specific.

A linked-list data structure is addressed following the value of the UT1, UT2, UB1, USA, UDA and ULL bits, plus UB2 and UT3, in GPDMA_CxLLR.

In linked-list mode, each GPDMA linked-list register (GPDMA_CxTR1, GPDMA_CxTR2, GPDMA_CxBR1, GPDMA_CxSAR, GPDMA_CxDAR or GPDMA_CxLLR, plus GPDMA_CxTR3 or GPDMA_CxBR2) is conditionally and automatically updated from the next linked-list data structure in the memory, following the current value of the

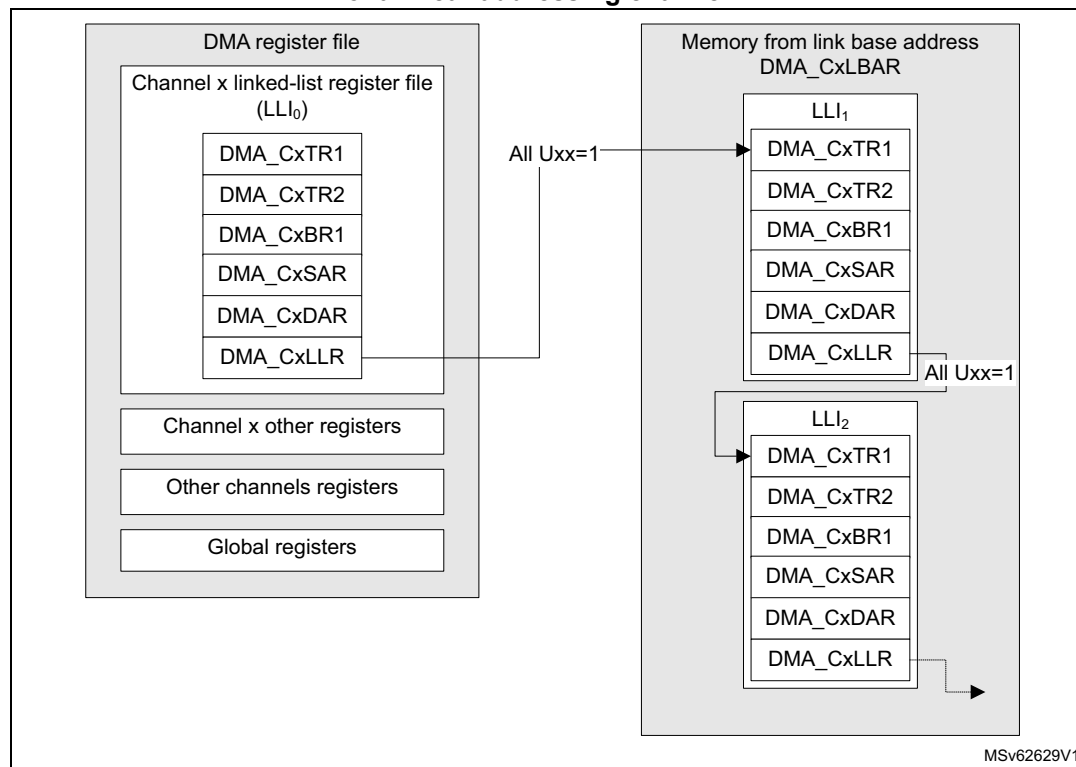
GPDMA_CxLLR register that was conditionally updated from the linked-list data structure of the previous LLI.

Static linked-list data structure

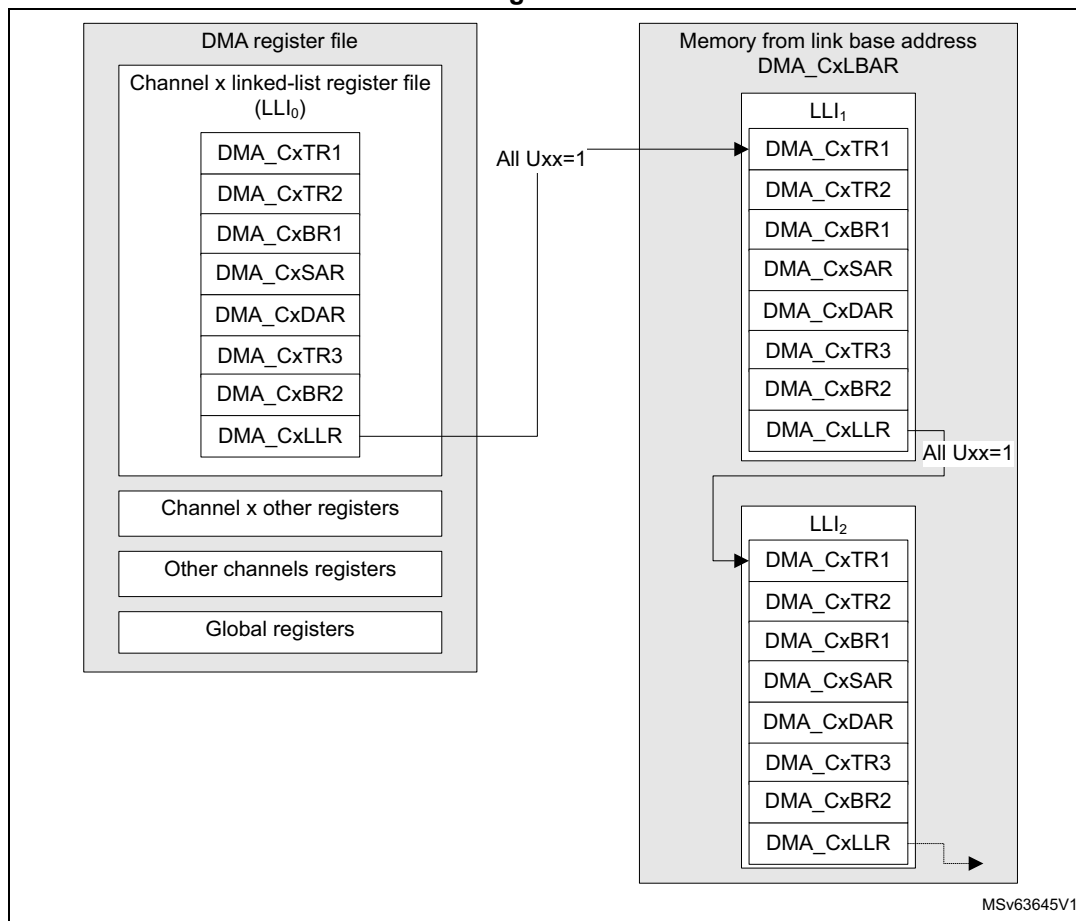
For example, when the update bits (UT1, UT2, UB1, USA, UDA and ULL, plus UB2 and UT3) in GPDMA_CxLLR are all asserted, the linked-list data structure in the memory is maximal with:

- channel x (x = 0 to 5) contiguous 32-bit locations, including GPDMA_CxTR1, GPDMA_CxTR2, GPDMA_CxBR1, GPDMA_CxSAR, GPDMA_CxDAR and GPDMA_CxLLR (see [Figure 74](#)) and including the first linked-list register file (LLI₀) and the next LLIs (such as LLI₁, LLI₂) in the memory
- channel x (x = 6 to 7), contiguous 32-bit locations, including GPDMA_CxTR1, GPDMA_CxTR2, GPDMA_CxBR1, GPDMA_CxSAR, GPDMA_CxDAR, and GPDMA_CxLLR, plus GPDMA_CxTR3 and GPDMA_CxBR2 (see [Figure 75](#)), and including the first linked-list register file (LLI₀) and the next LLIs (such as LLI₁, LLI₂) in the memory

**Figure 74. Static linked-list data structure (all Uxx = 1)
of a linear addressing channel x**



**Figure 75. Static linked-list data structure (all Uxx = 1)
of a 2D addressing channel x**



Dynamic linked-list data structure

Alternatively, the memory organization for the full list of LLIs can be compacted with specific data structure for each LLI.

If UT1 = 0 and UT2 = 1, the link address offset of the register GPDMA_CxLLR is pointing to the updated value of the GPDMA_CxTR2 instead of the GPDMA_CxTR1 which is not to be modified (see [Figure 76](#)).

Example: if UT1 = UB1 = USA = 0 and if UT3 = UB2 = 0, when channel x is with 2D addressing, and if UT2 = UDA = ULL = 1, the next LLI does not contain an (updated) value for GPDMA_CxTR1, nor GPDMA_CxBR1, nor GPDMA_CxSAR, nor GPDMA_CxTR3, nor GPDMA_CxBR2 when channel x is with 2D addressing. The next LLI contains an updated value for GPDMA_CxTR2, GPDMA_CxDAR, and GPDMA_CxLLR, as shown in [Figure 77](#).

Figure 76. GPDMA dynamic linked-list data structure of a linear addressing channel x

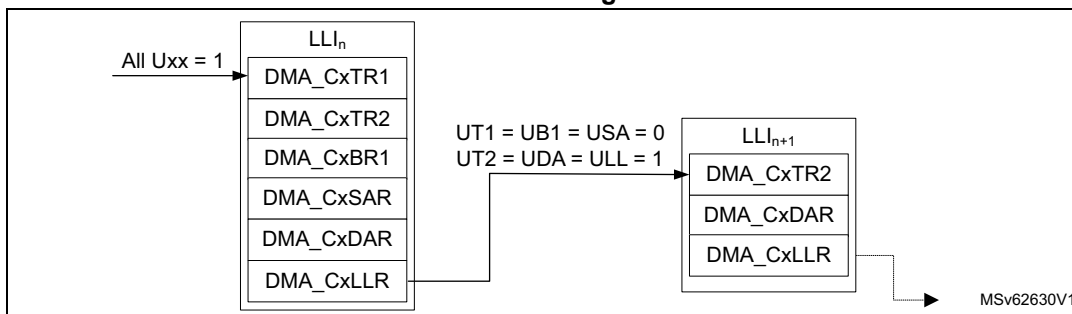
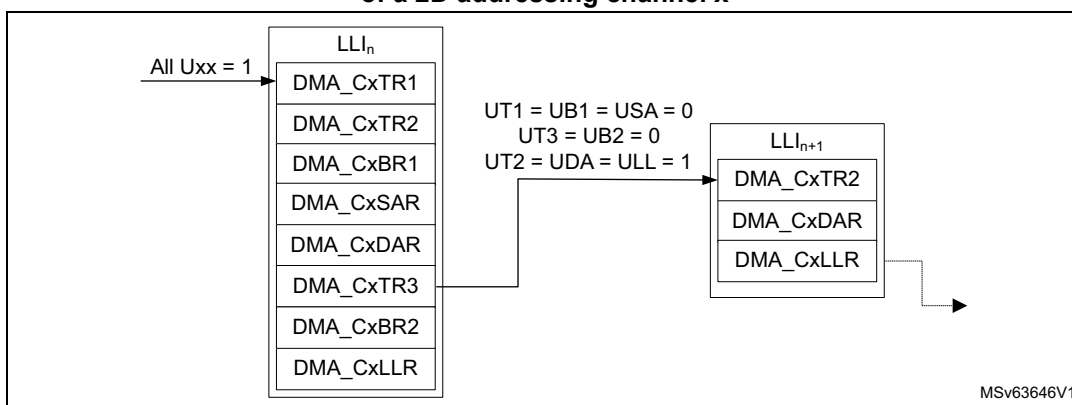


Figure 77. GPDMA dynamic linked-list data structure of a 2D addressing channel x



The user must program GPDMA_CxLLR for each LLI_n to be 32-bit aligned and not to exceed the 64-Kbyte addressable space pointed by GPDMA_CxLBAR.

16.4.6 Linked-list item transfer execution

A LLI_n transfer is the sequence of:

1. a data transfer: GPDMA executes the data transfer as described by the GPDMA internal register file (this data transfer can be void/null for LLI₀)
2. a conditional link transfer: GPDMA automatically and conditionally updates its internal register file by the data structure of the next LLI_{n+1}, as defined by the GPDMA_CxLLR value of the LLI_n.

Note: *The initial data transfer as defined by the internal register file (LLI₀) can be null (GPDMA_CxBR1.BNDT[15:0] = 0 and GPDMA_CxTR2.PFREQ = 0) provided that the conditional update bit UB1 in GPDMA_CxLLR is set (meaning there is a non-null data transfer described by the next LLI₁ in the memory to be executed).*

Depending on the intended GPDMA usage, a GPDMA channel x can be executed as described by the full linked-list (run-to-completion mode, GPDMA_CxCR.LSM = 0) or a GPDMA channel x can be programmed for a single execution of a LLI (link step mode, GPDMA_CxCR.LSM = 1), as described in the next sections.

16.4.7 GPDMA channel state and linked-list programming in run-to-completion mode

When GPDMA_CxCR.LSM = 0 (in full list execution mode, execution of the full sequence of LLIs, named run-to-completion mode), a GPDMA channel x is initially programmed, started by writing 1 to GPDMA_CxCR.EN, and after completed at channel level. The channel transfer is:

- configured with at least the following:
 - the first LLI₀, internal linked-list register file: GPDMA_CxTR1, GPDMA_CxTR2, GPDMA_CxBR1, GPDMA_CxSAR, GPDMA_CxDAR, and GPDMA_CxLLR, plus GPDMA_CxTR3 and GPDMA_CxBR2
 - the last LLI_N, described by the linked-list data structure in memory, as defined by the GPDMA_CxLLR reflecting the before last LLI_{N-1}
- completed when GPDMA_CxLLR[31:0] = 0, GPDMA_CxBR1.BRC[10:0] = 0, and GPDMA_CxBR1.BNDT[15:0] = 0, at the end of the last LLI_{N-1} transfer

GPDMA_CxLLR[31:0] = 0 is the condition of a linked-list based channel completion and means the following:

- The 16 low significant bits GPDMA_CxLLR.LA[15:0] of the next link address are null.
- All the update bits GPDMA_CxLLR.Uxx are null (UT1, UT2, UB1, USA, UDA and ULL, plus UB2 and UT3).

The channel may never be completed when GPDMA_CxLLR.LSM = 0:

- If the last LLI_N is recursive, pointing to itself as a next LLI:
 - either GPDMA_CxLLR.ULL = 1 and GPDMA_CxLLR.LA[15:2] is updated by the same value
 - or GPDMA_CxLLR.ULL = 0
- If LLI_N is pointing to a previous LLI

In the regular data transfer completion at a block level, GPDMA_CxBR1.BNDT[15:0] = 0 and GPDMA_CxBR1.BRC[10:0] = 0 (if present). Alternatively, a block transfer may be early completed by a peripheral (such as an I3C in Rx mode), and then BNDT[15:0] is not null (see [Section 16.4.14](#) for more details).

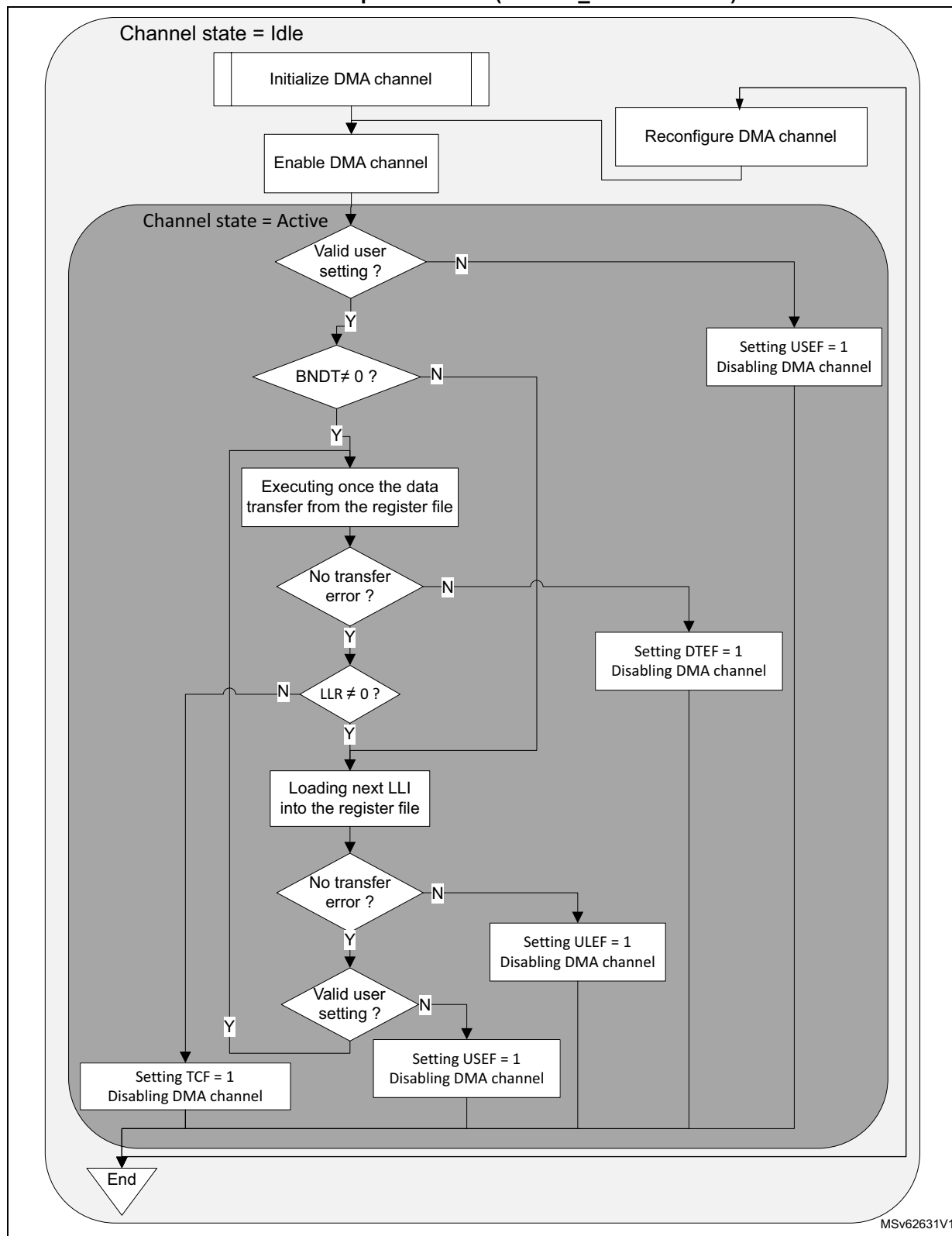
In the typical run-to-completion mode, the allocation of a GPDMA channel, including its fine programming, is done once during the GPDMA initialization. In order to have a reserved data communication link and GPDMA service during run-time, for continuously repeated transfers (from/to a peripheral respectively to/from memory or for memory-to-memory transfers). This reserved data communication link can consist of a channel, or the channel can be shared and a repeated transfer consists of a sequence of LLIs.

[Figure 78](#) depicts the GPDMA channel execution and its registers programming in run-to-completion mode.

Note: [Figure 78](#) is not intended to illustrate how often a TCEF can be raised, depending on the programmed value of TCEM[1:0] in GPDMA_CxTR2. It can be raised at (each) block completion, at (each) 2D block completion, at (each) LLI completion, or only at channel completion. In run-to-completion mode, whatever is the value of TCEM[1:0], at the channel completion, the hardware always set TCEF = 1 and disables the channel.

In [Figure 78](#), BNDT ≠ 0 is the typical condition for starting the first data transfer in this figure. This condition becomes (BNDT ≠ 0 and PFREQ = 1) if the peripheral requests a data transfer with early termination (see [Section 16.3.6](#)).

Figure 78. GPDMA channel execution and linked-list programming in run-to-completion mode (GPDMA_CxCR.LSM = 0)



Run-time inserting a LLI_n via an auxiliary channel, in run-to-completion mode

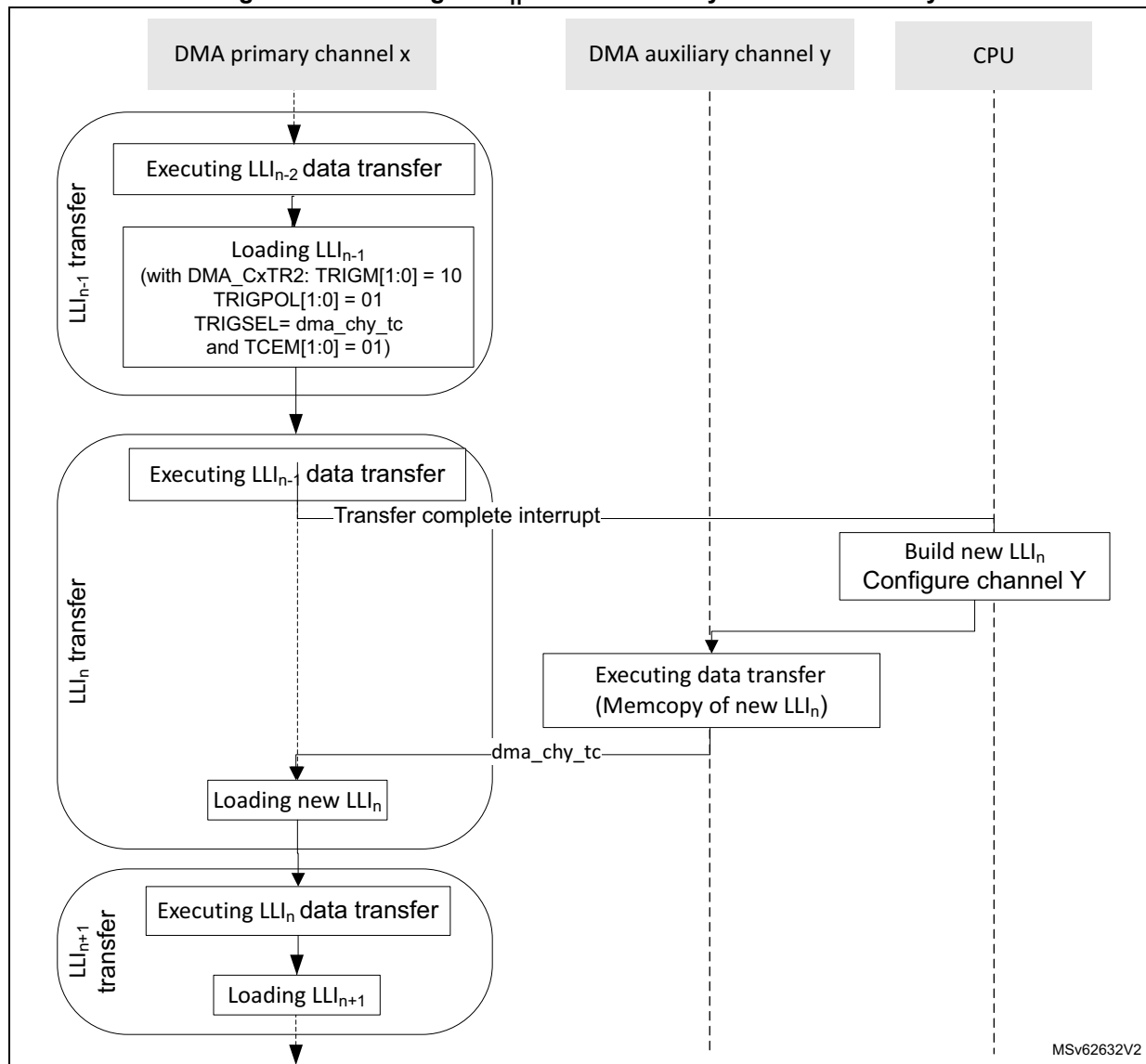
The start of the link transfer of the LLI_{n-1} (start of the LLI_n loading) can be conditioned by the occurrence of a trigger, when programming the following fields of the GPDMA_CxTR2 in the data structure of the LLI_{n-1} :

- TRIGM[1:0] = 10 (link transfer triggering mode)
- TRIGPOL[1:0] = 01 or 10 (rising or falling edge)
- TRIGSEL[5:0] (see [Section 16.3.7](#) for the trigger selection details)

Another auxiliary channel y can be used to store the channel x LLI_n in the memory and to generate a transfer complete event gpdma_chy_tc. By selecting this event as the input trigger of the link transfer of the LLI_{n-1} of the channel x, the software can pause the primary channel x after its LLI_{n-1} data transfer, until it is indeed written the LLI_n .

[Figure 79](#) depicts such a dynamic elaboration of a linked-list of a primary channel x, via another auxiliary channel y.

Caution: This use case is restricted to an application with a LLI_{n-1} data transfer that does not need a trigger. The triggering mode of this LLI_{n-1} is used to load the next LLI_n .

Figure 79. Inserting a LLI_n with an auxiliary GPDMA channel y

MSv62632V2

16.4.8 GPDMA channel state and linked-list programming in link step mode

When $GPDMA_CxCR.LSM = 1$ (in link step execution mode, single execution of one LLI), a channel transfer is executed and completed after each single execution of a LLI, including its (conditional) data transfer and its (conditional) link transfer.

A GPDMA channel transfer can be programmed at LLI level, started by writing 1 into $GPDMA_CxCR.EN$, and after completed at LLI level:

- The current LLI_n transfer is described with:
 - $GPDMA_CxTR1$ defines the source/destination elementary single/burst transfers.
 - $GPDMA_CxBR1$ defines the number of bytes at a block level ($BNDT[15:0]$) and, for channel x ($x = 6$ to 7), the number of blocks at a 2D/repeated block level ($BRC[10:0]+1$) and the incrementing/decrementing mode for address offsets.

- GPDMA_CxTR2 defines the input control (request, trigger) and the output control (transfer complete event) of the transfer.
- GPDMA_CxSAR/GPDMA_CxDAR define the source/destination transfer start address.
- GPDMA_CxTR3 for channel x (x = 6 to 7) defines the source/destination additional address offset between burst transfers.
- GPDMA_CxBR2 for channel x (x = 6 to 7) defines the source/destination additional address offset between blocks at a 2D/repeated block level.
- GPDMA_CxLLR defines the data structure and the address offset of the next LLI_{n+1} in the memory.
- The current LLI_n transfer is completed after the single execution of the current LLI_n:
 - after the (conditional) data transfer completion (when GPDMA_CxBR1.BRC[10:0] = 0, and GPDMA_CxBR1.BNDT[15:0] = 0
 - after the (conditional) update of the GPDMA link register file from the data structure of the next LLI_{n+1} in memory

Note: *If a LLI is recursive (pointing to itself as a next LLI, either GPDMA_CxLLR.ULL = 1 and GPDMA_CxLLR.LA[15:2] is updated by the same value, or GPDMA_CxLLR.ULL = 0), a channel in link step mode is completed after each repeated single execution of this LLI.*

In the regular data transfer completion at a block level, GPDMA_CxBR1.BNDT[15:0] = 0 and GPDMA_CxBR1.BRC[10:0] = 0. Alternatively, a block transfer may be early completed by a peripheral (such as an I3C in Rx mode), and then BNDT[15:0] is not null (see [Section 16.4.14](#) for more details).

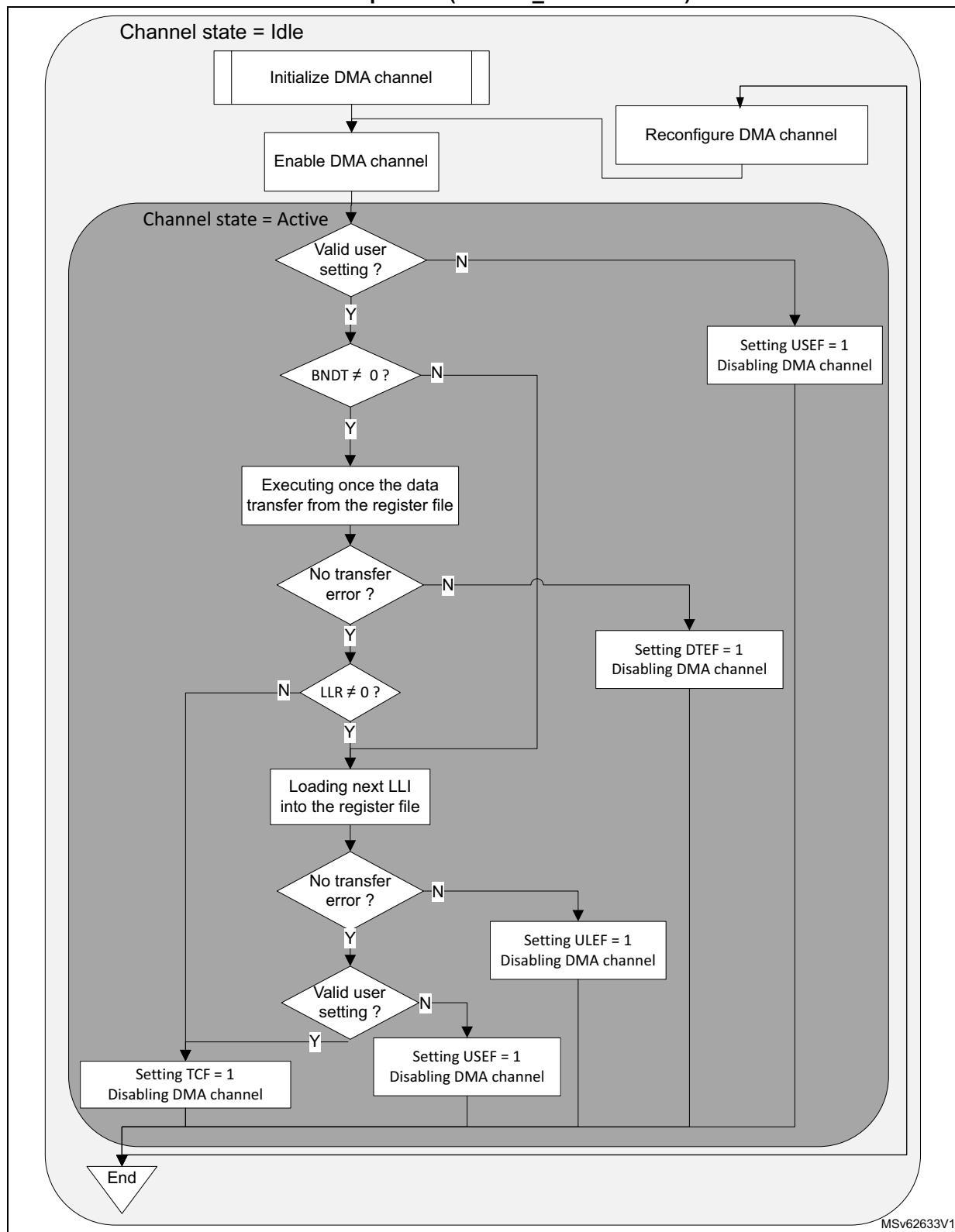
The link step mode can be used to elaborate dynamically LLIs in memory during run-time. The software can be facilitated by using a static data structure for any LLI_n (all update bits of GPDMA_CxLLR have a static value, LLI_n.LLR.LA = LLI_{n-1}.LLR.LA + constant).

[Figure 80](#) depicts the GPDMA channel execution mode, and its programming in link step mode.

Note: *[Figure 80](#) is not intended to illustrate how often a TCEF can be raised, depending on the programmed value of TCEM[1:0] in GPDMA_CxTR2. It can be raised at (each) block completion, at (each) 2D block completion, at (each) LLI completion, or only at the last LLI data transfer completion. In link step mode, the channel is disabled after each single execution of a LLI, and depending on the value of TCEM[1:0] a TCEF is raised or not.*

In [Figure 80](#), BNDT ≠ 0 is the typical condition for starting the first data transfer. This condition becomes (BNDT ≠ 0 and PFREQ = 1) if the peripheral requests a data transfer with early termination (see [Section 16.3.6](#)).

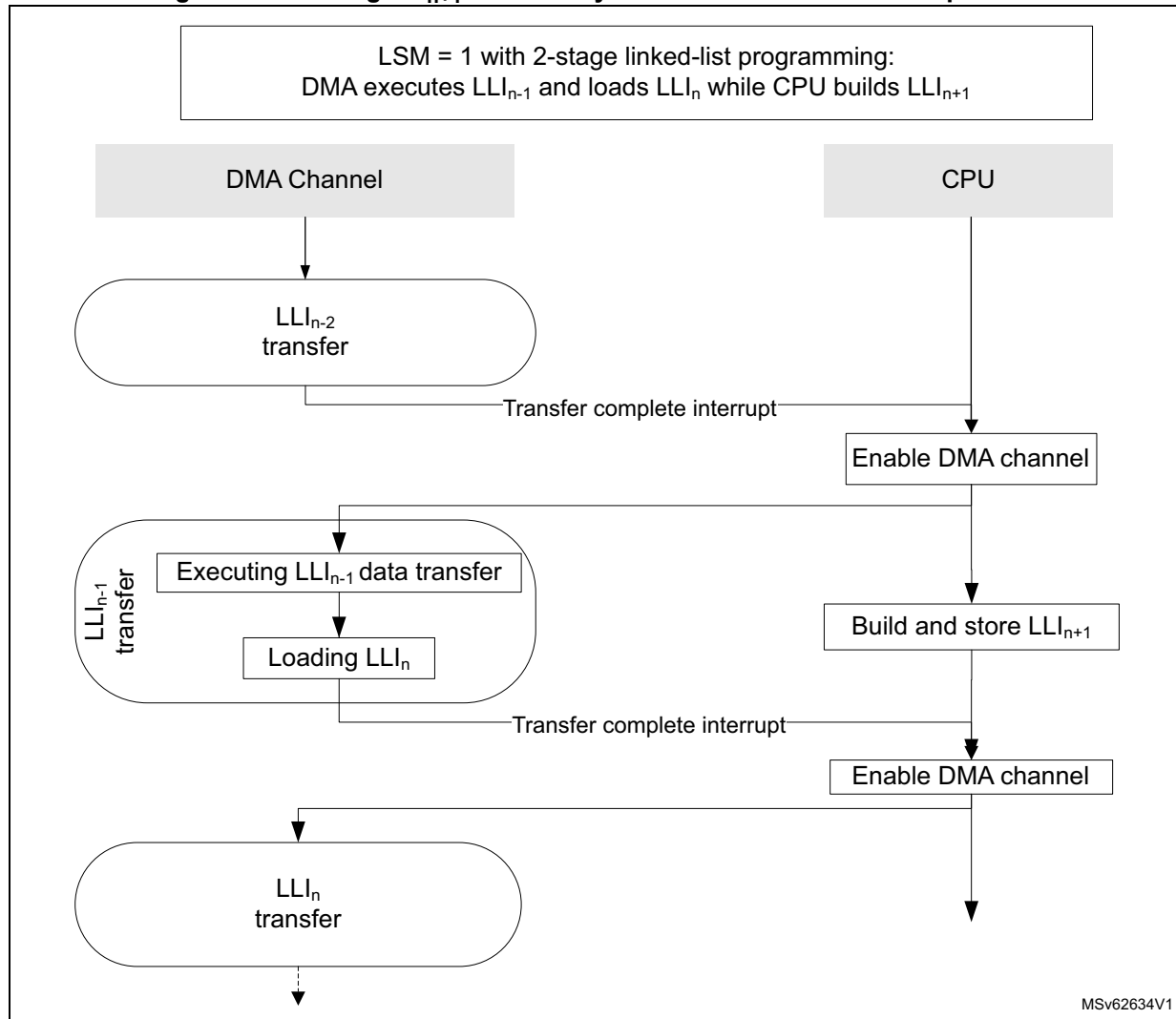
Figure 80. GPDMA channel execution and linked-list programming in link step mode (GPDMA_CxCR.LSM = 1)



Run-time adding a LLI_{n+1} in link step mode

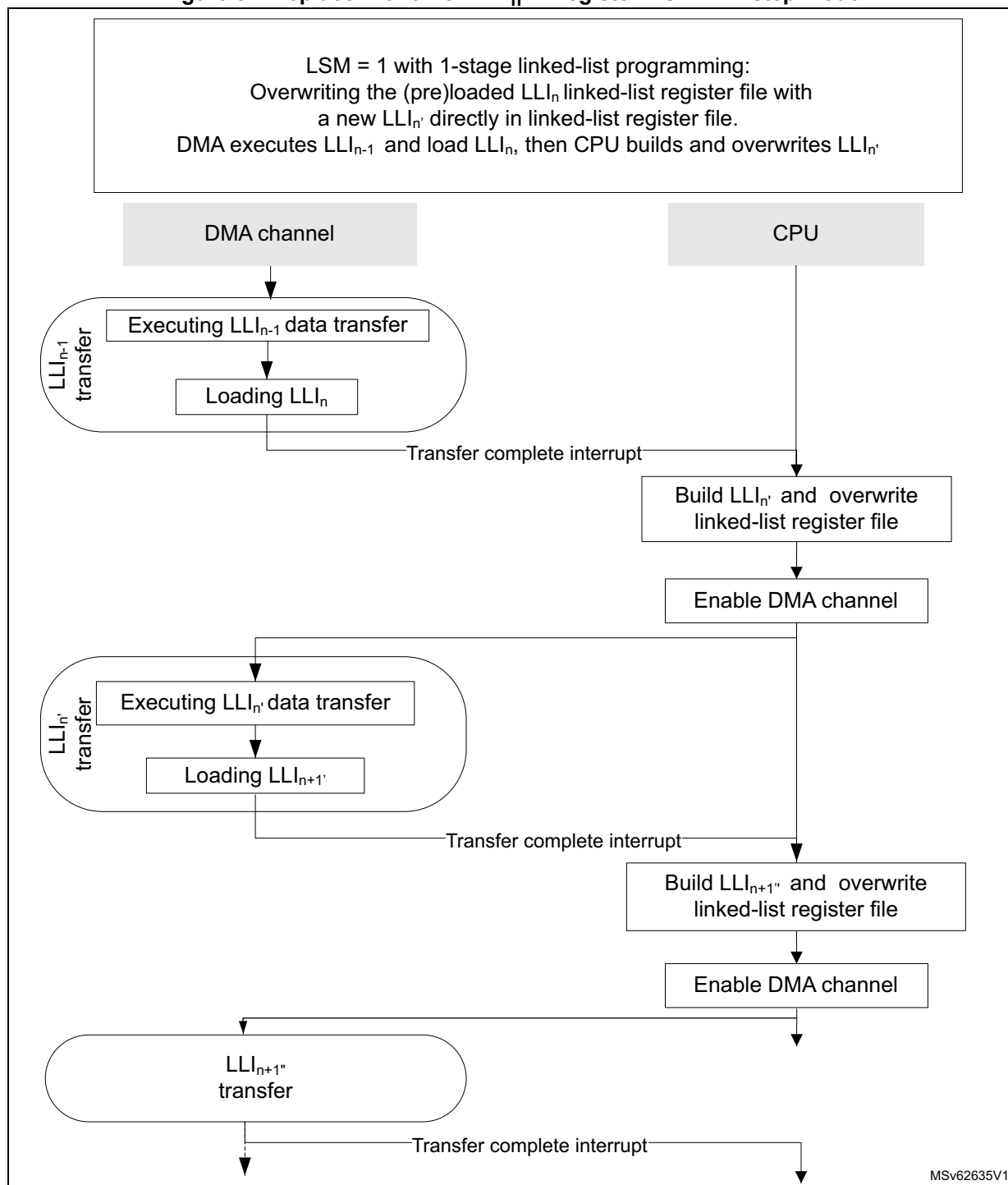
During run-time, the software can defer the elaboration of the LLI_{n+1} (and next LLIs), until/after GPDMA executed the transfer from the LLI_{n-1} and loaded the LLI_n from the memory, as shown in [Figure 81](#).

Figure 81. Building LLI_{n+1} : GPDMA dynamic linked-lists in link step mode

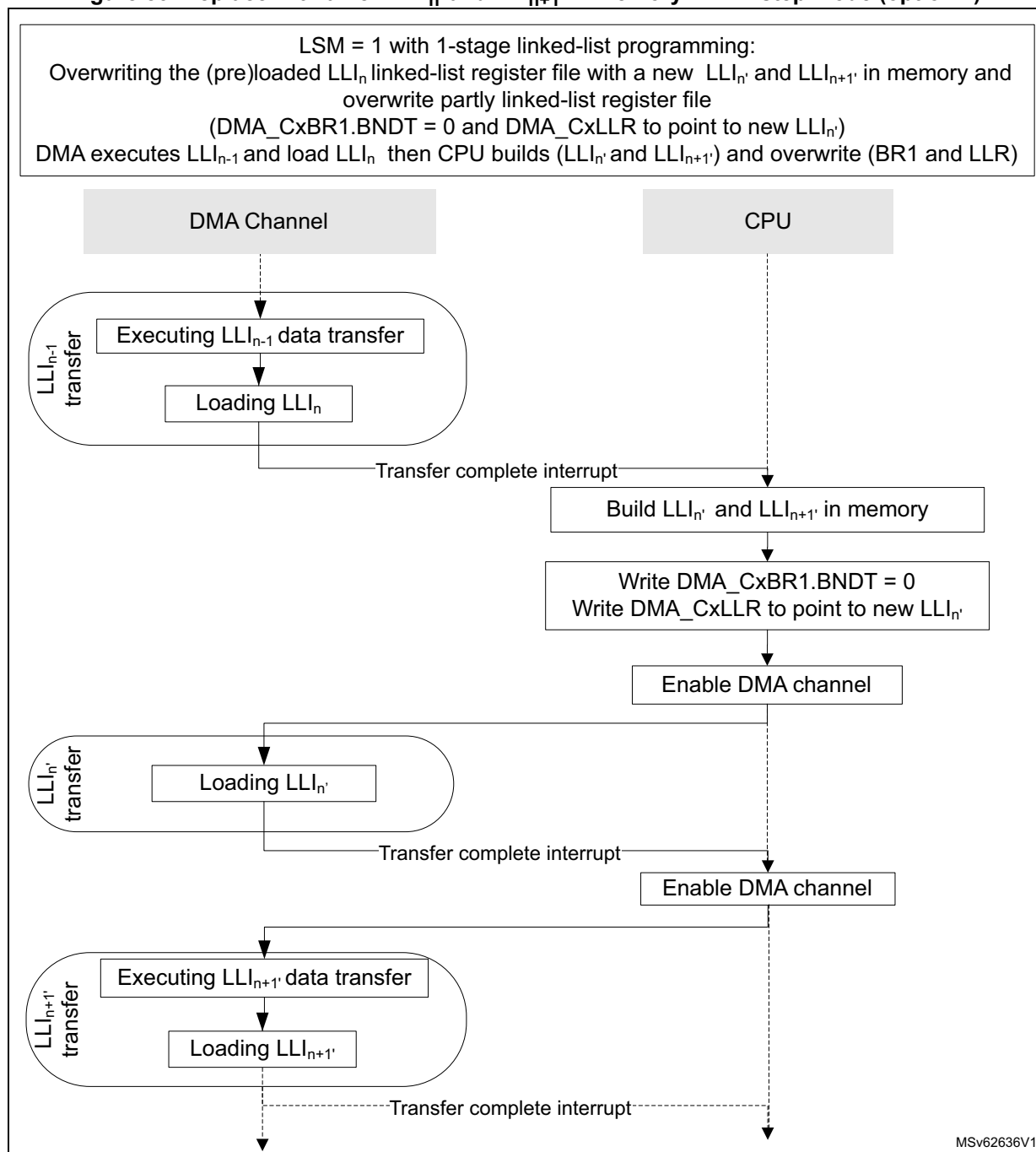


Run-time replacing a LLI_n with a new LLI_n' in link step mode (in linked-list register file)

In this link step mode, during run-time, the software can build and insert a new LLI_n' , after GPDMA executed the transfer from the LLI_{n-1} and loaded a formerly elaborated LLI_n from the memory by overwriting directly the linked-list register file with the new LLI_n' , as shown in [Figure 82](#).

Figure 82. Replace with a new LLI_n' in register file in link step mode**Run-time replacing a LLI_n with a new LLI_n' in link step mode (in the memory)**

The software can build and insert a new LLI_n' and LLI_{n+1}' in the memory, after GPDMA executed the transfer from the LLI_{n-1} and loaded a formerly elaborated LLI_n from the memory, by overwriting partly the linked-list register file (GPDMA_CxBR1.BNDT[15:0] to be null and GPDMA_CxLLR to point to new LLI_n') as shown in [Figure 83](#).

Figure 83. Replace with a new LLI_n and LLI_{n+1} in memory in link step mode (option 1)

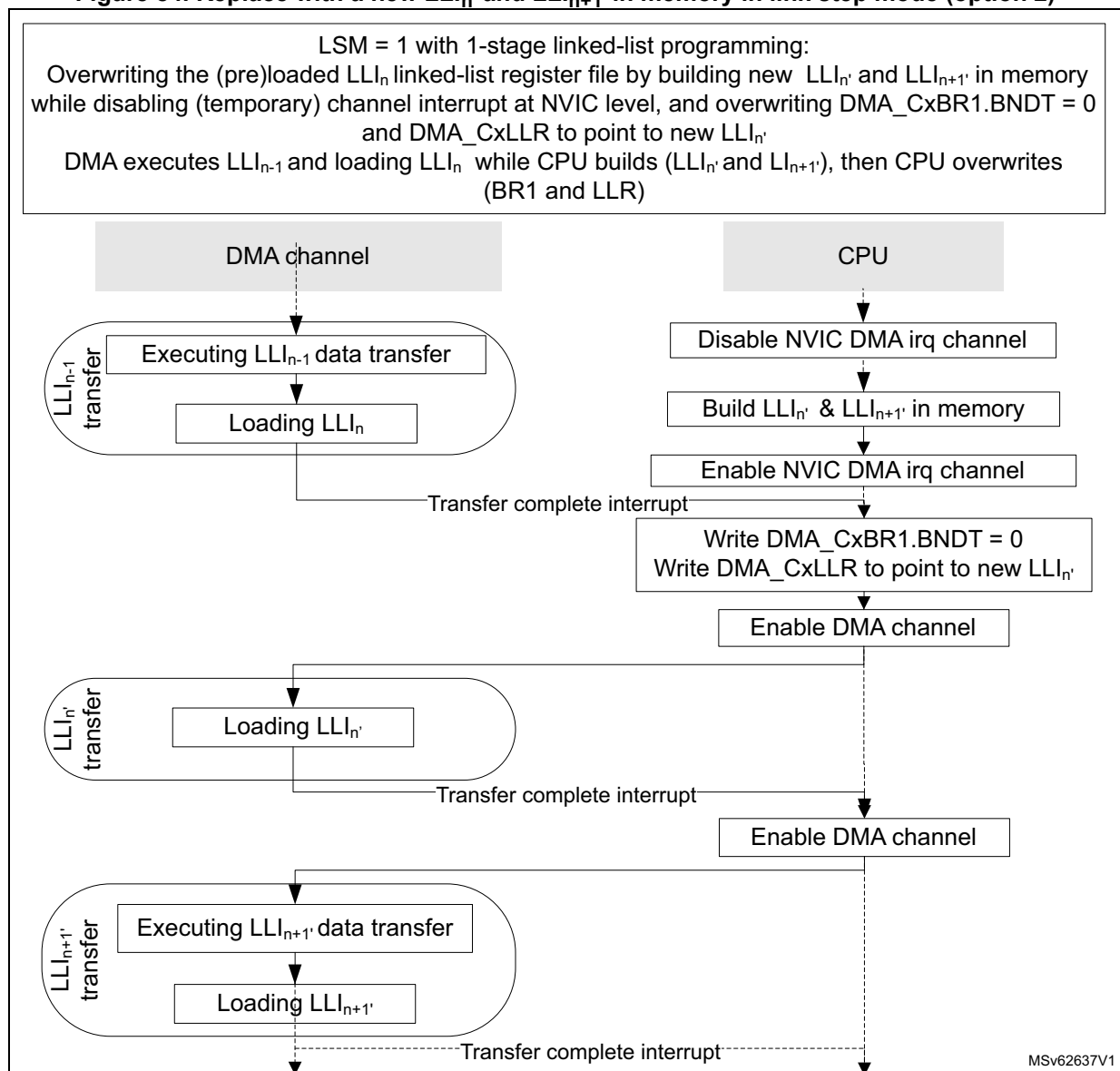
Run-time replacing a LLI_n with a new LLI_n in link step mode

Other software implementations exist. Meanwhile GPDMA executes the transfer from the LLI_{n-1} and loads a formerly elaborated LLI_n from the memory (or even earlier), the software can do the following:

1. Disable the NVIC for not being interrupted by the interrupt handling.
2. Build a new LLI_n and a new LLI_{n+1} .
3. Enable again the NVIC for the channel interrupt (transfer complete) notification.

The software in the interrupt handler for LLI_{n-1} is then restricted to overwrite $GPDMA_CxBR1.BNDT[15:0]$ to be null and $GPDMA_CxLLR$ to point to new LLI_n , as shown in [Figure 84](#).

Figure 84. Replace with a new LLI_n and LLI_{n+1} in memory in link step mode (option 2)



16.4.9 GPDMA channel state and linked-list programming

The software can reconfigure a channel when the channel is disabled (GPDMA_CxCR.EN = 0) and update the execution mode (GPDMA_CxCR.LSM) to change from/to run-to-completion mode to/from link step mode.

In any execution mode, the software can:

- reprogram LLI_{n+1} in the memory to finally complete the channel by this LLI_{n+1} (clear the GPDMA_CxLLR of this LLI_{n+1}), before that this LLI_{n+1} is loaded/used by the GPDMA channel
- abort and reconfigure the channel with a LSM update (see [Section 16.4.4](#).)

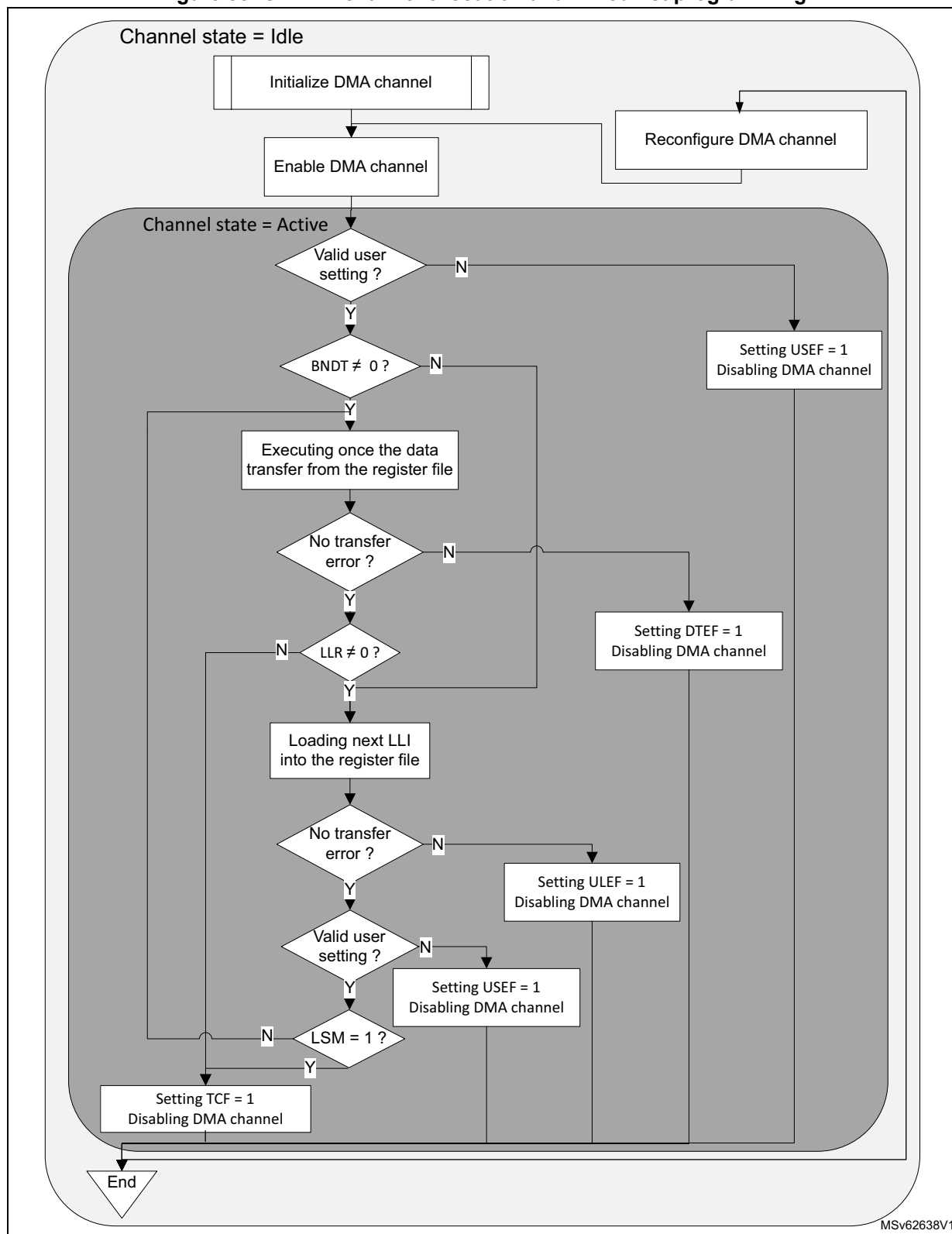
In link step mode, the software can clear LSM after each a single execution of any LLI, during LLI_{n-1} .

[Figure 85](#) shows the overall and unified GPDMA linked-list programming, whatever is the execution mode.

Note: [Figure 85](#) is not intended to illustrate how often a TCEF can be raised, depending on the programmed value of TCEM[1:0] in GPDMA_CxTR2. It can be raised at (each) block completion, at (each) 2D block completion, at (each) LLI completion, or only at the last LLI data transfer completion. In run-to-completion mode, whatever is the value of TCEM[1:0], at the channel completion the hardware always set TCEF = 1 and disables the channel. In link step mode, the channel is disabled after each single execution of a LLI, and depending on the value of TCEM[1:0] a TCEF is raised or not.

In [Figure 85](#), $BNDT \neq 0$ is the typical condition for starting the first data transfer. This condition becomes ($BNDT \neq 0$ and $PFREQ = 1$) if the peripheral requests a data transfer with early termination (see [Section 16.3.6](#)).

Figure 85. GPDMA channel execution and linked-list programming



16.4.10 GPDMA FIFO-based transfers

There is a single transfer operation mode: the FIFO mode. There are FIFO-based transfers. Any channel x is implemented with a dedicated FIFO whose size is defined by `dma_fifo_size[x]` (see [Section 16.3.2](#) for more details).

GPDMA burst

A programmed transfer at the lowest level is a GPDMA burst.

A GPDMA burst is a burst of data received from the source, or a burst of data sent to the destination. A source (and destination) burst is programmed with a burst length by the field `SBL_1[5:0]` (respectively `DBL_1[5:0]`), and with a data width defined by the field `SDW_LOG2[1:0]` (respectively `DDW_LOG2[1:0]`) in the `GPDMA_CxTR1` register.

The addressing mode after each data (named beat) of a GPDMA burst is defined by `SINC` and `DINC` in `GPDMA_CxTR1`, for source and destination respectively: either a fixed addressing or an incremented addressing with contiguous data.

The start and next addresses of a GPDMA source/destination burst (defined by `GPDMA_CxSAR` and `GPDMA_CxDAR`) must be aligned with the respective data width.

The table below lists the main characteristics of a GPDMA burst.

Table 141. Programmed GPDMA source/destination burst

SDW_LOG2[1:0] DDW_LOG2[1:0]	Data width (bytes)	SINC/DINC	SBL_1[5:0] DBL_1[5:0]	Burst length (data/beats)	Next data/ beat address	Next burst address	Burst address alignment
00	1	0 (fixed)	n = 0 to 63 ⁽¹⁾	n+1	+ 0	+ 0	1
01	2						2
10	4						4
00	1	1 (contiguously incremented)			+ 1	+ (n + 1)	1
01	2				+ 2	+ 2 * (n + 1)	2
10	4				+ 4	+ 4 * (n + 1)	4
11	forbidden user setting, causing USEF generation and none burst to be issued.						

1. When `S/DBL_1[5:0] = 0`, burst is of length 1. Then burst can be also named as single.

The next burst address in the above table is the next source/destination default address pointed by `GPDMA_CxSAR` or `GPDMA_CxDAR`, once the programmed source/destination burst is completed. This default value refers to the fixed/contiguously incremented address.

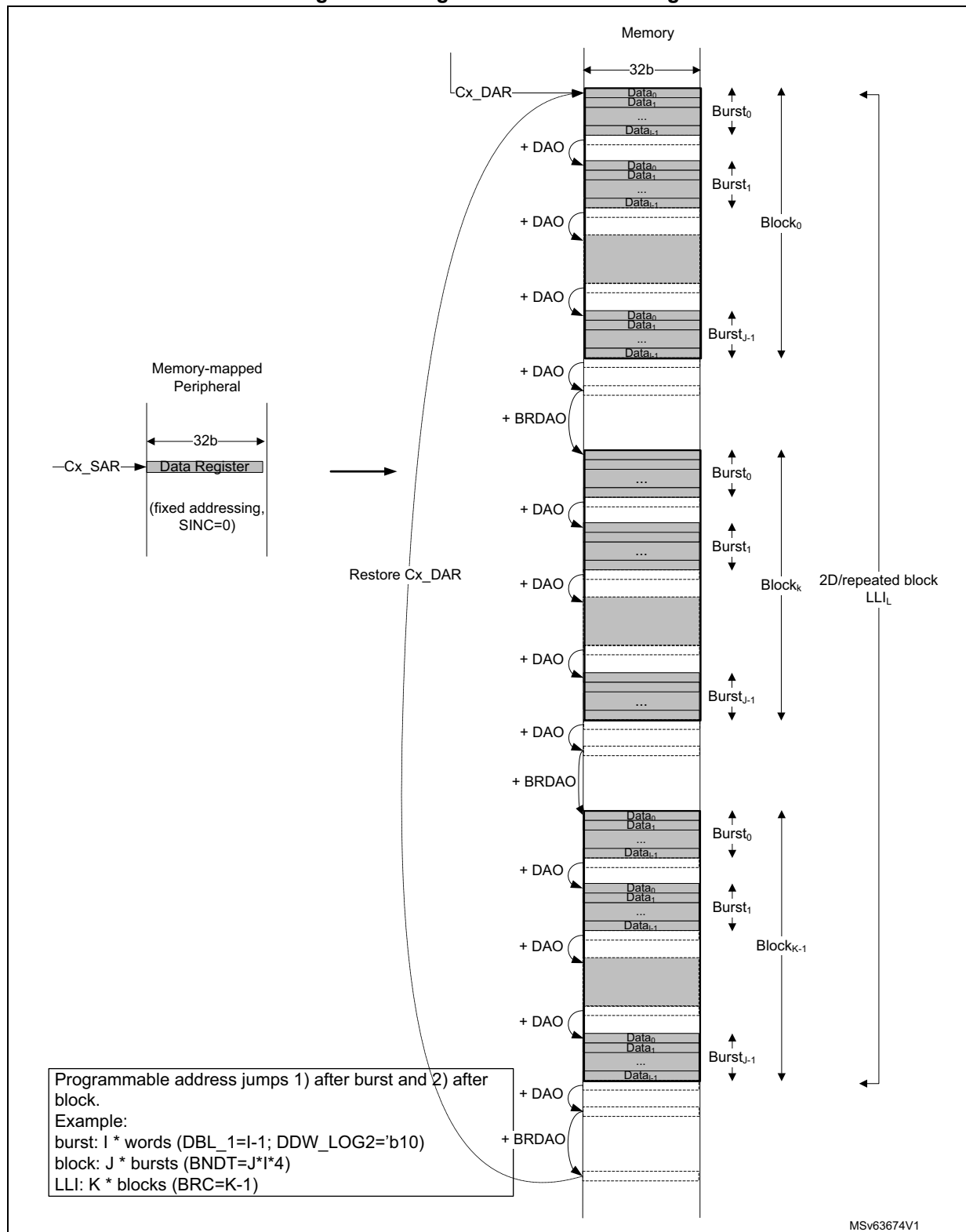
GPDMA burst with 2D addressing (channel x = 6 to 7)

When the channel has additional 2D addressing feature, this default value refers to the value without taking into account the two programmed incremented or decremented offsets. These two additional offsets (with a null default value) are applied:

- after each completed source/destination burst, as defined respectively by GPDMA_CxTR2.SAO[12:0]/DAO[12:0] and GPDMA_CxBR1.SDEC/DDEC
- after each completed block, as defined respectively by GPDMA_CxBR2.BRSAO[15:0]/BRDAO[15:0] and GPDMA_CxBR1.BRSDEC/BRDDEC)

Then, a 2D/repeated block can be addressed with a first programmed address jump after each completed burst, and with a second programmed address jump after each block, as depicted by [Figure 86](#) with a 2D destination buffer.

Figure 86. Programmed 2D addressing



GPDMA FIFO-based burst

In FIFO-mode, a transfer generally consists of two pipelined and separated burst transfers:

- one burst from the source to the FIFO over the allocated source master port, as defined by GPDMA_CxTR1.SAP
- one burst from the FIFO to the destination over the allocated destination master port, as defined by GPDMA_CxTR1.DAP

GPDMA source burst

The requested source burst transfer to the FIFO can be scheduled as early as possible over the allocated port, depending on the current FIFO level versus the programmed burst size (when the FIFO is ready to get one new burst from the source):

when $\text{FIFO level} \leq 2^{\text{dma_fifo_size}[x]} - (\text{SBL_1}[5:0]+1) * 2^{\text{SDW_LOG2}[1:0]}$

where:

- FIFO level is the current filling level of the FIFO, in bytes.
- $2^{\text{dma_fifo_size}[x]}$ is the half of the FIFO size of the channel x, in bytes (see [Section 16.3.2](#) for the implementation details and dma_fifo_size[x] value).
- $(\text{SBL_1}[5:0]+1) * 2^{\text{SDW_LOG2}[1:0]}$ is the size of the programmed source burst transfer, in bytes.

Based on the channel priority (GPDMA_CxCR.PRIO[1:0]), this ready FIFO-based source transfer is internally arbitrated versus the other requested and active channels.

GPDMA destination burst

The requested destination burst transfer from the FIFO can be scheduled as early as possible over the allocated port, depending on the current FIFO level versus the programmed burst size (when the FIFO is ready to push one new burst to the destination):

when $\text{FIFO level} \geq (\text{DBL_1}[5:0]+1) * 2^{\text{DDW_LOG2}[1:0]}$

where:

- FIFO level is the current filling level of the FIFO, in bytes.
- $(\text{DBL_1}[5:0]+1) * 2^{\text{DDW_LOG2}[1:0]}$ is the size of the programmed destination burst transfer, in bytes.

Based on the channel priority, this ready FIFO-based destination transfer is internally arbitrated versus the other requested and active channels.

GPDMA burst vs source block size, 1-Kbyte address boundary and FIFO size

The programmed source/destination GPDMA burst is implemented with an AHB burst as is, unless one of the following conditions is met:

- When half of the FIFO size of the channel x is lower than the programmed source/destination burst size, the programmed source/destination GPDMA burst is implemented with a series of singles or bursts of a lower size, each transfer being of a size that is lower or equal than half of the FIFO size, without any user constraint.
- if the source block size (GPDMA_CxBR1.BNDT[15:0]) is not a multiple of the source burst size but is a multiple of the data width of the source burst (GPDMA_CxTR1.SDW_LOG2[1:0]), the GPDMA modifies and shortens bursts into singles or bursts of lower length, in order to transfer exactly the source block size, without any user constraint.

- if the source/destination burst transfer have crossed the 1-Kbyte address boundary on a AHB transfer, the GPDMA modifies and shortens the programmed burst into singles or bursts of lower length, to be compliant with the AHB protocol, without any user constraint.
- If the source/destination burst length exceeds 16 on a AHB transfer, the GPDMA modifies and shortens the programmed burst into singles or bursts of lower length, to be compliant with the AHB protocol, without any user constraint.

In any case, the GPDMA keeps ensuring source/destination data (and address) integrity without any user constraint. The current FIFO level (software readable in GPDMA_CxSR) is compared to and updated with the effective transfer size, and the GPDMA re-arbitrates between each AHB single or burst transfer, possibly modified.

Based on the channel priority, each single or burst of a lower burst size versus the programmed burst, is internally arbitrated versus the other requested and active channels.

Note: In linked-list mode, the GPDMA read transfers related to the update of the linked-list parameters from the memory to the internal GPDMA registers, are scheduled over the link allocated port, as programmed by GPDMA_CxCR.LAP.

GPDMA data handling: byte-based reordering, packing/unpacking, padding/truncation, sign extension and left/right alignment

The data handling is controlled by GPDMA_CxTR1. The source/destination data width of the programmed burst is byte, half-word or word, as per the SDW_LOG2[1:0] and DDW_LOG2[1:0] fields (see [Table 142](#)).

The user can configure the data handling between transferred data from the source and transfer to the destination. More specifically, programmed data handling is orderly performed with:

1. Byte-based source reordering
 - If SBX = 1 and if source data width is a word, the two bytes of the unaligned half-word at the middle of each source data word are exchanged.
2. Data width conversion by packing, unpacking, padding or truncation, if destination data width is different than the source data width, depending on PAM[1:0]:
 - If destination data width > source data width, the post SBX source data is either right-aligned and padded with 0 s, or sign extended up to the destination data width, or is FIFO queued and packed up to the destination data width.
 - If destination data width < source data width, the post SBX data is either right-aligned and left-truncated down to the destination data width, or is FIFO queued and unpacked and streamed down to the destination data width.
3. Byte-based destination re-ordering:
 - If DBX = 1 and if the destination data width is not a byte, the two bytes are exchanged within the aligned post PAM[1:0] half-words.
 - If DHX = 1 and if the destination data width is neither a byte nor a half-word, the two aligned half-words are exchanged within the aligned post PAM[1:0] words.

Note: Left-alignment with 0s-padding can be achieved by programming both a right-alignment with a 0s-padding and a destination byte-based re-ordering.

The table below lists the possible data handling from the source to the destination.

Table 142. Programmed data handling

SDW_LOG2 [1:0]	Source data	Source data stream ⁽¹⁾	SB X	DDW_LOG2 [1:0]	Destination data	PAM[1:0] ⁽²⁾	DB X	DH X	Destination data stream ⁽¹⁾
00	Byte	B ₇ ,B ₆ ,B ₅ , B ₄ ,B ₃ ,B ₂ , B ₁ ,B ₀	x	00	Byte	xx	x		B ₇ ,B ₆ ,B ₅ ,B ₄ ,B ₃ ,B ₂ ,B ₁ ,B ₀
				01	Half-word	00 (RA, 0P)	0	x	0B ₃ ,0B ₂ ,0B ₁ ,0B ₀
							1		B ₃ 0,B ₂ 0,B ₁ 0,B ₀ 0
						01 (RA, SE)	0		SB ₃ ,SB ₂ ,SB ₁ ,SB ₀
							1		B ₃ S,B ₂ S,B ₁ S,B ₀ S
						1x (PACK)	0		B ₇ B ₆ ,B ₅ B ₄ ,B ₃ B ₂ ,B ₁ B ₀
							1		B ₆ B ₇ ,B ₄ B ₅ ,B ₂ B ₃ ,B ₀ B ₁
				10	Word	00 (RA, 0P)	0	0	000B ₁ ,000B ₀
							1		00B ₁ 0,00B ₀ 0
							0	1	0B ₁ 00,0B ₀ 00
							1		B ₁ 000,B ₀ 000
						01 (RA, SE)	0	0	SSSB ₁ ,SSSB ₀
							1		SSB ₁ S,SSB ₀ S
							0	1	SB ₁ SS,SB ₀ SS
							1		B ₁ SSS,B ₀ SSS
						1x (PACK)	0	0	B ₇ B ₆ B ₅ B ₄ ,B ₃ B ₂ B ₁ B ₀
							1		B ₆ B ₇ B ₄ B ₅ ,B ₂ B ₃ B ₀ B ₁
							0	1	B ₅ B ₄ B ₇ B ₆ ,B ₁ B ₀ B ₃ B ₂
							1		B ₄ B ₅ B ₆ B ₇ ,B ₀ B ₁ B ₂ B ₃
01	Half-word	B ₇ B ₆ ,B ₅ B ₄ , B ₃ B ₂ , B ₁ B ₀	x	00	Byte	00 (RA, LT)	x	x	B ₆ ,B ₄ ,B ₂ ,B ₀
						01 (LA, RT)			B ₇ ,B ₅ ,B ₃ ,B ₁
						1x (UNPACK)			B ₇ ,B ₆ ,B ₅ ,B ₄ ,B ₃ ,B ₂ ,B ₁ ,B ₀

Table 142. Programmed data handling (continued)

SDW_ LOG2 [1:0]	Source data	Source data stream ⁽¹⁾	SB X	DDW_ LOG2 [1:0]	Destination data	PAM[1:0] ⁽²⁾	DB X	DH X	Destination data stream ⁽¹⁾
01	Half- word	B ₇ B ₆ ,B ₅ B ₄ , B ₃ B ₂ , B ₁ B ₀	x	01	Half-word	xx	0	x	B ₇ B ₆ ,B ₅ B ₄ ,B ₃ B ₂ ,B ₁ B ₀
							1		B ₆ B ₇ ,B ₄ B ₅ ,B ₂ B ₃ ,B ₀ B ₁
				10	Word	00 (RA, 0P)	0	0	00B ₃ B ₂ ,00B ₁ B ₀
							1		00B ₂ B ₃ ,00B ₀ B ₁
							0	1	B ₃ B ₂ 00,B ₁ B ₀ 00
							1		B ₂ B ₃ 00,B ₀ B ₁ 00
						01 (RA, SE)	0	0	SSB ₃ B ₂ ,SSB ₁ B ₀
							1		SSB ₂ B ₃ ,SSB ₀ B ₁
							0	1	B ₃ B ₂ SS,B ₁ B ₀ SS
							1		B ₂ B ₃ SS,B ₀ B ₁ SS
						1x (PACK)	0	0	B ₇ B ₆ B ₅ B ₄ ,B ₃ B ₂ B ₁ B ₀
							1		B ₆ B ₇ B ₄ B ₅ ,B ₂ B ₃ B ₀ B ₁
							0	1	B ₅ B ₄ B ₇ B ₆ ,B ₁ B ₀ B ₃ B ₂
							1		B ₄ B ₅ B ₆ B ₇ ,B ₀ B ₁ B ₂ B ₃
10	Word	B ₇ B ₆ B ₅ B ₄ , B ₃ B ₂ B ₁ B ₀	0	00	Byte	00 (RA, LT)	x	x	B ₁₂ ,B ₈ ,B ₄ ,B ₀
						01 (LA, RT)			B ₁₅ ,B ₁₁ ,B ₇ ,B ₃
						10 (UNPACK)			B ₇ ,B ₆ ,B ₅ ,B ₄ ,B ₃ ,B ₂ ,B ₁ ,B ₀
				01	Half-word	00 (RA, LT)	0		B ₅ B ₄ ,B ₁ B ₀
							1		B ₄ B ₅ ,B ₀ B ₁
						01 (LA, RT)	0		B ₇ B ₆ ,B ₃ B ₂
							1		B ₆ B ₇ ,B ₂ B ₃
						1x (UNPACK)	0		B ₇ B ₆ ,B ₅ B ₄ ,B ₃ B ₂ ,B ₁ B ₀
							1		B ₆ B ₇ ,B ₄ B ₅ ,B ₂ B ₃ ,B ₀ B ₁

Table 142. Programmed data handling (continued)

SDW_ LOG2 [1:0]	Source data	Source data stream ⁽¹⁾	SB X	DDW_ LOG2 [1:0]	Destination data	PAM[1:0] ⁽²⁾	DB X	DH X	Destination data stream ⁽¹⁾
10	Word	B ₇ B ₆ B ₅ B ₄ , B ₃ B ₂ B ₁ B ₀	0	10	Word	xx	0	0	B ₇ B ₆ B ₅ B ₄ ,B ₃ B ₂ B ₁ B ₀
							1		B ₆ B ₇ B ₄ B ₅ ,B ₂ B ₃ B ₀ B ₁
							0	1	B ₅ B ₄ B ₇ B ₆ ,B ₁ B ₀ B ₃ B ₂
							1		B ₄ B ₅ B ₆ B ₇ ,B ₀ B ₁ B ₂ B ₃
			1	00	Byte	00 (RA, LT)	x	x	B ₁₂ ,B ₈ ,B ₄ ,B ₀
						01 (LA, RT)			B ₁₅ ,B ₁₁ ,B ₇ ,B ₃
						1x (UNPACK)			B ₇ ,B ₅ ,B ₆ ,B ₄ ,B ₃ ,B ₁ ,B ₂ ,B ₀
				01	Half-word	00 (RA, LT)	0		B ₆ B ₄ ,B ₂ B ₀
							1		B ₄ B ₆ ,B ₀ B ₂
						01 (LA, RT)	0		B ₇ B ₅ ,B ₃ B ₁
							1		B ₅ B ₇ ,B ₁ B ₃
						1x (UNPACK)	0		B ₇ B ₅ ,B ₆ B ₄ ,B ₃ B ₁ ,B ₂ B ₀
							1		B ₅ B ₇ ,B ₄ B ₆ ,B ₁ B ₃ ,B ₀ B ₂
				10	Word	xx	0	0	B ₇ B ₅ B ₆ B ₄ ,B ₃ B ₁ B ₂ B ₀
							1		B ₅ B ₇ B ₄ B ₆ ,B ₁ B ₃ B ₀ B ₂
							0	1	B ₆ B ₄ B ₇ B ₅ ,B ₂ B ₀ B ₃ B ₁
							1		B ₄ B ₆ B ₅ B ₇ ,B ₀ B ₂ B ₁ B ₃

1. Data stream is timely ordered starting from the byte with the lowest index (B₀).

2. RA= right aligned, LA = left aligned, RT = right truncated, LT = left truncated, OP = zero bit padding up to the destination data width, SE = sign bit extended up to the destination data width.

16.4.11 GPDMA transfer request and arbitration

GPDMA transfer request

As defined by GPDMA_CxTR2, a programmed GPDMA data transfer is requested with one of the following:

- a software request if the control bit SWREQ = 1: This is used typically by the CPU for a data transfer from a memory-mapped address to another memory mapped address (memory-to-memory, GPIO to/from memory)
- an input hardware request coming from a peripheral if SWREQ = 0: The selection of the GPDMA hardware peripheral request is driven by the REQSEL[7:0] field (see [Section 16.3.4](#)). The selected hardware request can be one of the following:
 - an hardware request from a peripheral configured in GPDMA mode (for a transfer from/to the peripheral data register respectively to/from the memory)
 - an hardware request from a peripheral for its control registers update from the memory
 - an hardware request from a peripheral for a read of its status registers transferred to the memory

Caution: The user must not assign a same input hardware peripheral GPDMA request via GPDMA_CxTR.REQSEL[7:0] to two different channels, if at a given time this request is asserted by the peripheral and each channel is ready to execute this requested data transfer. There is no user setting error reporting.

GPDMA transfer request for arbitration

A ready FIFO-based GPDMA source single/burst transfer (from the source address to the FIFO) to be scheduled over the allocated master port (GPDMA_CxTR1.SAP) is arbitrated based on the channel priority (GPDMA_CxCR.PRIO[1:0]) versus the other simultaneous requested GPDMA transfers to the same master port.

A ready FIFO-based GPDMA destination single/burst transfer (from the FIFO to the destination address) to be scheduled over the allocated master port (GPDMA_CxTR1.DAP) is arbitrated based on the channel priority (GPDMA_CxCR.PRIO[1:0]) versus the other simultaneous requested GPDMA transfers to the same master port.

An arbitrated GPDMA requested link transfer consists of one 32-bit read from the linked-list data structure in memory to one of the linked-list registers (GPDMA_CxTR1, GPDMA_CxTR2, GPDMA_CxBR1, GPDMA_CxSAR, GPDMA_CxDAR or GPDMA_CxLLR, plus GPDMA_CxTR3, GPDMA_CxBR2). Each 32-bit read from memory is arbitrated with the same channel priority as for data transfers, in order to be scheduled over the allocated master port (GPDMA_CxCR.LAP).

Whatever the requested data transfer is programmed with a software request for a memory-to-memory transfer (GPDMA_CxTR2.SWREQ = 1), or with a hardware request (GPDMA_CxTR2.SWREQ = 0) for a memory-to-peripheral transfer or a peripheral-to-memory transfer and whatever is the hardware request type, re-arbitration occurs after each granted single/burst transfer.

When an hardware request is programmed from a destination peripheral (GPDMA_CxTR2.SWREQ = 0 and GPDMA_CxTR2.DREQ = 1), the first memory read of a (possibly 2D/repeated) block (the first ready FIFO-based source burst request), is gated by the occurrence of the corresponding and selected hardware request. This first read request to memory is not taken into account earlier by the arbiter (not as soon as the block transfer is enabled and executable).

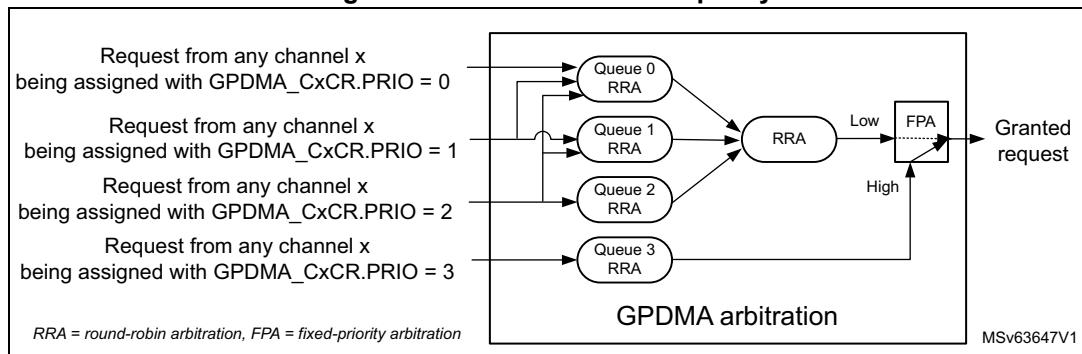
GPDMA arbitration

The GPDMA arbitration is directed from the 4-grade assigned channel priority (GPDMA_CxCR.PRIO[1:0]). The arbitration policy, as illustrated in [Figure 87](#), is defined by:

- one high-priority traffic class (queue 3), dedicated to the assigned channels with priority 3, for time-sensitive channels
This traffic class is granted via a fixed-priority arbitration against any other low-priority traffic class. Within this class, requested single/burst transfers are round-robin arbitrated.
- three low-priority traffic classes (queues 0, 1, or 2) for non time-sensitive channels with priority 0, 1, or 2
Each requested single/burst transfer within this class is round-robin arbitrated, with a weight that is monotonically driven from the programmed priority:
 - Requests with priority 0 are allocated to the queue 0.
 - Requests with priority 1 are allocated and replicated to the queue 0 and queue 1.

- Requests with priority 2 are allocated and replicated to the queue 0, queue 1, and queue 2.
- Any queue 0, 1, or 2 equally grants any of its active input requests in a round-robin manner, provided there are simultaneous requests.
- Additionally, there is a second stage for the low-traffic with a round-robin arbiter that fairly alternates between simultaneous selected requests from queue 0, queue 1, and queue 2.

Figure 87. GPDMA arbitration policy



GPDMA arbitration and bandwidth

With this arbitration policy, the following is guaranteed:

- Equal maximum bandwidth between requests with same priority
- Reserved bandwidth (noted as B_{Q3}) to the time-sensitive requests (with priority 3)
- Residual weighted bandwidth between different low-priority requests (priority 0 versus priority 1 versus priority 2).

The two following examples highlight that the weighted round-robin arbitration is driven by the programmed priorities:

- **Example 1:** basic application with two non time-sensitive GPDMA requests: req0 and req1. There are the following programming possibilities:
 - If they are assigned with same priority, the allocated bandwidth by the arbiter to req0 (B_{req0}) is **equal** to the allocated bandwidth to req1 (B_{req1}).

$$B_{req0} = B_{req1} = 1/2 * (1 - B_{Q3})$$
 - If req0 is assigned to priority 0 and req1 to priority 1, the allocated bandwidth to req0 (B_{P0}) is **3 times less** than the allocated bandwidth to req1 (B_{P1}).

$$B_{req0} = B_{P0} = 1/2 * 1/2 * (1 - B_{Q3}) = 1/4 * (1 - B_{Q3})$$

$$B_{req1} = B_{P1} = (1/2 + 1) * 1/2 * (1 - B_{Q3}) = 3/4 * (1 - B_{Q3})$$
 - If req0 is assigned to priority 0 and req1 to priority 2, the allocated bandwidth to req0 (B_{P0}) is **5 times less** than the allocated bandwidth to req1 (B_{P2}).

$$B_{req0} = B_{P0} = 1/2 * 1/3 * (1 - B_{Q3}) = 1/6 * (1 - B_{Q3})$$

$$B_{req1} = B_{P2} = (1/2 + 1 + 1) * 1/3 * (1 - B_{Q3}) = 5/6 * (1 - B_{Q3})$$

The above computed bandwidth calculation is based on a theoretical input request, always active for any GPDMA clock cycle. This computed bandwidth from the arbiter must be weighted by the frequency of the request given by the application, that cannot be always active and may be quite much variable from one GPDMA client (example I2C at 400 kHz) to another one (PWM at 1 kHz) than the above x3 and x5 ratios.

- **Example 2:** application where the user distributes a same non-null N number of GPDMA requests to every non time-sensitive priority 0, 1 and 2. The bandwidth calculation is then the following:
 - The allocated bandwidth to the set of requests of priority 0 (B_{P0}) is

$$B_{P0} = 1/3 * 1/3 * (1 - B_{Q3}) = 1/9 * (1 - B_{Q3})$$
 - The allocated bandwidth to the set of requests of priority 1 (B_{P1}) is

$$B_{P1} = (1/3 + 1/2) * 1/3 * (1 - B_{Q3}) = 5/18 * (1 - B_{Q3})$$
 - The allocated bandwidth to the set of requests of priority 2 (B_{P2}) is

$$B_{P2} = (1/3 + 1/2 + 1) * 1/3 * (1 - B_{Q3}) = 11/18 * (1 - B_{Q3})$$
 - The allocated bandwidth to any request n (B_n) among the N requests of that priority P_i ($i = 0$ to 2) is $B_n = 1/N * B_{P_i}$
 - The allocated bandwidth to any request n of priority 0_i (B_{n, P_i}) is

$$B_{n, P0} = 1/N * 1/9 * (1 - B_{Q3})$$

$$B_{n, P1} = 1/N * 5/18 * (1 - B_{Q3})$$

$$B_{n, P2} = 1/N * 11/18 * (1 - B_{Q3})$$

In this example, when the master port bus bandwidth is not totally consumed by the time-sensitive queue 3, the residual bandwidth is such that 2.5 times less bandwidth is allocated to any request of priority 0 versus priority 1, and 5.5 times less bandwidth is allocated to any request of priority 0 versus priority 2.

More generally, assume that the following requests are present:

- I requests ($I \geq 0$) assigned to priority 0
 If $I > 0$, these requests are noted from $i = 0$ to $I-1$.
- J requests ($J \geq 0$) assigned to priority 1
 If $J > 0$, these requests are noted from $j = 0$ to $J-1$.
- K requests ($K > 0$) assigned to priority 2
 These requests are noted from $k = 0$ to $K-1$
- L requests ($L \geq 0$) assigned to priority 3
 If $L > 0$, these requests are noted from $l = 0$ to $L-1$.

As B_{Q3} is the reserved bandwidth to time-sensitive requests, the bandwidth for each request L with priority 3 is:

- $B_l = B_{Q3} / L$ for $L > 0$ (else: $B_l = 0$)

The bandwidth for each non-time sensitive queue is:

- $B_{Q0} = 1/3 * (1 - B_{Q3})$
- $B_{Q1} = 1/3 * (1 - B_{Q3})$
- $B_{Q2} = 1/3 * (1 - B_{Q3})$

The bandwidth for the set of requests with priority 0 is:

- $B_{P0} = I / (I + J + K) * B_{Q0}$

The bandwidth for each request i with priority 0 is:

- $B_i = B_{P0} / I$ for $L > 0$ (else $B_{P0} = 0$)

The bandwidth for the set of requests with priority 1 and routed to queue 0 is:

- $B_{P1, Q0} = J / (I + J + K) * B_{Q0}$

The bandwidth for the set of requests with priority 1 and routed to queue 1 is:

- $B_{P1,Q1} = J / (J + K) * B_{Q1}$

The total bandwidth for the set of requests with priority 1 is:

- $B_{P1} = B_{P1,Q0} + B_{P1,Q1}$

The bandwidth for each request j with priority 1 is:

- $B_j = B_{P1} / J$ for $J > 0$ (else $B_j = 0$)

The bandwidth for the set of requests with priority 2 and routed to queue 0 is:

- $B_{P2,Q0} = K / (I + J + K) * B_{Q0}$

The bandwidth for the set of requests with priority 2 and routed to queue 1 is:

- $B_{P2,Q1} = K / (J + K) * B_{Q1}$

The bandwidth for the set of requests with priority 2 and routed to queue 2 is:

- $B_{P2,Q2} = B_{Q2}$

The total bandwidth for the set of requests with priority 2 is:

- $B_{P2} = B_{P2,Q0} + B_{P2,Q1} + B_{P2,Q2}$

The bandwidth for each request k with priority 2 is:

- $B_k = B_{P2} / K$ ($K > 0$ in the general case)

Thus finally the maximum allocated residual bandwidths for any i, j, k non-time sensitive request are:

- in the general case (when there is at least one request k with a priority 2 ($K > 0$)):
 - $B_i = 1/I * 1/3 * I/(I + J + K) * (1 - B_{Q3})$
 - $B_j = 1/J * 1/3 * [J/(I + J + K) + J/(J + K)] * (1 - B_{Q3})$
 - $B_k = 1/K * 1/3 * [K/(I + J + K) + K/(J + K) + 1] * (1 - B_{Q3})$
- in the specific case (when there is no request k with a priority 2 ($K = 0$)):
 - $B_i = 1/I * 1/2 * I/(I + J) * (1 - B_{Q3})$
 - $B_j = 1/J * 1/2 * [J/(I + J) + 1] * (1 - B_{Q3})$

Consequently, the GPDMA arbiter can be used as a programmable weighted bandwidth limiter, for each queue and more generally for each request/channel. The different weights are monotonically resulting from the programmed channel priorities.

16.4.12 GPDMA triggered transfer

A programmed GPDMA transfer can be triggered by a rising/falling edge of a selected input trigger event, as defined by GPDMA_CxTR2.TRIGPOL[1:0] and GPDMA_CxTR2.TRIGSEL[5:0] (see [Section 16.3.7](#) for the trigger selection).

The triggered transfer, as defined by the trigger mode in GPDMA_CxTR2.TRIGM[1:0], can be at LLI data transfer level, to condition the first burst read of a block, the first burst read of a 2D/repeated block for channel x ($x = 6$ to 7), or each programmed single read. The trigger mode can also be programmed to condition the LLI link transfer (see TRIGM[1:0] in GPDMA_CxTR2 for more details).

Trigger hit memorization and trigger overrun flag generation

The GPDMA monitoring of a trigger for a channel x is started when the channel is enabled/loaded with a new active trigger configuration: rising or falling edge on a selected trigger (respectively TRIGPOL[1:0] = 01 or TRIGPOL[1:0] = 10).

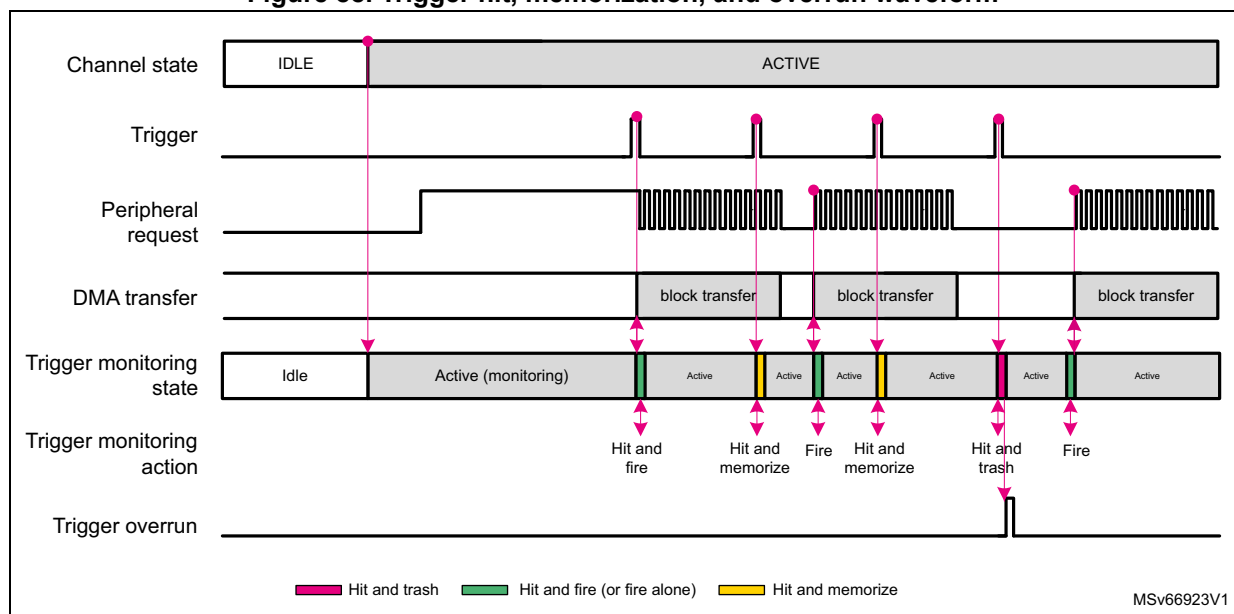
The monitoring of this trigger is kept active during the triggered and uncompleted (data or link) transfer. If a new trigger is detected, this hit is internally memorized to grant the next transfer, as long as the defined rising/falling edge and TRIGSEL[5:0] are not modified, and the channel is enabled.

Transferring a next LLI_{n+1} , that updates the GPDMA_CxTR2 with a new value for any of TRIGSEL[5:0] or TRIGPOL[1:0], resets the monitoring, trashing the possible memorized hit of the formerly defined LLI_n trigger.

Caution: After a first new trigger hit_{n+1} is memorized, if another trigger hit_{n+2} is detected and if the hit_n triggered transfer is still not completed, hit_{n+2} is lost and not memorized. A trigger overrun flag is reported (GPDMA_CxSR.TOF = 1) and an interrupt is generated if enabled (if GPDMA_CxCR.TOIE = 1). The channel is not automatically disabled by hardware due to a trigger overrun.

Figure 88 illustrates the trigger hit, memorization and overrun in the configuration example with a block-level trigger mode and a rising edge trigger polarity.

Figure 88. Trigger hit, memorization, and overrun waveform



Note: The user can assign the same input trigger event to different channels. This can be used to trigger different channels on a broadcast trigger event.

16.4.13 GPDMA circular buffering with linked-list programming

GPDMA circular buffering for memory-to-peripheral and peripheral-to-memory transfers, with a linear addressing channel

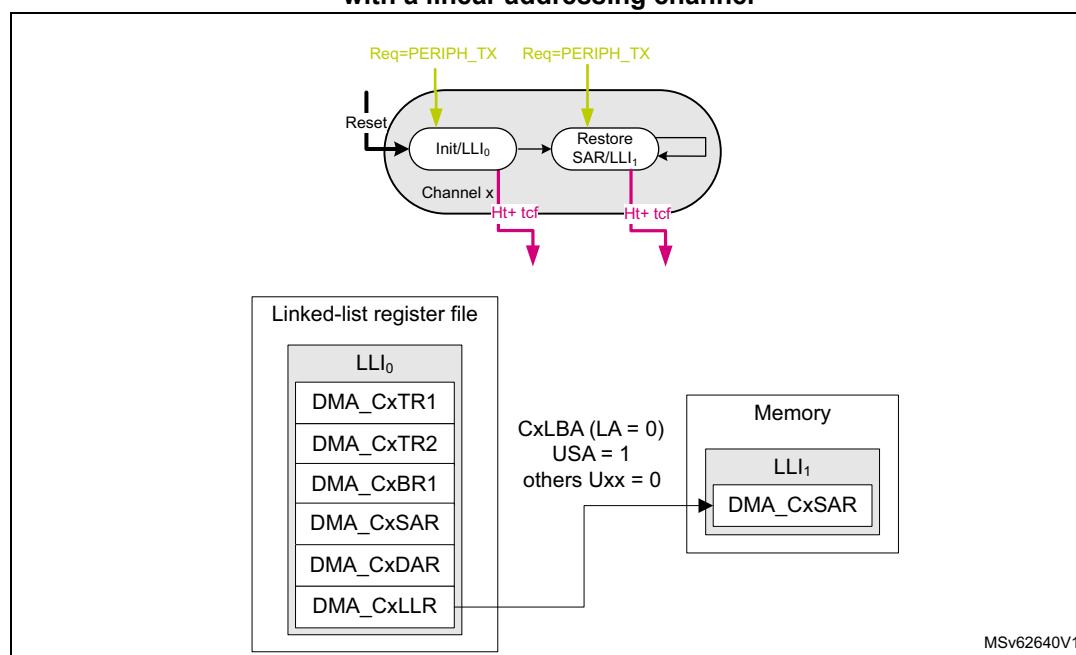
For a circular buffering, with a continuous memory-to-peripheral (or peripheral-to-memory) transfer, the software must set up a channel with half transfer and complete transfer

events/interrupts generation (GPDMA_CxCR.HTIE = 1 and GPDMA_CxCR.TCIE = 1), in order to enable a concurrent buffer software processing.

LLI₀ is configured for the first block transfer with the linear addressing channel. A continuously-executed LLI₁ is needed to restore the memory source (or destination) start address, for the memory-to-peripheral transfer (respectively the peripheral-to-memory transfer). GPDMA automatically reloads the initially programmed GPDMA_CxBR1.BNDT[15:0] when a block transfer is completed, and there is no need to restore GPDMA_CxBR1.

Figure 89 illustrates this programming with a linear addressing GPDMA channel and a source circular buffer.

Figure 89. GPDMA circular buffer programming: update of the memory start address with a linear addressing channel



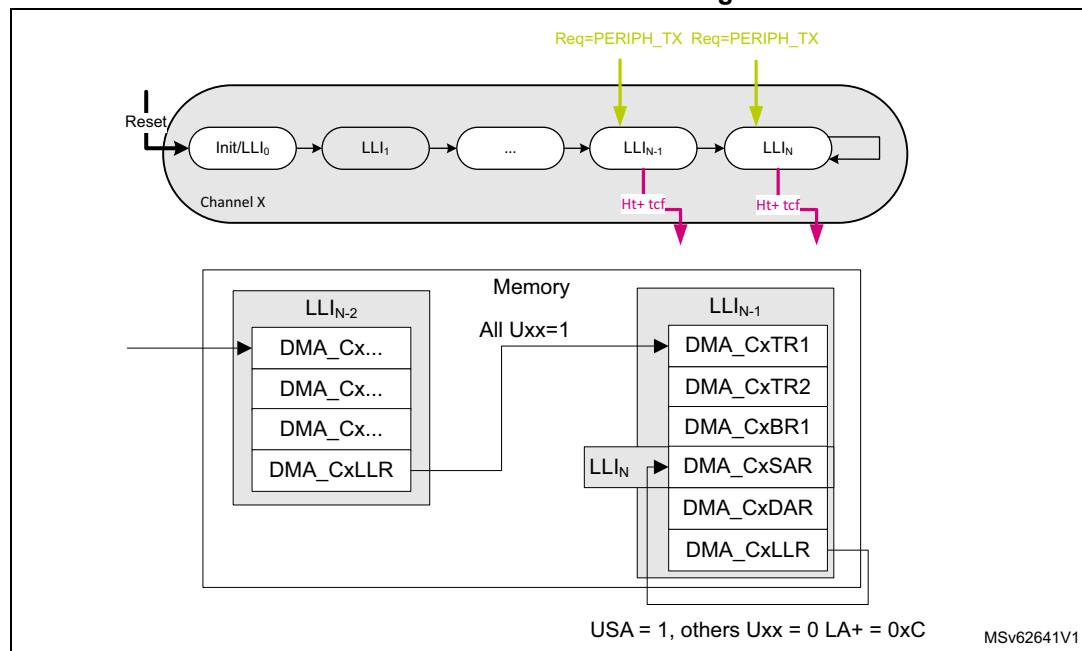
Note:

With a 2D addressing channel, the user may use a single LLI with GPDMA_CxBR1.BRC[10:0] = 1, and program a negative memory block address offset with GDMA_CxBR2 and GDMA_CxBR1, in order to jump back to the memory source or the destination start address.

If circular buffering must be executed after some other transfers over the shared GPDMA channel x, the before-last LLI_{N-1} in memory is needed to configure the first block transfer. And the last LLI_N restores the memory source (or destination) start address in memory-to-peripheral transfer (respectively in peripheral-to-memory transfer).

Figure 90 illustrates this programming with a linear addressing shared GPDMA channel, and a source circular buffer.

Figure 90. Shared GPDMA channel with circular buffering: update of the memory start address with a linear addressing channel



16.4.14 GPDMA transfer in peripheral flow-control mode

A peripheral with the peripheral flow-control mode feature can decide to early terminate a GPDMA block transfer, provided that the allocated channel is implemented with this feature (see [Section 16.3.6](#)).

If the related GPDMA channel x is also programmed in peripheral flow-control mode (GPDMA_CxTR2.PFREQ = 1):

- The GPDMA block transfer starts as follows:
 - If GPDMA_CxBR1.BNDT[15:0] $\neq 0$, the programmed value is internally taken into account by the GPDMA hardware.
 - If GPDMA_CxBR1.BNDT[15:0] = 0, the GPDMA hardware internally considers a 64-Kbyte value for the maximum source block size to be transferred.
- The GPDMA block transfer is completed as soon as the first occurrence of one of the following condition occurs:
 - when GPDMA_CxBR1.BNDT[15:0] = 0
 - when the peripheral early terminates the block. The complete transfer event is generated if programmed, depending on GPDMA_CxTR2 (see [GPDMA channel \$x\$ transfer register 2 \(GPDMA_CxTR2\)](#)). Then the software can read the current number of transferred bytes from the source (GPDMA_CxBR1.BNDT[15:0]), and/or read the current source or destination address of the buffer in memory (GPDMA_CxSAR[31:0] or GPDMA_CxDAR[31:0]).

In peripheral flow-control mode:

- a destination peripheral with a hardware requested transfer is not supported: memory-to-peripheral transfer is not supported.
- Data packing from a source peripheral is not supported.
- 2D/repeated block is not supported.
- GPDMA_CxBR1.BNDT[15:0] must be programmed as a multiple of the source (peripheral) burst size.

16.4.15 GPDMA secure/nonsecure channel

The GPDMA controller is compliant with the TrustZone hardware architecture at channel level, partitioning all its resources so that they exist in one of the secure and nonsecure worlds at any given time.

Any channel x is a secure or a nonsecure hardware resource, as configured by GPDMA_SECCFGR.SECx.

When a channel x is configured in secure state by a secure and privileged agent, the following access control rules are applied:

- A nonsecure read access to a register field of this channel is forced to return 0, except for GPDMA_SECCFGR, GPDMA_PRIVCFGR and GPDMA_RCFGLOCKR that are readable by a nonsecure agent.
- A nonsecure write access to a register field of this channel has no impact.

When a channel x is configured in secure state, a secure agent can configure separately as secure or nonsecure the GPDMA data transfer from the source (GPDMA_CxTR1.SSEC) and the GPDMA data transfer to the destination (GPDMA_CxTR1.DSEC).

When a channel x is configured in secure state and in linked-list mode, the loading of the next linked-list data structure from the GPDMA memory into its register file, is automatically performed with secure transfers via the GPDMA_CxCR.LAP allocated master port.

The GPDMA generates a secure bus that reflects GPDMA_SECCFGR, to keep the other peripherals informed of the secure/nonsecure state of each GPDMA channel x.

The GPDMA also generates a security illegal access pulse signal on an illegal nonsecure access to a secure GPDMA register. This signal is routed to the TrustZone interrupt controller.

When the secure software must switch a channel from a secure state to a nonsecure state, the secure software must abort the channel or wait until the secure channel is completed before switching. This is needed to dynamically re-allocate a channel to a next nonsecure transfer as a nonsecure software is not allowed to do so and must have GPDMA_CxCR.EN = 0 before the nonsecure software can reprogram the GPDMA_CxCR for a next transfer. The secure software may reset not only the channel x (GPDMA_CxCR.RESET = 1) but also the full channel x register file to its reset value.

16.4.16 GPDMA privileged/unprivileged channel

Any channel x is a privileged or unprivileged hardware resource, as configured by a privileged agent via GPDMA_PRIVCFGR.PRIVx.

When a channel x is configured in a privileged state by a privileged agent, the following access control rules are applied:

- An unprivileged read access to a register field of this channel is forced to return 0, except for GPDMA_PRIVCFGR, GPDMA_SECCFGR, and GPDMA_RCFGLOCKR that are readable by an unprivileged agent.
- An unprivileged write access to a register field of this channel has no impact.

When a channel is configured in a privileged (or unprivileged) state, the source and destination data transfers are privileged (respectively unprivileged) transfers over the AHB master port.

When a channel is configured in a privileged (or unprivileged) state and in linked-list mode, the loading of the next linked-list data structure from the GPDMA memory into its register file, is automatically performed with privileged (respectively unprivileged) transfers, via the GPDMA_CxCR.LAP allocated master port.

The GPDMA generates a privileged bus that reflects GPDMA_PRIVCFGR, to keep the other peripherals informed of the privileged/unprivileged state of each GPDMA channel x.

When the privileged software must switch a channel from a privileged state to an unprivileged state, the privileged software must abort the channel or wait until that the privileged channel is completed before switching. This is needed to dynamically re-allocate a channel to a next unprivileged transfer as an unprivileged software is not allowed to do so, and must have GPDMA_CxCR.EN = 0 before the unprivileged software can reprogram the GPDMA_CxCR for a next transfer. The privileged software may reset not only the channel x (GPDMA_CxCR.RESET = 1) but also the full channel x register file to its reset value.

16.4.17 GPDMA error management

The GPDMA is able to manage and report to the user a transfer error, as follows, depending on the root cause.

Data transfer error

On a bus access (as a AHB single or a burst) to the source or the destination:

- The source or destination target reports an AHB error.
- The programmed channel transfer is stopped (GPDMA_CxCR.EN cleared by the GPDMA hardware). The channel status register reports an idle state (GPDMA_CxSR.IDLEF = 1) and the data error (GPDMA_CxSR.DTEF = 1).
- After a GPDMA data transfer error, the user must perform a debug session, taking care of the product-defined memory mapping of the source and destination, including the protection attributes.
- After a GPDMA data transfer error, the user must issue a channel reset (set GPDMA_CxCR.RESET) to reset the hardware GPDMA channel data path and the content of the FIFO, before the user enables again the same channel for a next transfer.

Link transfer error

On a tentative update of a GPDMA channel register from the programmed LLI in the memory:

- The linked-list memory reports an AHB error.
- The programmed channel transfer is stopped (GPDMA_CxCR.EN cleared by the GPDMA hardware), the channel status register reports an idle state (GPDMA_CxSR.IDLEF = 1) and the link error (GPDMA_CxSR.ULEF = 1).
- After a GPDMA link error, the user must perform a debug session, taking care of the product-defined memory mapping of the linked-list data structure (GPDMA_CxLBAR and GPDMA_CxLLR), including the protection attributes.
- After a GPDMA link error, the user must explicitly write the linked-list register file (GPDMA_CxTR1, GPDMA_CxTR2, GPDMA_CxBR1, GPDMA_CxSAR, GPDMA_CxDAR and GPDMA_CxLLR, plus GPDMA_CxTR3 and GPDMA_CxBR2), before the user enables again the same channel for a next transfer.

User setting error

On a tentative execution of a GPDMA transfer with an unauthorized user setting:

- The programmed channel transfer is disabled (GPDMA_CxCR.EN forced and cleared by the GPDMA hardware) preventing the next unauthorized programmed data transfer from being executed. The channel status register reports an idle state (GPDMA_CxSR.IDLEF = 1) and a user setting error (GPDMA_CxSR.USEF = 1).
- After a GPDMA user setting error, the user must perform a debug session, taking care of the GPDMA channel programming. A user setting error can be caused by one of the following:
 - a programmed null source block size without a programmed update of this value from the next LLI₁ (GPDMA_CxBR1.BNDT[15:0] = 0 and GPDMA_CxLLR.UB1 = 0)
 - a programmed non-null source block size being not a multiple of the programmed data width of a source burst transfer (GPDMA_CxBR1.BNDT[2:0] versus GPDMA_CxTR1.SDW_LOG2[1:0])
 - when in packing/unpacking mode (if PAM[1] = 1), a programmed non-null source block size being not a multiple of the programmed data width of a destination burst transfer (GPDMA_CxBR1.BNDT[2:0] versus GPDMA_CxTR1.DDW_LOG2[1:0])
 - a programmed unaligned source start address, being not a multiple of the programmed data width of a source burst transfer (GPDMA_CxSAR[2:0] versus GPDMA_CxTR1.SDW_LOG2[1:0])
 - for channel x (x = 6 to 7): a programmed unaligned source address offset being not a multiple of the programmed data width of a source burst transfer (GPDMA_CxTR3.SAO[2:0] versus GPDMA_CxTR1.SDW_LOG2[1:0])
 - for channel x (x = 6 to 7): a programmed unaligned block repeated source address offset being not a multiple of the programmed data width of a source burst transfer (GPDMA_CxBR2.BRSAO[2:0] versus GPDMA_CxTR1.SDW_LOG2[1:0])
 - a programmed unaligned destination start address, being not a multiple of the programmed data width of a destination burst transfer (GPDMA_CxDAR[2:0] versus GPDMA_CxTR1.DDW_LOG2[1:0])
 - for channel x (x = 6 to 7): a programmed unaligned destination address offset being not a multiple of the programmed data width of a destination burst transfer (GPDMA_CxTR3.DAO[2:0] versus GPDMA_CxTR1.DDW_LOG2[1:0])

- for channel x (x = 6 to 7): a programmed unaligned block repeated destination address offset being not a multiple of the programmed data width of a destination burst transfer (GPDMA_CxBR2.BRDAO[2:0] versus GPDMA_CxTR1.DDW_LOG2[1:0])
- a programmed double-word source data width (GPDMA_CxTR1.SDW_LOG2[1:0] = 11)
- a programmed double-word destination data width (GPDMA_CxTR1.DDW_LOG2[1:0] = 11)
- a programmed linked-list item LLI_{n+1} with a null data transfer (GPDMA_CxLLR.UB1 = 1 and GPDMA_CxBR1.BNDT = 0)

16.5 GPDMA in debug mode

When the microcontroller enters debug mode (core halted), any channel x can be individually either continued (default) or suspended, depending on the programmable control bit in the DBGMCU module.

Note: In debug mode, GPDMA_CxSR.SUSPF is not altered by a suspension from the programmable control bit in the DBGMCU module. In this case, GPDMA_CxSR.IDLEF can be checked to know the completion status of the channel suspension.

16.6 GPDMA in low-power modes

Table 143. Effect of low-power modes on GPDMA

Mode	Description
Sleep	No effect. GPDMA interrupts cause the device to exit Sleep mode.
Stop ⁽¹⁾	The content of the GPDMA registers is kept when entering Stop mode.
Standby	The GPDMA is powered down and must be reinitialized after exiting Standby mode.

1. Refer to [Section 16.3.3](#) to know if any Stop mode is supported.

16.7 GPDMA interrupts

There is one GPDMA interrupt line for each channel, and separately for each CPU (if several ones in the devices).

Table 144. GPDMA interrupt requests

Interrupt acronym	Interrupt event	Interrupt enable	Event flag	Event clear method
GPDMA_CHx	Transfer complete	GPDMA_CxCR.TCIE	GPDMA_CxSR.TCF	Write 1 to GPDMA_CxFCR.TCF
	Half transfer	GPDMA_CxCR.HTIE	GPDMA_CxSR.HTF	Write 1 to GPDMA_CxFCR.HTF
	Data transfer error	GPDMA_CxCR.DTEIE	GPDMA_CxSR.DTEF	Write 1 to GPDMA_CxFCR.DTEF
	Update link error	GPDMA_CxCR.ULEIE	GPDMA_CxSR.ULEF	Write 1 to GPDMA_CxFCR.ULEF
	User setting error	GPDMA_CxCR.USEIE	GPDMA_CxSR.USEF	Write 1 to GPDMA_CxFCR.USEF
	Suspended	GPDMA_CxCR.SUSPIE	GPDMA_CxSR.SUSPF	Write 1 to GPDMA_CxFCR.SUSPF
	Trigger overrun	GPDMA_CxCR.TOFIE	GPDMA_CxSR.TOF	Write 1 to GPDMA_CxFCR.TOF

A GPDMA channel x event may be:

- a transfer complete
- a half-transfer complete
- a transfer error, due to either:
 - a data transfer error
 - an update link error
 - a user setting error completed suspension
- a trigger overrun

Note: When a channel x transfer complete event occurs, the output signal *gpdma_chx_tc* is generated as a high pulse of one clock cycle.

An interrupt is generated following any xx event, provided that both:

- the corresponding interrupt event xx is enabled (GPDMA_CxCR.xxIE = 1)
- the corresponding event flag is cleared (GPDMA_CxSR.xxF = 0). This means that, after a previous same xx event occurrence, a software agent must have written 1 into the corresponding xx flag clear control bit (write 1 into GPDMA_CxFCR.xxF).

TCF (transfer complete) and HTF (half transfer) events generation is controlled by GPDMA_CxTR2.TCEM[1:0] as follows:

- A transfer complete event is a block transfer complete, a 2D/repeated block transfer complete, or a LLI transfer complete including the upload of the next LLI if any, or the full linked-list completion, depending on the transfer complete event mode GPDMA_CxTR2.TCEM[1:0].

- A half transfer event is an half block transfer or a half 2D/repeated block transfer, depending on the transfer complete event mode GPDMA_CxTR2.TCEM[1:0].
A half-block transfer occurs when half of the source block size bytes (rounded-up integer of $\text{GPDMA_CxBR1.BNDT}[15:0] / 2$) is transferred to the destination.
A half 2D/repeated block transfer occurs when half of the repeated blocks (rounded-up integer of $(\text{GPDMA_CxBR1.BRC}[10:0] + 1) / 2$) is transferred to the destination.

See [GPDMA channel x transfer register 2 \(GPDMA_CxTR2\)](#) for more details.

A transfer error rises in one of the following situations:

- during a single/burst data transfer from the source or to the destination (DTEF)
- during an update of a GPDMA channel register from the programmed LLI in memory (ULEF)
- during a tentative execution of a GPDMA channel with an unauthorized setting (USEF)
The user must perform a debug session to correct the GPDMA channel programming versus the USEF root causes list (see [Section 16.4.17](#)).

A trigger overrun is described in [Trigger hit memorization and trigger overrun flag generation](#).

16.8 GPDMA registers

The GPDMA registers must be accessed with an aligned 32-bit word data access.

16.8.1 GPDMA secure configuration register (GPDMA_SECCFGR)

Address offset: 0x00

Reset value: 0x0000 0000

A write access to this register must be secure and privileged. A read access is secure or nonsecure, privileged or unprivileged.

A write access is ignored at bit level if the corresponding channel x is locked ($\text{GPDMA_RCFGLOCKR.LOCKx} = 1$).

This register must be written when $\text{GPDMA_CxCR.EN} = 0$.

This register is read-only when $\text{GPDMA_CxCR.EN} = 1$.

This register must be programmed at a bit level, at the initialization/closure of a GPDMA channel (when $\text{GPDMA_CxCR.EN} = 0$), to securely allocate individually any channel x to the secure or nonsecure world.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SEC7	SEC6	SEC5	SEC4	SEC3	SEC2	SEC1	SEC0
								rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **SECx**: secure state of channel x (x = 7 to 0)

0: Nonsecure

1: Secure

16.8.2 GPDMA privileged configuration register (GPDMA_PRIVCFGR)

Address offset: 0x04

Reset value: 0x0000 0000

A write access to this register must be privileged. A read access can be privileged or unprivileged, secure or nonsecure.

This register can mix secure and nonsecure information. If a channel x is configured as secure (GPDMA_SECCFGR.SECx = 1), the PRIVx bit can be written only by a secure (and privileged) agent.

A write access is ignored at bit level if the corresponding channel x is locked (GPDMA_RCFGLOCKR.LOCKx = 1).

This register must be written when GPDMA_CxCR.EN = 0.

This register is read-only when GPDMA_CxCR.EN = 1.

This register must be programmed at a bit level, at the initialization/closure of a GPDMA channel (GPDMA_CxCR.EN = 0), to individually allocate any channel x to the privileged or unprivileged world.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PRIV7	PRIV6	PRIV5	PRIV4	PRIV3	PRIV2	PRIV1	PRIV0
								rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **PRIVx**: privileged state of channel x (x = 7 to 0)

0: unprivileged

1: privileged

16.8.3 GPDMA configuration lock register (GPDMA_RCFGLOCKR)

Address offset: 0x08

Reset value: 0x0000 0000

This register can be written by a software agent with secure privileged attributes in order to individually lock, for example at boot time, the secure privileged attributes of any GPDMA

channel/resource (to lock the setting of GPDMA_CxSECCFGR and GPDMA_CxPRIVCFGR for any channel x, for example at boot time).

A read access may be privileged or unprivileged, secure or nonsecure.

Note: If $TZEN = 0$, this register cannot be written.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	LOCK7	LOCK6	LOCK5	LOCK4	LOCK3	LOCK2	LOCK1	LOCK0
								rs	rs	rs	rs	rs	rs	rs	rs

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **LOCKx**: lock the configuration of GPDMA_SECCFGR.SECx and GPDMA_PRIVCFGR.PRIVx, until a global GPDMA reset (x = 7 to 0)

This bit is cleared after reset and, once set, it cannot be reset until a global GPDMA reset.

0: secure privilege configuration of the channel x is writable.

1: secure privilege configuration of the channel x is not writable.

16.8.4 GPDMA nonsecure masked interrupt status register (GPDMA_MISR)

Address offset: 0x0C

Reset value: 0x0000 0000

This register is a read register.

This is a nonsecure register, containing the masked interrupt status bit MISx for each nonsecure channel x (channel x configured with GPDMA_SECCFGR.SECx = 0). It is a logical OR of all the flags of GPDMA_CxSR, each source flag being enabled by the corresponding interrupt enable bit of GPDMA_CxFCR.

Every bit is deasserted by hardware when writing 1 to the corresponding flag clear bit in GPDMA_CxFCR.

If a channel x is in secure state (GPDMA_SECCFGR.SECx = 1), a read access to the masked interrupt status bit MISx of this channel x returns zero.

This register may mix privileged and unprivileged information, depending on the privileged state of each channel GPDMA_PRIVCFGR.PRIVx. A privileged software can read the full nonsecure interrupt status. An unprivileged software is restricted to read the status of unprivileged (and nonsecure) channels, other privileged bit fields returning zero.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	MIS7	MIS6	MIS5	MIS4	MIS3	MIS2	MIS1	MIS0
								r	r	r	r	r	r	r	r

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **MISx**: masked interrupt status of channel x (x = 7 to 0)

0: no interrupt occurred on channel x

1: an interrupt occurred on channel x

16.8.5 GPDMA secure masked interrupt status register (GPDMA_SMISR)

Address offset: 0x10

Reset value: 0x0000 0000

This is a secure read register, containing the masked interrupt status bit MISx for each secure channel x (GPDMA_SECCFGR.SECx = 1). It is a logical OR of all the GPDMA_CxSR flags, each source flag being enabled by the corresponding GPDMA_CxCR interrupt enable bit.

Every bit is deasserted by hardware when securely writing 1 to the corresponding GPDMA_CxFCR flag clear bit.

This register does not contain any information about a nonsecure channel.

This register can mix privileged and unprivileged information, depending on the privileged state of each channel GPDMA_PRIVCFGR.PRIVx. A privileged software can read the full secure interrupt status. An unprivileged software is restricted to read the status of unprivileged and secure channels, other privileged bit fields returning zero.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	MIS7	MIS6	MIS5	MIS4	MIS3	MIS2	MIS1	MIS0
								r	r	r	r	r	r	r	r

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **MISx**: masked interrupt status of the secure channel x (x = 7 to 0)

0: no interrupt occurred on the secure channel x

1: an interrupt occurred on the secure channel x

16.8.6 GPDMA channel x linked-list base address register (GPDMA_CxLBAR)

Address offset: 0x50 + 0x80 * x (x = 0 to 7)

Reset value: 0x0000 0000

This register must be written by a privileged software. It is either privileged readable or not, depending on the privileged state of the channel x GPDMA_PRIVCFGR.PRIVx.

This register is either secure or nonsecure depending on the secure state of the channel x (GPDMA_SECCFGR.SECx).

This register must be written when GPDMA_CxCR.EN = 0.

This register is read-only when GPDMA_CxCR.EN = 1.

This channel-based register is the linked-list base address of the memory region, for a given channel x , from which the LLIs describing the programmed sequence of the GPDMA transfers, are conditionally and automatically updated.

This 64-Kbyte aligned channel x linked-list base address is offset by the 16-bit GPDMA_CxLLR register that defines the word-aligned address offset for each LLI.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LBA[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:16 **LBA[31:16]**: linked-list base address of GPDMA channel x

Bits 15:0 Reserved, must be kept at reset value.

16.8.7 GPDMA channel x flag clear register (GPDMA_CxFCR)

Address offset: $0x5C + 0x80 * x$ ($x = 0$ to 7)

Reset value: $0x0000\ 0000$

This is a write register, secure or nonsecure depending on the secure state of channel x (GPDMA_SECCFGR.SECx) and privileged or unprivileged, depending on the privileged state of the channel x (GPDMA_PRIVCFGR.PRIVx).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	TOF	SUSPF	USEF	ULEF	DTEF	HTF	TCF	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	w	w	w	w	w	w	w								

Bits 31:15 Reserved, must be kept at reset value.

Bit 14 **TOF**: trigger overrun flag clear

0: no effect

1: corresponding TOF flag cleared

Bit 13 **SUSPF**: completed suspension flag clear

0: no effect

1: corresponding SUSPF flag cleared

Bit 12 **USEF**: user setting error flag clear

0: no effect

1: corresponding USEF flag cleared

Bit 11 **ULEF**: update link transfer error flag clear

0: no effect

1: corresponding ULEF flag cleared

- Bit 10 **DTEF**: data transfer error flag clear
 0: no effect
 1: corresponding DTEF flag cleared
- Bit 9 **HTF**: half transfer flag clear
 0: no effect
 1: corresponding HTF flag cleared
- Bit 8 **TCF**: transfer complete flag clear
 0: no effect
 1: corresponding TCF flag cleared

Bits 7:0 Reserved, must be kept at reset value.

16.8.8 GPDMA channel x status register (GPDMA_CxSR)

Address offset: $0x60 + 0x80 * x$ ($x = 0$ to 7)

Reset value: 0x0000 0001

This is a read register, reporting the channel status.

This register is secure or nonsecure, depending on the secure state of channel x (GPDMA_SECCFGR.SECx), and privileged or unprivileged, depending on the privileged state of the channel (GPDMA_PRIVCFGR.PRIVx).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FIFOL[7:0]							
								r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	TOF	SUSPF	USEF	ULEF	DTEF	HTF	TCF	Res.	Res.	Res.	Res.	Res.	Res.	Res.	IDLEF
	r	r	r	r	r	r	r								r

Bits 31:24 Reserved, must be kept at reset value.

Bits 23:16 **FIFOL[7:0]**: monitored FIFO level

Number of available write beats in the FIFO, in units of the programmed destination data width (see GPDMA_CxTR1.DDW_LOG2[1:0], in units of bytes, half-words, or words).

Note: After having suspended an active transfer, the user may need to read FIFOL[7:0], additionally to GPDMA_CxBR1.BDNT[15:0] and GPDMA_CxBR1.BRC[10:0], to know how many data have been transferred to the destination. Before reading, the user may wait for the transfer to be suspended (GPDMA_CxSR.SUSPF = 1).

Bit 15 Reserved, must be kept at reset value.

Bit 14 **TOF**: trigger overrun flag
 0: no trigger overrun event
 1: a trigger overrun event occurred

Bit 13 **SUSPF**: completed suspension flag
 0: no completed suspension event
 1: a completed suspension event occurred

Bit 12 **USEF**: user setting error flag
 0: no user setting error event
 1: a user setting error event occurred

Bit 11 **ULEF**: update link transfer error flag
 0: no update link transfer error event
 1: a master bus error event occurred while updating a linked-list register from memory

Bit 10 **DTEF**: data transfer error flag
 0: no data transfer error event
 1: a master bus error event occurred on a data transfer

Bit 9 **HTF**: half transfer flag
 0: no half transfer event
 1: a half transfer event occurred
 A half transfer event is either a half block transfer or a half 2D/repeated block transfer, depending on the transfer complete event mode (GPDMA_CxTR2.TCEM[1:0]).
 A half block transfer occurs when half of the bytes of the source block size (rounded up integer of GPDMA_CxBR1.BNDT[15:0]/2) has been transferred to the destination.
 A half 2D/repeated block transfer occurs when half of the repeated blocks (rounded up integer of (GPDMA_CxBR1.BRC[10:0] + 1) / 2)) has been transferred to the destination.

Bit 8 **TCF**: transfer complete flag
 0: no transfer complete event
 1: a transfer complete event occurred
 A transfer complete event is either a block transfer complete, a 2D/repeated block transfer complete, or a LLI transfer complete including the upload of the next LLI if any, or the full linked-list completion, depending on the transfer complete event mode (GPDMA_CxTR2.TCEM[1:0]).

Bits 7:1 Reserved, must be kept at reset value.

Bit 0 **IDLEF**: idle flag
 0: channel not in idle state
 1: channel in idle state
 This idle flag is deasserted by hardware when the channel is enabled (GPDMA_CxCR.EN = 1) with a valid channel configuration (no USEF to be immediately reported).
 This idle flag is asserted after hard reset or by hardware when the channel is back in idle state (in suspended or disabled state).

16.8.9 GPDMA channel x control register (GPDMA_CxCR)

Address offset: $0x64 + 0x80 * x$ ($x = 0$ to 7)

Reset value: 0x0000 0000

This register is secure or nonsecure depending on the secure state of channel x (GPDMA_SECCFGR.SECx), and privileged or unprivileged, depending on the privileged state of the channel x (GPDMA_PRIVCFGR.PRIVx).

This register is used to control a channel (activate, suspend, abort or disable it).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PRIO[1:0]		Res.	Res.	Res.	Res.	LAP	LSM
								rw	rw					rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	TOIE	SUSP E	USEIE	ULEIE	DTEIE	HTIE	TCIE	Res.	Res.	Res.	Res.	Res.	SUSP	RESET	EN
	rw	rw	rw	rw	rw	rw	rw						rw	w	rw

Bits 31:24 Reserved, must be kept at reset value.

Bits 23:22 **PRIO[1:0]**: priority level of the channel x GPDMA transfer versus others

- 00: low priority, low weight
- 01: low priority, mid weight
- 10: low priority, high weight
- 11: high priority

Note: This bit must be written when EN = 0. This bit is read-only when EN = 1.

Bits 21:18 Reserved, must be kept at reset value.

Bit 17 **LAP**: linked-list allocated port

This bit is used to allocate the master port for the update of the GPDMA linked-list registers from the memory.

- 0: port 0 (AHB) allocated
- 1: port 1 (AHB) allocated

Note: This bit must be written when EN = 0. This bit is read-only when EN = 1.

Bit 16 **LSM**: Link step mode

0: channel executed for the full linked-list and completed at the end of the last LLI (GPDMA_CxLLR = 0). The 16 low-significant bits of the link address are null (LA[15:0] = 0) and all the update bits are null (UT1 = UB1 = UT2 = USA = UDA = ULL = 0 and UT3 = UB2 = 0). Then GPDMA_CxBR1.BNDT[15:0] = 0 and GPDMA_CxBR1.BRC[10:0] = 0.

1: channel executed **once** for the current LLI

First the (possible 1D/repeated) block transfer is executed as defined by the current internal register file until GPDMA_CxBR1.BNDT[15:0] = 0 and GPDMA_CxBR1.BRC[10:0] = 0.

Secondly the next linked-list data structure is conditionally uploaded from memory as defined by GPDMA_CxLLR. Then channel execution is completed.

Note: This bit must be written when EN = 0. This bit is read-only when EN = 1.

Bit 15 Reserved, must be kept at reset value.

Bit 14 **TOIE**: trigger overrun interrupt enable

- 0: interrupt disabled
- 1: interrupt enabled

Bit 13 **SUSPIE**: completed suspension interrupt enable

- 0: interrupt disabled
- 1: interrupt enabled

Bit 12 **USEIE**: user setting error interrupt enable

- 0: interrupt disabled
- 1: interrupt enabled

Bit 11 **ULEIE**: update link transfer error interrupt enable

- 0: interrupt disabled
- 1: interrupt enabled

Bit 10 **DTEIE**: data transfer error interrupt enable

- 0: interrupt disabled
- 1: interrupt enabled

Bit 9 **HTIE**: half transfer complete interrupt enable

- 0: interrupt disabled
- 1: interrupt enabled

Bit 8 **TCIE**: transfer complete interrupt enable

0: interrupt disabled

1: interrupt enabled

Bits 7:3 Reserved, must be kept at reset value.

Bit 2 **SUSP**: suspend

Writing 1 into the field RESET (bit 1) causes the hardware to de-assert this bit, whatever is written into this bit 2. Else:

Software must write 1 in order to suspend an active channel (channel with an ongoing GPDMA transfer over its master ports).

The software must write 0 in order to resume a suspended channel, following the programming sequence detailed in [Figure 72](#).

0: write: resume channel, read: channel not suspended

1: write: suspend channel, read: channel suspended.

Bit 1 **RESET**: reset

This bit is write only. Writing 0 has no impact. Writing 1 implies the reset of the following: the FIFO, the channel internal state, SUSP and EN bits (whatever is written receptively in bit 2 and bit 0).

The reset is effective when the channel is in steady state, meaning one of the following:

- active channel in suspended state (GPDMA_CxSR.SUSPF = 1 and GPDMA_CxSR.IDLEF = GPDMA_CxCR.EN = 1)

- channel in disabled state (GPDMA_CxSR.IDLEF = 1 and GPDMA_CxCR.EN = 0).

After writing a RESET, to continue using this channel, the user must explicitly reconfigure the channel including the hardware-modified configuration registers (GPDMA_CxBR1, GPDMA_CxSAR, and GPDMA_CxDAR) before enabling again the channel (see the programming sequence in [Figure 73](#)).

0: no channel reset

1: channel reset

Bit 0 **EN**: enable

Writing 1 into the field RESET (bit 1) causes the hardware to de-assert this bit, whatever is written into this bit 0. Else:

this bit is deasserted by hardware when there is a transfer error (master bus error or user setting error) or when there is a channel transfer complete (channel ready to be configured, for example if LSM = 1 at the end of a single execution of the LLI).

Else, this bit can be asserted by software.

Writing 0 into this EN bit is ignored.

0: write: ignored, read: channel disabled

1: write: enable channel, read: channel enabled

16.8.10 GPDMA channel x transfer register 1 (GPDMA_CxTR1)

Address offset: $0x90 + 0x80 * x$ ($x = 0$ to 7)

Reset value: 0x0000 0000

This register is secure or nonsecure depending on the secure state of channel x (GPDMA_SECCFGR.SECx) except for secure DSEC and SSEC, privileged or non-privileged, depending on the privileged state of the channel x in GPDMA_PRIVCFGR.PRIVx.

This register controls the transfer of a channel x.

This register must be written when GPDMA_CxCR.EN = 0.

This register is read-only when GPDMA_CxCR.EN = 1.

This register must be written when the channel is completed. Then the hardware has deasserted GPDMA_CxCR.EN). A channel transfer can be completed and programmed at different levels: block, 2D/repeated block, LLI or full linked-list.

In linked-list mode, during the link transfer, this register is automatically updated by GPDMA from the memory if GPDMA_CxLLR.UT1 = 1.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DSEC	DAP	Res.	Res.	DHX	DBX	DBL_1[5:0]						DINC	Res.	DDW_LOG2[1:0]	
rw	rw			rw	rw	rw	rw	rw	rw	rw	rw	rw		rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SSEC	SAP	SBX	PAM[1:0]		Res.	SBL_1[5:0]						SINC	Res.	SDW_LOG2[1:0]	
rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw		rw	rw

Bit 31 **DSEC**: security attribute of the GPDMA transfer to the destination

If GPDMA_SECCFGR.SECx = 1 and the access is secure:

0: GPDMA transfer nonsecure

1: GPDMA transfer secure

This is a secure register bit. This bit can only be read by a secure software. This bit must be written by a secure software when GPDMA_SECCFGR.SECx = 1. A secure write is ignored when GPDMA_SECCFGR.SECx = 0.

When GPDMA_SECCFGR.SECx is deasserted, this DSEC bit is also deasserted by hardware (on a secure reconfiguration of the channel as nonsecure), and the GPDMA transfer to the destination is nonsecure.

Bit 30 **DAP**: destination allocated port

This bit is used to allocate the master port for the destination transfer

0: port 0 (AHB) allocated

1: port 1 (AHB) allocated

Note: This bit must be written when EN = 0. This bit is read-only when EN = 1.

Bits 29:28 Reserved, must be kept at reset value.

Bit 27 **DHX**: destination half-word exchange

If the destination data size is shorter than a word, this bit is ignored.

If the destination data size is a word:

0: no halfword-based exchanged within word

1: the two consecutive (post PAM) half-words are exchanged in each destination word.

Bit 26 **DBX**: destination byte exchange

If the destination data size is a byte, this bit is ignored.

If the destination data size is not a byte:

0: no byte-based exchange within half-word

1: the two consecutive (post PAM) bytes are exchanged in each destination half-word.

Bits 25:20 **DBL_1[5:0]**: destination burst length minus 1, between 0 and 63

The burst length unit is one data named beat within a burst. If **DBL_1[5:0]** = 0, the burst can be named as single. Each data/beat has a width defined by the destination data width **DDW_LOG2[1:0]**.

Note: If a burst transfer crossed a 1-Kbyte address boundary on a AHB transfer, the GPDMA modifies and shortens the programmed burst into singles or bursts of lower length, to be compliant with the AHB protocol.

If a burst transfer is of length greater than the FIFO size of the channel x, the GPDMA modifies and shortens the programmed burst into singles or bursts of lower length, to be compliant with the FIFO size. Transfer performance is lower, with GPDMA re-arbitration between effective and lower singles/bursts, but the data integrity is guaranteed.

Bit 19 **DINC**: destination incrementing burst

0: fixed burst

1: contiguously incremented burst

The destination address, pointed by **GPDMA_CxDAR**, is kept constant after a burst beat/single transfer, or is incremented by the offset value corresponding to a contiguous data after a burst beat/single transfer.

Bit 18 Reserved, must be kept at reset value.

Bits 17:16 **DDW_LOG2[1:0]**: binary logarithm of the destination data width of a burst, in bytes

00: byte

01: half-word (2 bytes)

10: word (4 bytes)

11: user setting error reported and no transfer issued

*Note: A destination burst transfer must have an aligned address with its data width (start address **GPDMA_CxDAR[2:0]** and if present address offset **GPDMA_CxTR3.DAO[2:0]**, versus **DDW_LOG2[1:0]**). Otherwise a user setting error is reported and no transfer is issued.*

*When configured in packing mode (**PAM[1]** = 1 and destination data width different from source data width), a source block size must be a multiple of the destination data width (see **GPDMA_CxBR1.BNDT[2:0]** versus **DDW_LOG2[1:0]**). Else a user setting error is reported and none transfer is issued.*

Setting a 8-byte data width causes a user setting error to be reported and none transfer is issued.

Bit 15 **SSEC**: security attribute of the GPDMA transfer from the source

If **GPDMA_SECCFGR.SECx** = 1 and the access is secure:

0: GPDMA transfer nonsecure

1: GPDMA transfer secure

This is a secure register bit. This bit can only be read by a secure software. This bit must be written by a secure software when **GPDMA_SECCFGR.SECx** = 1. A secure write is ignored when **GPDMA_SECCFGR.SECx** = 0.

When **GPDMA_SECCFGR.SECx** is deasserted, this **SSEC** bit is also deasserted by hardware (on a secure reconfiguration of the channel as nonsecure), and the GPDMA transfer from the source is nonsecure.

Bit 14 **SAP**: source allocated port

This bit is used to allocate the master port for the source transfer

0: port 0 (AHB) allocated

1: port 1 (AHB) allocated

*Note: This bit must be written when **EN** = 0. This bit is read-only when **EN** = 1.*

Bit 13 **SBX**: source byte exchange within the unaligned half-word of each source word
 If the source data width is shorter than a word, this bit is ignored.
 If the source data width is a word:
 0: no byte-based exchange within the unaligned half-word of each source word
 1: the two consecutive bytes within the unaligned half-word of each source word are exchanged.

Bits 12:11 **PAM[1:0]**: padding/alignment mode
 If $DDW_LOG2[1:0] = SDW_LOG2[1:0]$: if the data width of a burst destination transfer is equal to the data width of a burst source transfer, these bits are ignored.
 Else, in the following enumerated values, the condition PAM_1 is when destination data width is higher than source data width, and the condition PAM_2 is when source data width is higher than destination data width.
 Condition: PAM_1
 00: source data is transferred as right aligned, padded with 0s up to the destination data width
 01: source data is transferred as right aligned, sign extended up to the destination data width
 10-11: successive source data are FIFO queued and packed at the destination data width, in a left (LSB) to right (MSB) order (named little endian), before a destination transfer
 Condition: PAM_2
 00: source data is transferred as right aligned, left-truncated down to the destination data width
 01: source data is transferred as left-aligned, right-truncated down to the destination data width
 10-11: source data is FIFO queued and unpacked at the destination data width, to be transferred in a left (LSB) to right (MSB) order (named little endian) to the destination
Note: If the transfer from the source peripheral is configured with peripheral flow-control mode ($SWREQ = 0$ and $PFREQ = 1$ and $DREQ = 0$), and if the destination data width > the source data width, packing is not supported.

Bit 10 Reserved, must be kept at reset value.

Bits 9:4 **SBL_1[5:0]**: source burst length minus 1, between 0 and 63
 The burst length unit is one data named beat within a burst. If $SBL_1[5:0] = 0$, the burst can be named as single. Each data/beat has a width defined by the destination data width $SDW_LOG2[1:0]$.
Note: If a burst transfer crossed a 1-Kbyte address boundary on a AHB transfer, the GPDMA modifies and shortens the programmed burst into singles or bursts of lower length, to be compliant with the AHB protocol.
If a burst transfer is of length greater than the FIFO size of the channel x, the GPDMA modifies and shortens the programmed burst into singles or bursts of lower length, to be compliant with the FIFO size. Transfer performance is lower, with GPDMA re-arbitration between effective and lower singles/bursts, but the data integrity is guaranteed.

Bit 3 **SINC**: source incrementing burst
 0: fixed burst
 1: contiguously incremented burst
 The source address, pointed by GPDMA_CxSAR, is kept constant after a burst beat/single transfer or is incremented by the offset value corresponding to a contiguous data after a burst beat/single transfer.

Bit 2 Reserved, must be kept at reset value.

Bits 1:0 **SDW_LOG2[1:0]**: binary logarithm of the source data width of a burst in bytes

00: byte

01: half-word (2 bytes)

10: word (4 bytes)

11: user setting error reported and no transfer issued

Note: A source block size must be a multiple of the source data width (GPDMA_CxBR1.BNDT[2:0] versus SDW_LOG2[1:0]). Otherwise, a user setting error is reported and no transfer is issued.

A source burst transfer must have an aligned address with its data width (start address GPDMA_CxSAR[2:0] versus SDW_LOG2[1:0]). Otherwise, a user setting error is reported and none transfer is issued.

Setting a 8-byte data width causes a user setting error to be reported and no transfer is issued.

16.8.11 GPDMA channel x transfer register 2 (GPDMA_CxTR2)

Address offset: $0x94 + 0x80 * x$ ($x = 0$ to 7)

Reset value: 0x0000 0000

This register is secure or nonsecure depending on the secure state of channel x (GPDMA_SECCFGR.SECx), and privileged or unprivileged, depending on the privileged state of channel x (GPDMA_PRIVCFGR.PRIVx).

This register controls the transfer of a channel x.

This register must be written when GPDMA_CxCR.EN = 0.

This register is read-only when GPDMA_CxCR.EN = 1.

This register must be written when the channel is completed (the hardware deasserted GPDMA_CxCR.EN). A channel transfer can be completed and programmed at different levels: block or LLI or full linked-list.

In linked-list mode, during the link transfer, this register is automatically updated by GPDMA from the memory, if GPDMA_CxLLR.UT2 = 1.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TCM[1:0]		Res.	Res.	Res.	Res.	TRIGPOL[1:0]		Res.	Res.	TRIGSEL[5:0]					
rw	rw					rw	rw			rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TRIGM[1:0]		Res.	PFREQ	BREQ	DREQ	SWREQ	Res.	REQSEL[7:0]							
rw	rw		rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30 **TCEM[1:0]**: transfer complete event mode

These bits define the transfer granularity for the transfer complete and half transfer complete events generation.

00: at block level (when GPDMA_CxBR1.BNDT[15:0] = 0): the complete (and the half) transfer event is generated at the (respectively half of the) end of a block.

Note: If the initial LLI₀ data transfer is null/void (directly programmed by the internal register file with GPDMA_CxBR1.BNDT[15:0] = 0), then neither the complete transfer event nor the half transfer event is generated.

01: channel x (x = 0 to 5), same as 00, channel x (x = 6 to 7), at 2D/repeated block level (when GPDMA_CxBR1.BRC[10:0] = 0 and GPDMA_CxBR1.BNDT[15:0] = 0). The complete (and the half) transfer event is generated at the end (respectively half of the end) of the 2D/repeated block.

Note: If the initial LLI₀ data transfer is null/void (directly programmed by the internal register file with GPDMA_CxBR1.BNDT[15:0] = 0), then neither the complete transfer event nor the half transfer event is generated.

10: at LLI level: the complete transfer event is generated at the end of the LLI transfer, including the update of the LLI if any. The half transfer event is generated at the half of the LLI data transfer. The LLI data transfer is a block transfer or a 2D/repeated block transfer for channel x (x = 6 to 7), if any data transfer.

Note: If the initial LLI₀ data transfer is null/void (directly programmed by the internal register file with GPDMA_CxBR1.BNDT[15:0] = 0), then the half transfer event is not generated, and the transfer complete event is generated when is completed the loading of the LLI₁.

11: at channel level: the complete transfer event is generated at the end of the last LLI transfer. The half transfer event is generated at the half of the data transfer of the last LLI. The last LLI updates the link address GPDMA_CxLLR.LA[15:2] to zero and clears all the GPDMA_CxLLR update bits (UT1, UT2, UB1, USA, UDA and ULL, plus UT3 and UB2). If the channel transfer is continuous/infinite, no event is generated.

Bits 29:26 Reserved, must be kept at reset value.

Bits 25:24 **TRIGPOL[1:0]**: trigger event polarity

These bits define the polarity of the selected trigger event input defined by TRIGSEL[5:0].

00: no trigger (masked trigger event)

01: trigger on the rising edge

10: trigger on the falling edge

11: same as 00

Bits 23:22 Reserved, must be kept at reset value.

Bits 21:16 **TRIGSEL[5:0]**: trigger event input selection

These bits select the trigger event input of the GPDMA transfer (as per [Section 16.3.7](#)), with an active trigger event if TRIGPOL[1:0] ≠ 00.

Bits 15:14 **TRIGM[1:0]**: trigger mode

These bits define the transfer granularity for its conditioning by the trigger.

If the channel x is enabled (GPDMA_CxCR.EN asserted) with TRIGPOL[1:0] = 00 or 11, these TRIGM[1:0] bits are ignored.

Else, a GPDMA transfer is conditioned by at least one trigger hit:

00: at block level: the first burst read of each block transfer is conditioned by one hit trigger (channel x ($x = 6$ to 7), for each block if a 2D/repeated block is configured with GPDMA_CxBR1.BRC[10:0] $\neq 0$).

01: channel x ($x = 0$ to 5), same as 00; channel x ($x = 6$ to 7), at 2D/repeated block level. The first burst read of a 2D/repeated block transfer is conditioned by one hit trigger.

10: at link level: a LLI link transfer is conditioned by one hit trigger. The LLI data transfer (if any) is not conditioned.

11: at programmed burst level: If SWREQ = 1, each programmed burst read is conditioned by one hit trigger. If SWREQ = 0, each programmed burst that is requested by the selected peripheral, is conditioned by one hit trigger.

– If the peripheral is programmed as a source (DREQ = 0) of the LLI data transfer, each programmed burst read is conditioned.

– If the peripheral is programmed as a destination (DREQ = 1) of the LLI data transfer, each programmed burst write is conditioned. The first memory burst read of a (possibly 2D/repeated) block, also named as the first ready FIFO-based source burst, is gated by the occurrence of both the hardware request and the first trigger hit.

The GPDMA monitoring of a trigger for channel x is started when the channel is enabled/loaded with a new active trigger configuration: rising or falling edge on a selected trigger (TRIGPOL[1:0] = 01 or respectively TRIGPOL[1:0] = 10).

The monitoring of this trigger is kept active during the triggered and uncompleted (data or link) transfer; and if a new trigger is detected then, this hit is internally memorized to grant the next transfer, as long as the defined rising or falling edge is not modified, and the TRIGSEL[5:0] is not modified, and the channel is enabled.

Transferring a next LLI _{$n+1$} that updates the GPDMA_CxTR2 with a new value for any of TRIGSEL[5:0] or TRIGPOL[1:0], resets the monitoring, trashing the memorized hit of the formerly defined LLI _{n} trigger.

After a first new trigger hit _{$n+1$} is memorized, if another second trigger hit _{$n+2$} is detected and if the hit _{n} triggered transfer is still not completed, hit _{$n+2$} is lost and not memorized. A trigger overrun flag is reported (GPDMA_CxSR.TOF = 1), and an interrupt is generated if enabled (GPDMA_CxCR.TOIE = 1). The channel is not automatically disabled by hardware due to a trigger overrun.

Note: When the source block size is not a multiple of the source burst size and is a multiple of the source data width, then the last programmed source burst is not completed and is internally shorten to match the block size. In this case, if TRIGM[1:0] = 11 and (SWREQ = 1 or (SWREQ = 0 and DREQ = 0)), the shortened burst transfer (by singles or/and by bursts of lower length) is conditioned once by the trigger.

When the programmed destination burst is internally shortened by singles or/and by bursts of lower length (versus FIFO size, versus block size, 1-Kbyte boundary address crossing): if the trigger is conditioning the programmed destination burst (if TRIGM[1:0] = 11 and SWREQ = 0 and DREQ = 1), this shortened destination burst transfer is conditioned once by the trigger.

Bit 13 Reserved, must be kept at reset value.

Bit 12 **PFREQ**: Hardware request in peripheral flow control mode

Important: If a given channel x is not implemented with this feature, this bit is reserved and PFREQ is not present (see Section 16.3.2 for the list of the implemented channels with this feature).

If the channel x is activated (GPDMA_CxCR.EN asserted) with SWREQ = 1 (software request for a memory-to-memory transfer), this bit is ignored. Else:

0: the selected hardware request is driven by a peripheral with a hardware request/acknowledge protocol in GPDMA control mode. The GPDMA is programmed with GPDMA_CxTR1.BNDT[15:0] and this is internally used by the hardware for the block transfer completion.

1: the selected hardware request is driven by a peripheral with a hardware request/acknowledge protocol in peripheral control mode. The GPDMA block transfer can be early completed by the peripheral itself (see Section 16.3.6 for more details).

Note: In peripheral flow control mode, there are the following restrictions:

- no 2D/repeated block support (GPDMA_CxBR1.BRC[10:0] must be set to 0)
- the peripheral must be set as the source of the transfer (DREQ = 0).
- data packing to a wider destination width is not supported (if destination width > source data width, GPDMA_CxTR1.PAM[1] must be set to 0).
- GPDMA_CxBR1.BNDT[15:0] must be programmed as a multiple of the source (peripheral) burst size.

Bit 11 **BREQ**: Block hardware request

If the channel x is activated (GPDMA_CxCR.EN asserted) with SWREQ = 1 (software request for a memory-to-memory transfer), this bit is ignored. Else:

0: the selected hardware request is driven by a peripheral with a hardware request/acknowledge protocol at a burst level.

1: the selected hardware request is driven by a peripheral with a hardware request/acknowledge protocol at a block level (see Section 16.3.4).

Bit 10 **DREQ**: destination hardware request

This bit is ignored if channel x is activated (GPDMA_CxCR.EN asserted) with SWREQ = 1 (software request for a memory-to-memory transfer). Else:

0: selected hardware request driven by a source peripheral (request signal taken into account by the GPDMA transfer scheduler over the source/read port)

1: selected hardware request driven by a destination peripheral (request signal taken into account by the GPDMA transfer scheduler over the destination/write port)

Note: If the channel x is activated (GPDMA_CxCR.EN is asserted) with SWREQ = 0 and PFREQ = 1 (peripheral hardware request with peripheral flow-control mode), any software assertion to this DREQ bit is ignored: in peripheral flow-control mode, only a peripheral-to-memory transfer is supported.

Bit 9 **SWREQ**: software request

This bit is internally taken into account when GPDMA_CxCR.EN is asserted.

0: no software request. The selected hardware request REQSEL[7:0] is taken into account.

1: software request for a memory-to-memory transfer. The default selected hardware request as per REQSEL[7:0] is ignored.

Bit 8 Reserved, must be kept at reset value.

Bits 7:0 **REQSEL[7:0]**: GPDMA hardware request selection

These bits are ignored if channel x is activated (GPDMA_CxCR.EN asserted) with SWREQ = 1 (software request for a memory-to-memory transfer). Else, the selected hardware request is internally taken into account as per [Section 16.3.4](#).

Caution: The user must not assign a same input hardware request (same REQSEL[7:0] value) to different active GPDMA channels (GPDMA_CxCR.EN = 1 and GPDMA_CxTR2.SWREQ = 0 for these channels). GPDMA is not intended to hardware support the case of simultaneous enabled channels incorrectly configured with a same hardware peripheral request signal, and there is no user setting error reporting.

16.8.12 GPDMA channel x block register 1 (GPDMA_CxBR1)

Address offset: $0x98 + 0x80 * x$ ($x = 0$ to 5)

Reset value: 0x0000 0000

This register is secure or nonsecure depending on the secure state of channel x (GPDMA_SECCFGR.SECx), and privileged or unprivileged, depending on the privileged state of channel x (GPDMA_PRIVCFGR.PRIVx).

This register controls the transfer of a channel x at a block level.

This register must be written when GPDMA_CxCR.EN = 0.

This register is read-only when GPDMA_CxCR.EN = 1.

This register must be written when channel x is completed (then the hardware has deasserted GPDMA_CxCR.EN). A channel transfer can be completed and programmed at different levels: block, or LLI or full linked-list.

In linked-list mode, during the link transfer:

- if GPDMA_CxLLR.UB1 = 1, this register is automatically updated by the GPDMA from the next LLI in memory.
- If GPDMA_CxLLR.UB1 = 0 and if there is at least one linked-list register to be updated from the next LLI in memory, this register is automatically and internally restored with the programmed value for the field BNDT[15:0].
- If all the update bits GPDMA_CxLLR.Uxx are null and if GPDMA_CxLLR.LA[15:0] ≠ 0, the current LLI is the last one and is continuously executed: this register is automatically and internally restored with the programmed value for BNDT[15:0] after each execution of this final LLI
- If GPDMA_CxLLR = 0, this register and BNDT[15:0] are kept as null, channel x is completed.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BNDT[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **BNDT[15:0]**: block number of data bytes to transfer from the source

Block size transferred from the source. When the channel is enabled, this field becomes read-only and is decremented, indicating the remaining number of data items in the current source block to be transferred. BNDT[15:0] is programmed in number of bytes, maximum source block size is 64 Kbytes -1.

Once the last data transfer is completed (BNDT[15:0] = 0):

- if GPDMA_CxLLR.UB1 = 1, this field is updated by the LLI in the memory.
- if GPDMA_CxLLR.UB1 = 0 and if there is at least one non null Uxx update bit, this field is internally restored to the programmed value.
- if all GPDMA_CxLLR.Uxx = 0 and if GPDMA_CxLLR.LA[15:0] = 0, this field is internally restored to the programmed value (infinite/continuous last LLI).
- if GPDMA_CxLLR = 0, this field is kept as zero following the last LLI data transfer.

Note: A non-null source block size must be a multiple of the source data width (BNDT[2:0] versus GPDMA_CxTR1.SDW_LOG2[1:0]). Else a user setting error is reported and no transfer is issued.

When configured in packing mode (GPDMA_CxTR1.PAM[1] = 1 and destination data width different from source data width), a non-null source block size must be a multiple of the destination data width (BNDT[2:0] versus GPDMA_CxTR1.DDW_LOG2[1:0]). Else a user setting error is reported and no transfer is issued.

16.8.13 GPDMA channel x alternate block register 1 (GPDMA_CxBR1)

Address offset: $0x98 + 0x80 * x$ ($x = 6$ to 7)

Reset value: 0x0000 0000

This register is secure or nonsecure depending on the secure state of channel x (GPDMA_SECCFGR.SECx), and privileged or unprivileged, depending on the privileged state of channel x (GPDMA_PRIVCFGR.PRIVx).

This register controls the transfer of a channel x at a block level.

This register must be written when GPDMA_CxCR.EN = 0.

This register is read-only when GPDMA_CxCR.EN = 1.

This register must be written when channel x is completed (then the hardware has deasserted GPDMA_CxCR.EN). A channel transfer can be completed and programmed at different levels: block, or LLI or full linked-list.

In linked-list mode, during the link transfer:

- if GPDMA_CxLLR.UB1 = 1, this register is automatically updated by the GPDMA from the next LLI in memory.
- If GPDMA_CxLLR.UB1 = 0 and if there is at least one linked-list register to be updated from the next LLI in memory, this register is automatically and internally restored with the programmed value for the fields BNDT[15:0] and BRC[10:0].
- If all the update bits GPDMA_CxLLR.Uxx are null and if GPDMA_CxLLR.LA[15:0] ≠ 0, the current LLI is the last one and is continuously executed: this register is

automatically and internally restored with the programmed value for the fields BNDT[15:0] and BRC[10:0] after each execution of this final LLI

- If GPDMA_CxLLR = 0, BNDT[15:0] and BRC[10:0] are kept as null, channel x is completed.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BRDDE C	BRSDE C	DDEC	SDEC	Res.	BRC[10:0]										
rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BNDT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 BRDDEC: Block repeat destination address decrement

0: at the end of a block transfer, the GPDMA_CxDAR register is updated by adding the programmed offset GPDMA_CxBR2.BRDAO to the current GPDMA_CxDAR value (current destination address)

1: at the end of a block transfer, the GPDMA_CxDAR register is updated by subtracting the programmed offset GPDMA_CxBR2.BRDAO from the current GPDMA_CxDAR value (current destination address)

Note: On top of this increment/decrement (depending on BRDDEC), GPDMA_CxDAR is in the same time also updated by the increment/decrement (depending on DDEC) of the GPDMA_CxTR3.DAO value, as it is usually done at the end of each programmed burst transfer.

Bit 30 BRSDEC: Block repeat source address decrement

0: at the end of a block transfer, the GPDMA_CxSAR register is updated by adding the programmed offset GPDMA_CxBR2.BRSAO to the current GPDMA_CxSAR value (current source address)

1: at the end of a block transfer, the GPDMA_CxSAR register is updated by subtracting the programmed offset GPDMA_CxBR2.BRSAO from the current GPDMA_CxSAR value (current source address)

Note: On top of this increment/decrement (depending on BRSDEC), GPDMA_CxSAR is in the same time also updated by the increment/decrement (depending on SDEC) of the GPDMA_CxTR3.SAO value, as it is done after any programmed burst transfer.

Bit 29 DDEC: destination address decrement

0: At the end of a programmed burst transfer to the destination, the GPDMA_CxDAR register is updated by adding the programmed offset GPDMA_CxTR3.DAO to the current GPDMA_CxDAR value (current destination address)

1: At the end of a programmed burst transfer to the destination, the GPDMA_CxDAR register is updated by subtracting the programmed offset GPDMA_CxTR3.DAO to the current GPDMA_CxDAR value (current destination address)

Bit 28 SDEC: source address decrement

0: At the end of a programmed burst transfer from the source, the GPDMA_CxSAR register is updated by adding the programmed offset GPDMA_CxTR3.SAO to the current GPDMA_CxSAR value (current source address)

1: At the end of a programmed burst transfer from the source, the GPDMA_CxSAR register is updated by subtracting the programmed offset GPDMA_CxTR3.SAO to the current GPDMA_CxSAR value (current source address)

Bit 27 Reserved, must be kept at reset value.

Bits 26:16 **BRC[10:0]**: Block repeat counter

This field contains the number of repetitions of the current block (0 to 2047).

When the channel is enabled, this field becomes read-only. After decrements, this field indicates the remaining number of blocks, excluding the current one. This counter is hardware decremented for each completed block transfer.

Once the last block transfer is completed ($BRC[10:0] = BNDT[15:0] = 0$):

- If $GPDMA_CxLLR.UB1 = 1$, all $GPDMA_CxBR1$ fields are updated by the next LLI in the memory.
- If $GPDMA_CxLLR.UB1 = 0$ and if there is at least one not null Uxx update bit, this field is internally restored to the programmed value.
- if all $GPDMA_CxLLR.Uxx = 0$ and if $GPDMA_CxLLR.LA[15:0] \neq 0$, this field is internally restored to the programmed value (infinite/continuous last LLI).
- if $GPDMA_CxLLR = 0$, this field is kept as zero following the last LLI and data transfer.

Bits 15:0 **BNDT[15:0]**: block number of data bytes to transfer from the source

Block size transferred from the source. When the channel is enabled, this field becomes read-only and is decremented, indicating the remaining number of data items in the current source block to be transferred. $BNDT[15:0]$ is programmed in number of bytes, maximum source block size is 64 Kbytes -1.

Once the last data transfer is completed ($BNDT[15:0] = 0$):

- if $GPDMA_CxLLR.UB1 = 1$, this field is updated by the LLI in the memory.
- if $GPDMA_CxLLR.UB1 = 0$ and if there is at least one not null Uxx update bit, this field is internally restored to the programmed value.
- if all $GPDMA_CxLLR.Uxx = 0$ and if $GPDMA_CxLLR.LA[15:0] \neq 0$, this field is internally restored to the programmed value (infinite/continuous last LLI).
- if $GPDMA_CxLLR = 0$, this field is kept as zero following the last LLI data transfer.

Note: A non-null source block size must be a multiple of the source data width ($BNDT[2:0]$ versus $GPDMA_CxTR1.SDW_LOG2[1:0]$). Else a user setting error is reported and no transfer is issued.

When configured in packing mode ($GPDMA_CxTR1.PAM[1] = 1$ and destination data width different from source data width), a non-null source block size must be a multiple of the destination data width ($BNDT[2:0]$ versus $GPDMA_CxTR1.DDW_LOG2[1:0]$). Else a user setting error is reported and no transfer is issued.

16.8.14 GPDMA channel x source address register (GPDMA_CxSAR)

Address offset: $0x9C + 0x80 * x$ ($x = 0$ to 7)

Reset value: $0x0000\ 0000$

This register is secure or nonsecure depending on the secure state of channel x (GPDMA_SECCFGR.SECx), and privileged or unprivileged, depending on the privileged state of channel x (GPDMA_PRIVCFGR.PRIVx).

This register configures the source start address of a transfer.

This register must be written when GPDMA_CxCR.EN = 0.

This register is read-only when GPDMA_CxCR.EN = 1, and continuously updated by hardware, in order to reflect the address of the next burst transfer from the source.

This register must be written when the channel is completed (then the hardware has deasserted GPDMA_CxCR.EN). A channel transfer can be completed and programmed at different levels: block, 2D/repeated block, LLI or full linked-list.

In linked-list mode, during the link transfer, this register is automatically updated by the GPDMA from the memory if GPDMA_CxLLR.USA = 1.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SA[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SA[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **SA[31:0]**: source address

This field is the pointer to the address from which the next data is read.

During the channel activity, depending on the source addressing mode (GPDMA_CxTR1.SINC), this field is kept fixed or incremented by the data width (GPDMA_CxTR1.SDW_LOG2[1:0]) after each burst source data, reflecting the next address from which data is read.

During the channel activity, this address is updated after each completed source burst, consequently to:

- the programmed source burst; either in fixed addressing mode or in contiguous-data incremented mode. If contiguously incremented (GPDMA_CxTR1.SINC = 1), then the additional address offset value is the programmed burst size, as defined by GPDMA_CxTR1.SBL_1[5:0] and GPDMA_CxTR1.SDW_LOG2[1:0]
- the additional source incremented/decremented offset value as programmed by GPDMA_CxBR1.SDEC and GPDMA_CxTR3.SAO[12:0].
- once/if completed source block transfer, for a channel x with 2D addressing capability (x = 6 to 7). additional block repeat source incremented/decremented offset value as programmed by GPDMA_CxBR1.BRSDEC and GPDMA_CxBR2.BRSAO[15:0]

In linked-list mode, after a LLI data transfer is completed, this register is automatically updated by GPDMA from the memory, provided the LLI is set with GPDMA_CxLLR.USA = 1.

Note: A source address must be aligned with the programmed data width of a source burst (SA[2:0] versus GPDMA_CxTR1.SDW_LOG2[1:0]). Else, a user setting error is reported and no transfer is issued.

When the source block size is not a multiple of the source burst size and is a multiple of the source data width, the last programmed source burst is not completed and is internally shorten to match the block size. In this case, the additional GPDMA_CxTR3.SAO[12:0] is not applied.

16.8.15 GPDMA channel x destination address register (GPDMA_CxDAR)

Address offset: $0xA0 + 0x80 * x$ ($x = 0$ to 7)

Reset value: $0x0000\ 0000$

This register is secure or nonsecure depending on the secure state of channel x (GPDMA_SECCFGR.SECx), and privileged or unprivileged, depending on the privileged state of channel x (GPDMA_PRIVCFGR.PRIVx).

This register configures the destination start address of a transfer.

This register must be written when GPDMA_CxCR.EN = 0.

This register is read-only when GPDMA_CxCR.EN = 1, and continuously updated by hardware, in order to reflect the address of the next burst transfer to the destination.

This register must be written when the channel is completed (then the hardware has deasserted GPDMA_CxCR.EN). A channel transfer can be completed and programmed at different levels: block, 2D/repeated block, LLI or full linked-list.

In linked-list mode, during the link transfer, this register is automatically updated by GPDMA from the memory if GPDMA_CxLLR.UDA = 1.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DA[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DA[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **DA[31:0]**: destination address

This field is the pointer to the address from which the next data is written.

During the channel activity, depending on the destination addressing mode (GPDMA_CxTR1.DINC), this field is kept fixed or incremented by the data width (GPDMA_CxTR1.DDW_LOG2[1:0]) after each burst destination data, reflecting the next address from which data is written.

During the channel activity, this address is updated after each completed destination burst, consequently to:

- the programmed destination burst; either in fixed addressing mode or in contiguous-data incremented mode. If contiguously incremented (GPDMA_CxTR1.DINC = 1), then the additional address offset value is the programmed burst size, as defined by GPDMA_CxTR1.DBL_1[5:0] and GPDMA_CxTR1.DDW_LOG2[1:0]
- the additional destination incremented/decremented offset value as programmed by GPDMA_CxBR1.DDEC and GPDMA_CxTR3.DAO[12:0].
- once/if completed destination block transfer, for a channel x with 2D addressing capability ($x = 6$ to 7), the additional block repeat destination incremented/decremented offset value as programmed by GPDMA_CxBR1.BRDDEC and GPDMA_CxBR2.BRDAO[15:0]

In linked-list mode, after a LLI data transfer is completed, this register is automatically updated by the GPDMA from the memory, provided the LLI is set with GPDMA_CxLLR.UDA = 1.

Note: A destination address must be aligned with the programmed data width of a destination burst (DA[2:0] versus GPDMA_CxTR1.DDW_LOG2[1:0]). Else, a user setting error is reported and no transfer is issued.

16.8.16 GPDMA channel x transfer register 3 (GPDMA_CxTR3)

Address offset: $0xA4 + 0x80 * x$ ($x = 6$ to 7)

Reset value: 0x0000 0000

This register is secure or nonsecure depending on the secure state of channel x (GPDMA_SECCFGR.SECx), and privileged or unprivileged, depending on the privileged state of channel x (GPDMA_PRIVCFGR.PRIVx).

This register controls the transfer of a channel x.

This register must be written when GPDMA_CxCR.EN = 0.

This register is read-only when GPDMA_CxCR.EN = 1.

This register must be written when the channel is completed (then the hardware has deasserted GPDMA_CxCR.EN). A channel transfer can be completed and programmed at different levels: block or LLI or full linked-list.

In linked-list mode, during the link transfer, this register is automatically updated by the GPDMA from the memory if GPDMA_CxLLR.UT3 = 1.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	DAO[12:0]												
			rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	SAO[12:0]												
			rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:29 Reserved, must be kept at reset value.

Bits 28:16 **DAO[12:0]**: destination address offset increment

The destination address, pointed by GPDMA_CxDAR, is incremented or decremented (depending on GPDMA_CxBR1.DDEC) by this offset DAO[12:0] for each programmed destination burst. This offset is not including and is added to the programmed burst size when the completed burst is addressed in incremented mode (GPDMA_CxTR1.DINC = 1).

Note: A destination address offset must be aligned with the programmed data width of a destination burst (DAO[2:0] versus GPDMA_CxTR1.DDW_LOG2[1:0]). Else, a user setting error is reported and no transfer is issued.

Bits 15:13 Reserved, must be kept at reset value.

Bits 12:0 **SAO[12:0]**: source address offset increment

The source address, pointed by GPDMA_CxSAR, is incremented or decremented (depending on GPDMA_CxBR1.SDEC) by this offset SAO[12:0] for each programmed source burst. This offset is not including and is added to the programmed burst size when the completed burst is addressed in incremented mode (GPDMA_CxTR1.SINC = 1).

Note: A source address offset must be aligned with the programmed data width of a source burst (SAO[2:0] versus GPDMA_CxTR1.SDW_LOG2[1:0]). Else a user setting error is reported and none transfer is issued.

When the source block size is not a multiple of the destination burst size, and is a multiple of the source data width, then the last programmed source burst is not completed and is internally shorten to match the block size. In this case, the additional GPDMA_CxTR3.SAO[12:0] is not applied.

16.8.17 GPDMA channel x block register 2 (GPDMA_CxBR2)

Address offset: $0xA8 + 0x80 * x$ ($x = 6$ to 7)

Reset value: 0x0000 0000

This register is secure or nonsecure depending on the secure state of channel x (GPDMA_SECCFGR.SECx), and privileged or unprivileged, depending on the privileged state of channel x (GPDMA_PRIVCFGR.PRIVx).

This register controls the transfer of a channel x at a 2D/repeated block level.

This register must be written when GPDMA_CxCR.EN = 0.

This register is read-only when GPDMA_CxCR.EN = 1.

This register must be written when the channel is completed (then the hardware has deasserted GPDMA_CxCR.EN). A channel transfer can be completed and programmed at different levels: block, 2D/repeated block, LLI or full linked-list.

In linked-list mode, during the link transfer, this register is automatically updated by the GPDMA from the memory if GPDMA_CxLLR.UB2 = 1.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BRDAO[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BRSAO[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:16 **BRDAO[15:0]**: Block repeated destination address offset

For a channel with 2D addressing capability, this field is used to update (by addition or subtraction depending on GPDMA_CxBR1.BRDDEC) the current destination address (GPDMA_CxDAR) at the end of a block transfer.

Note: A block repeated destination address offset must be aligned with the programmed data width of a destination burst (BRDAO[2:0] versus GPDMA_CxTR1.DDW_LOG2[1:0]). Else a user setting error is reported and no transfer is issued. BRDAO[15:0] must be set to 0 in peripheral flow-control mode (if GPDMA_CxTR2.PFREQ = 1).

Bits 15:0 **BRSAO[15:0]**: Block repeated source address offset

For a channel with 2D addressing capability, this field is used to update (by addition or subtraction depending on GPDMA_CxBR1.BRSDEC) the current source address (GPDMA_CxSAR) at the end of a block transfer.

Note: A block repeated source address offset must be aligned with the programmed data width of a source burst (BRSAO[2:0] versus GPDMA_CxTR1.SDW_LOG2[1:0]). Else a user setting error is reported and no transfer is issued. BRSAO[15:0] must be set to 0 in peripheral flow-control mode (if GPDMA_CxTR2.PFREQ = 1).

16.8.18 GPDMA channel x linked-list address register (GPDMA_CxLLR)

Address offset: $0xCC + 0x80 * x$ ($x = 0$ to 5)

Reset value: 0x0000 0000

This register is secure or nonsecure depending on the secure state of channel x (GPDMA_SECCFGR.SECx), and privileged or unprivileged, depending on the privileged state of channel x (GPDMA_PRIVCFGR.PRIVx).

This register configures the data structure of the next LLI in the memory and its address pointer. A channel transfer is completed when this register is null.

This register must be written when GPDMA_CxCR.EN = 0.

This register is read-only when GPDMA_CxCR.EN = 1.

This register must be written when the channel is completed (then the hardware has deasserted GPDMA_CxCR.EN). A channel transfer can be completed and programmed at different levels: block or LLI or full linked-list.

In linked-list mode, during the link transfer, this register is automatically updated by the GPDMA from the memory if GPDMA_CxLLR.ULL = 1.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
UT1	UT2	UB1	USA	UDA	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ULL
rw	rw	rw	rw	rw											rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LA[15:2]														Res.	Res.
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		

Bit 31 **UT1**: Update GPDMA_CxTR1 from memory

This bit controls the update of GPDMA_CxTR1 from the memory during the link transfer.

0: no GPDMA_CxTR1 update

1: GPDMA_CxTR1 update

Bit 30 **UT2**: Update GPDMA_CxTR2 from memory

This bit controls the update of GPDMA_CxTR2 from the memory during the link transfer.

0: no GPDMA_CxTR2 update

1: GPDMA_CxTR2 update

Bit 29 **UB1**: Update GPDMA_CxBR1 from memory

This bit controls the update of GPDMA_CxBR1 from the memory during the link transfer.

If UB1 = 0 and if GPDMA_CxLLR ≠ 0, the linked-list is not completed.

GPDMA_CxBR1.BNDT[15:0] is then restored to the programmed value after data transfer is completed and before the link transfer.

0: no GPDMA_CxBR1 update from memory (GPDMA_CxBR1.BNDT[15:0] restored if any link transfer)

1: GPDMA_CxBR1 update

Bit 28 **USA**: update GPDMA_CxSAR from memory

This bit controls the update of GPDMA_CxSAR from the memory during the link transfer.

0: no GPDMA_CxSAR update

1: GPDMA_CxSAR update

Bit 27 **UDA**: Update GPDMA_CxDAR register from memory

This bit is used to control the update of GPDMA_CxDAR from the memory during the link transfer.

0: no GPDMA_CxDAR update

1: GPDMA_CxDAR update

Bits 26:17 Reserved, must be kept at reset value.

Bit 16 **ULL**: Update GPDMA_CxLLR register from memory

This bit is used to control the update of GPDMA_CxLLR from the memory during the link transfer.

0: no GPDMA_CxLLR update

1: GPDMA_CxLLR update

Bits 15:2 **LA[15:2]**: pointer (16-bit low-significant address) to the next linked-list data structure

If UT1 = UT2 = UB1 = USA = UDA = ULL = 0 and if LA[15:2] = 0, the current LLI is the last one. The channel transfer is completed without any update of the linked-list GPDMA register file.

Else, this field is the pointer to the memory address offset from which the next linked-list data structure is automatically fetched from, once the data transfer is completed, in order to conditionally update the linked-list GPDMA internal register file (GPDMA_CxTR1, GPDMA_CxTR2, GPDMA_CxBR1, GPDMA_CxSAR, GPDMA_CxDAR, and GPDMA_CxLLR).

Note: The user must program the pointer to be 32-bit aligned. The two low-significant bits are write ignored.

Bits 1:0 Reserved, must be kept at reset value.

16.8.19 GPDMA channel x alternate linked-list address register (GPDMA_CxLLR)

Address offset: $0xCC + 0x80 * x$ ($x = 6$ to 7)

Reset value: 0x0000 0000

This register is secure or nonsecure depending on the secure state of channel x (GPDMA_SECCFGR.SECx), and privileged or unprivileged, depending on the privileged state of channel x (GPDMA_PRIVCFGR.PRIVx).

This register configures the data structure of the next LLI in the memory and its address pointer. A channel transfer is completed when this register is null.

This register must be written when GPDMA_CxCR.EN = 0.

This register is read-only when GPDMA_CxCR.EN = 1.

This register must be written when the channel is completed (then the hardware has deasserted GPDMA_CxCR.EN). A channel transfer can be completed and programmed at different levels: block or LLI or full linked-list.

In linked-list mode, during the link transfer, this register is automatically updated by the GPDMA from the memory if GPDMA_CxLLR.ULL = 1.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
UT1	UT2	UB1	USA	UDA	UT3	UB2	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ULL
rw	rw	rw	rw	rw	rw	rw									rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LA[15:2]														Res.	Res.
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		

Bit 31 **UT1**: Update GPDMA_CxTR1 from memory

This bit controls the update of GPDMA_CxTR1 from the memory during the link transfer.

0: no GPDMA_CxTR1 update

1: GPDMA_CxTR1 update

Bit 30 **UT2**: Update GPDMA_CxTR2 from memory

This bit controls the update of GPDMA_CxTR2 from the memory during the link transfer.

0: no GPDMA_CxTR2 update

1: GPDMA_CxTR2 update

Bit 29 **UB1**: Update GPDMA_CxBR1 from memory

This bit controls the update of GPDMA_CxBR1 from the memory during the link transfer.

If UB1 = 0 and if GPDMA_CxLLR ≠ 0, the linked-list is not completed.

GPDMA_CxBR1.BNDT[15:0] is then restored to the programmed value after data transfer is completed and before the link transfer.

0: no GPDMA_CxBR1 update from memory (GPDMA_CxBR1.BNDT[15:0] restored if any link transfer)

1: GPDMA_CxBR1 update

Bit 28 **USA**: update GPDMA_CxSAR from memory

This bit controls the update of GPDMA_CxSAR from the memory during the link transfer.

0: no GPDMA_CxSAR update

1: GPDMA_CxSAR update

Bit 27 **UDA**: Update GPDMA_CxDAR register from memory

This bit is used to control the update of GPDMA_CxDAR from the memory during the link transfer.

0: no GPDMA_CxDAR update

1: GPDMA_CxDAR update

Bit 26 **UT3**: Update GPDMA_CxTR3 from memory

This bit controls the update of GPDMA_CxTR3 from the memory during the link transfer.

0: no GPDMA_CxTR3 update

1: GPDMA_CxTR3 update

Bit 25 **UB2**: Update GPDMA_CxBR2 from memory

This bit controls the update of GPDMA_CxBR2 from the memory during the link transfer.

0: no GPDMA_CxBR2 update

1: GPDMA_CxBR2 update

Bits 24:17 Reserved, must be kept at reset value.

Bit 16 **ULL**: Update GPDMA_CxLLR register from memory

This bit is used to control the update of GPDMA_CxLLR from the memory during the link transfer.

0: no GPDMA_CxLLR update

1: GPDMA_CxLLR update

Bits 15:2 **LA[15:2]**: pointer (16-bit low-significant address) to the next linked-list data structure

If UT1 = UT2 = UB1 = USA = UDA = ULL = 0 and if LA[15:2] = 0, the current LLI is the last one. The channel transfer is completed without any update of the linked-list GPDMA register file.

Else, this field is the pointer to the memory address offset from which the next linked-list data structure is automatically fetched from, once the data transfer is completed, in order to conditionally update the linked-list GPDMA internal register file (GPDMA_CxTR1, GPDMA_CxTR2, GPDMA_CxBR1, GPDMA_CxSAR, GPDMA_CxDAR, and GPDMA_CxLLR).

Note: The user must program the pointer to be 32-bit aligned. The two low-significant bits are write ignored.

Bits 1:0 Reserved, must be kept at reset value.

16.8.20 GPDMA register map

Table 145. GPDMA register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	GPDMA_SECCFGR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	SEC7	SEC6	SEC5	SEC4	SEC3	SEC2	SEC1	SEC0
	Reset value																									0	0	0	0	0	0	0	0
0x04	GPDMA_PRIVCFGR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	PRIV7	PRIV6	PRIV5	PRIV4	PRIV3	PRIV2	PRIV1	PRIV0
	Reset value																									0	0	0	0	0	0	0	0
0x08	GPDMA_RCFGLOCKR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	LOCK7	LOCK6	LOCK5	LOCK4	LOCK3	LOCK2	LOCK1	LOCK0
	Reset value																									0	0	0	0	0	0	0	0
0x0C	GPDMA_MISR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	MIS7	MIS6	MIS5	MIS4	MIS3	MIS2	MIS1	MIS0
	Reset value																									0	0	0	0	0	0	0	0

Table 145. GPDMA register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0x10	GPDMA_SMISR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	MIS7	MIS6	MIS5	MIS4	MIS3	MIS2	MIS1	MIS0		
	Reset value																									0	0	0	0	0	0	0	0		
0x14 - 0x4C	Reserved	Reserved																																	
0x50+0x80 * x (x=0 to 7)	GPDMA_CxLBAR	LBA[31:16]																Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0																		
0x54 to 0x58+0x80*x (x=0 to 7)	Reserved	Reserved																																	
0x5C+0x80 * x (x=0 to 7)	GPDMA_CxFCR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	TOF	SUSPF	USEF	ULEF	DTEF	HTF	TCF	Res	Res	Res	Res	Res	Res	Res			
	Reset value																		0	0	0	0	0	0	0										
0x60+ 0x80 * x (x=0 to 7)	GPDMA_CxSR	Res	Res	Res	Res	Res	Res	Res	FIFOL[7:0]								Res	TOF	SUSPF	USEF	ULEF	DTEF	HTF	TCF	Res	Res	Res	Res	Res	Res	Res	IDLEF			
	Reset value								0	0	0	0	0	0	0	0	0		0	0	0	0	0	0	0								1		
0x64+ 0x80 * x (x=0 to 7)	GPDMA_CxCr	Res	Res	Res	Res	Res	Res	Res	PRIO[1:0]		Res	Res	Res	Res	Res	LAP	LSM	Res	TOIE	SUSPIE	USEIE	ULEIE	DTEIE	HTIE	TCIE	Res	Res	Res	Res	Res	SUSP	RESET	EN		
	Reset value								0	0						0	0		0	0	0	0	0	0	0					0	0	0	0		
0x68 to 0x8C+0x80 * x (x=0 to 7)	Reserved	Reserved																																	
0x90+0x80 * x (x=0 to 7)	GPDMA_CxTR1	DSEC	DAP	Res	Res	DXH	DBX	DBL_1[5:0]					DINC	Res	DDW_LOG2[1:0]	SSEC	SAP	SBX	PAM[1:0]		Res	SBL_1[5:0]					SINC	Res	SDW_LOG2[1:0]						
	Reset value	0	0			0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x94+0x80 * x (x=0 to 7)	GPDMA_CxTR2	TCEM[1:0]		Res	Res	Res	Res	TRIGPOL[1:0]		Res	Res	TRIGSEL[5:0]					TRIGM[1:0]		Res	PFREQ	BREQ	DREQ	SWREQ	Res	REQSEL[7:0]										
	Reset value	0	0					0	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x98+0x80 * x (x=0 to 5)	GPDMA_CxBR1	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	BNDET[15:0]																	
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x98+0x80 * x (x=6 to 7)	GPDMA_CxBR1	BRDDEC	BRSEDEC	DDEC	SDEC	Res	BRC[10:0]										BNDET[15:0]																		
	Reset value	0	0	0	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x9C+0x80 * x (x=0 to 7)	GPDMA_CxSAR	SA[31:0]																																	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0xA0+0x80 * x (x=0 to 7)	GPDMA_CxDAR	DA[31:0]																																	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0xA4+0x80 * x (x=6 to 7)	GPDMA_CxTR3	Res	Res	Res	DAO[12:0]										Res	Res	Res	SAO[12:0]																	
	Reset value				0	0	0	0	0	0	0	0	0	0	0	0	0				0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0xA8+0x80 * x (x=6 to 7)	GPDMA_CxBR2	BRDAO[15:0]															BRSAO[15:0]																		
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0xCC+0x80 * x (x=0 to 5)	GPDMA_CxLLR	UT1	UT2	UB1	USA	UDA	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	ULL	LA[15:2]											Res						
	Reset value	0	0	0	0	0											0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		Res	
0xCC+0x80 * x (x=6 to 7)	GPDMA_CxLLR	UT1	UT2	UB1	USA	UDA	UT3	UB2	Res	Res	Res	Res	Res	Res	Res	Res	ULL	LA[15:2]											Res						
	Reset value	0	0	0	0	0	0	0									0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		Res	

Refer to [Section 2.3: Memory organization](#) for the register boundary addresses.

17 Nested vectored interrupt controller (NVIC)

17.1 NVIC main features

- Up to 133 maskable interrupt channels (not including the 16 Cortex-M33 with FPU interrupt lines)
- 16 programmable priority levels (four bits of interrupt priority used)
- Low-latency exception and interrupt handling
- Power management control
- Implementation of system control registers

The NVIC and the processor core interface are closely coupled, enabling low-latency interrupt processing and efficient processing of late arriving interrupts.

The NVIC registers are banked across secure and non-secure states.

All interrupts, including the core exceptions, are managed by the NVIC.

17.2 SysTick calibration value register

The Cortex-M33 with TrustZone mainline security extension embeds two SysTick timers.

When TrustZone is activated, the following SysTick timers are available:

- SysTick, secure instance
- SysTick, non-secure instance

When TrustZone is disabled, only one SysTick timer is available.

The SysTick timer calibration value (STCALIB) is 0x3E8. It gives a reference time base of 1 ms based on a SysTick clock frequency of 1 MHz. To match the 1 ms time base for an application running at a given frequency, the SysTick reload value must be programmed as follows in the SYST_RVR register:

- When SysTick clock source is CPU clock HCLK reload value = $(HCLK \times STCALIB) - 1$
- When SysTick clock source is external clock (HCLK/8) reload value = $((HCLK / 8) \times STCALIB) - 1$

The HCLK refers to the AHB frequency value in MHz.

Example: SysTick clock source is CPU clock HCLK of 100 MHz, to match a time base of 1 ms: SysTick reload value = $(100 \times STCALIB) - 1 = 0x1869F$.

17.3 Interrupt and exception vectors

The gray rows in [Table 146](#) and [Table 147](#) describe the vectors without specific position.

Table 146. STM32H562/563/573xx vector table

Position	Priority	Type of priority	Acronym	Description	Address
-	-	-	-	Reserved	0x0000 0000
-	-4	Fixed	SettableReset	Reset	0x0000 0004
-	-2	Fixed	NMI	Non maskable interrupt. The RCC clock security system (CSS) is linked to the NMI vector.	0x0000 0008
-	-3 or -1	Fixed	Secure HardFault	Secure Hard fault	0x0000 000C
-	-1	Fixed	Non-secure HardFault	Non-secure Hard fault. All classes of fault.	0x0000 000C
-	0	Settable	MemManage	Memory management	0x0000 0010
-	1	Settable	BusFault	Pre-fetch fault, memory access fault	0x0000 0014
-	2	Settable	UsageFault	Undefined instruction or illegal state	0x0000 0018
-	3	Settable	SecureFault	Secure fault	0x0000 001C
-	-	-	-	Reserved	0x0000 0020 - 0x0000 0028
-	4	-	SVC	System service call via SWI instruction	0x0000 002C
-	5	-	Debug Monitor	Monitor	0x0000 0030
-	-	-	-	Reserved	0x0000 0034
-	6	Settable	PendSV	Pendable request for system service	0x0000 0038
-	7	Settable	SysTick	System tick timer	0x0000 003C
0	8	Settable	WWDG	Window watchdog interrupt	0x0000 0040
1	9	Settable	PVD_AVD	Power voltage monitor/ Analog voltage monitor	0x0000 0044
2	10	Settable	RTC	RTC global non-secure interrupts	0x0000 0048
3	11	Settable	RTC_S	RTC global secure interrupts	0x0000 004C
4	12	Settable	TAMP	Tamper global interrupts	0x0000 0050
5	13	Settable	RAMCFG	RAM configuration global interrupt	0x0000 0054
6	14	Settable	FLASH	Flash non-secure global interrupt	0x0000 0058
7	15	Settable	FLASH_S	Flash secure global interrupt	0x0000 005C
8	16	Settable	GTZC	GTZC global interrupt	0x0000 0060
9	17	Settable	RCC	RCC non-secure global interrupt	0x0000 0064
10	18	Settable	RCC_S	RCC secure global interrupt	0x0000 0068
11	19	Settable	EXTI0	EXTI Line0 interrupt	0x0000 006C
12	20	Settable	EXTI1	EXTI Line1 interrupt	0x0000 0070

Table 146. STM32H562/563/573xx vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
13	21	Settable	EXTI2	EXTI Line2 interrupt	0x0000 0074
14	22	Settable	EXTI3	EXTI Line3 interrupt	0x0000 0078
15	23	Settable	EXTI4	EXTI Line4 interrupt	0x0000 007C
16	24	Settable	EXTI5	EXTI Line5 interrupt	0x0000 0080
17	25	Settable	EXTI6	EXTI Line6 interrupt	0x0000 0084
18	26	Settable	EXTI7	EXTI Line7 interrupt	0x0000 0088
19	27	Settable	EXTI8	EXTI Line8 interrupt	0x0000 008C
20	28	Settable	EXTI9	EXTI Line9 interrupt	0x0000 0090
21	29	Settable	EXTI10	EXTI Line10 interrupt	0x0000 0094
22	30	Settable	EXTI11	EXTI Line11 interrupt	0x0000 0098
23	31	Settable	EXTI12	EXTI Line12 interrupt	0x0000 009C
24	32	Settable	EXTI13	EXTI Line13 interrupt	0x0000 00A0
25	33	Settable	EXTI14	EXTI Line14 interrupt	0x0000 00A4
26	34	Settable	EXTI15	EXTI Line15 interrupt	0x0000 00A8
27	35	Settable	GPDMA1_CH0	GPDMA1 channel 0 global interrupt	0x0000 00AC
28	36	Settable	GPDMA1_CH1	GPDMA1 channel 1 global interrupt	0x0000 00B0
29	37	Settable	GPDMA1_CH2	GPDMA1 channel 2 global interrupt	0x0000 00B4
30	38	Settable	GPDMA1_CH3	GPDMA1 channel 3 global interrupt	0x0000 00B8
31	39	Settable	GPDMA1_CH4	GPDMA1 channel 4 global interrupt	0x0000 00BC
32	40	Settable	GPDMA1_CH5	GPDMA1 channel 5 global interrupt	0x0000 00C0
33	41	Settable	GPDMA1_CH6	GPDMA1 channel 6 global interrupt	0x0000 00C4
34	42	Settable	GPDMA1_CH7	GPDMA1 channel 7 global interrupt	0x0000 00C8
35	43	Settable	IWDG	Independent watchdog interrupt	0x0000 00CC
36	44	Settable	SAES ⁽¹⁾	Secure AES	0x0000 00D0
37	45	Settable	ADC1	ADC1 global interrupt	0x0000 00D4
38	46	Settable	DAC1	DAC1 global interrupt	0x0000 00D8
39	47	Settable	FDCAN1_IT0	FDCAN1 Interrupt 0	0x0000 00DC
40	48	Settable	FDCAN1_IT1	FDCAN1 Interrupt 1	0x0000 00E0
41	49	Settable	TIM1_BRK/TIM1_TERR/ TIM1_IERR	TIM1 break/transition error/ index error	0x0000 00E4
42	50	Settable	TIM1_UP	TIM1 update	0x0000 00E8
43	51	Settable	TIM1_TRG_COM/ TIM1_DIR/TIM1_IDX	TIM1 trigger and commutation/direction change interrupt/index	0x0000 00EC
44	52	Settable	TIM1_CC	TIM1 capture compare interrupt	0x0000 00F0

Table 146. STM32H562/563/573xx vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
45	53	Settable	TIM2	TIM2 global interrupt	0x0000 00F4
46	54	Settable	TIM3	TIM3 global interrupt	0x0000 00F8
47	55	Settable	TIM4	TIM4 global interrupt	0x0000 00FC
48	56	Settable	TIM5	TIM5 global interrupt	0x0000 0100
49	57	Settable	TIM6	TIM6 global interrupt	0x0000 0104
50	58	Settable	TIM7	TIM7 global interrupt	0x0000 0108
51	59	Settable	I2C1_EV	I2C1 event interrupt	0x0000 010C
52	60	Settable	I2C1_ER	I2C1 error interrupt	0x0000 0110
53	61	Settable	I2C2_EV	I2C2 event interrupt	0x0000 0114
54	62	Settable	I2C2_ER	I2C2 error interrupt	0x0000 0118
55	63	Settable	SPI1	SPI1 global interrupt	0x0000 011C
56	64	Settable	SPI2	SPI2 global interrupt	0x0000 0120
57	65	Settable	SPI3	SPI3 global interrupt	0x0000 0124
58	66	Settable	USART1	USART1 global interrupt	0x0000 0128
59	67	Settable	USART2	USART2 global interrupt	0x0000 012C
60	68	Settable	USART3	USART3 global interrupt	0x0000 0130
61	69	Settable	UART4	UART4 global interrupt	0x0000 0134
62	70	Settable	UART5	UART5 global interrupt	0x0000 0138
63	71	Settable	LPUART1	LPUART1 global interrupt or R wake-up or T wake-up through EXTI line	0x0000 013C
64	72	Settable	LPTIM1 or LPTIM1_AIT	LPTIM1 global interrupt or AIT through EXTI line	0x0000 0140
65	73	Settable	TIM8_BRK/TIM8_TERR/ TIM8_IERR	TIM8 break interrupt/transition error/index error	0x0000 0144
66	74	Settable	TIM8_UP	TIM8 update interrupt	0x0000 0148
67	75	Settable	TIM8_TRG_COM/ TIM8_DIR/TIM8_IDX	TIM8 trigger and commutation interrupt/ direction change interrupt/index	0x0000 014C
68	76	Settable	TIM8_CC	TIM8 capture compare interrupt	0x0000 0150
69	77	Settable	ADC2	ADC2 global interrupt	0x0000 0154
70	78	Settable	LPTIM2 or LPTIM2_AIT	LPTIM2 global interrupt or AIT through EXTI line	0x0000 0158
71	79	Settable	TIM15	TIM15 global interrupt	0x0000 015C
72	80	Settable	TIM16	TIM16 global interrupt	0x0000 0160
73	81	Settable	TIM17	TIM17 global interrupt	0x0000 0164
74	82	Settable	USB_FS	USB FS global interrupt	0x0000 0168

Table 146. STM32H562/563/573xx vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
75	83	Settable	CRS	Clock recovery system global interrupt	0x0000 016C
76	84	Settable	UCPD1	UCPD1 global interrupt	0x0000 0170
77	85	Settable	FMC	FMC global interrupt	0x0000 0174
78	86	Settable	OCTOSPI1	OCTOSPI1 global interrupt	0x0000 0178
79	87	Settable	SDMMC1	SDMMC1 global interrupt	0x0000 017C
80	88	Settable	I2C3_EV	I2C3 event interrupt	0x0000 0180
81	89	Settable	I2C3_ER	I2C3 error interrupt	0x0000 0184
82	90	Settable	SPI4	SPI4 global interrupt	0x0000 0188
83	91	Settable	SPI5	SPI5 global interrupt	0x0000 018C
84	92	Settable	SPI6	SPI6 global interrupt	0x0000 0190
85	93	Settable	USART6	USART6 global interrupt	0x0000 0194
86	94	Settable	USART10	USART10 global interrupt	0x0000 0198
87	95	Settable	USART11	USART11 global interrupt	0x0000 019C
88	96	Settable	SAI1	SAI1 global interrupt	0x0000 01A0
89	97	Settable	SAI2	SAI2 global interrupt	0x0000 01A4
90	98	Settable	GPDMA2_CH0	GPDMA2 channel0 global interrupt	0x0000 01A8
91	99	Settable	GPDMA2_CH1	GPDMA2 channel1 global interrupt	0x0000 01AC
92	100	Settable	GPDMA2_CH2	GPDMA2 channel2 global interrupt	0x0000 01B0
93	101	Settable	GPDMA2_CH3	GPDMA2 channel3 global interrupt	0x0000 01B4
94	102	Settable	GPDMA2_CH4	GPDMA2 channel4 global interrupt	0x0000 01B8
95	103	Settable	GPDMA2_CH5	GPDMA2 channel5 global interrupt	0x0000 01BC
96	104	Settable	GPDMA2_CH6	GPDMA2 channel6 global interrupt	0x0000 01C0
97	105	Settable	GPDMA2_CH7	GPDMA2 channel7 global interrupt	0x0000 01C4
98	106	Settable	UART7	UART7 global interrupt	0x0000 01C8
99	107	Settable	UART8	UART8 global interrupt	0x0000 01CC
100	108	Settable	UART9	UART9 global interrupt	0x0000 01D0
101	109	Settable	UART12	UART12 global interrupt	0x0000 01D4
102	110	Settable	SDMMC2 ⁽²⁾	SDMMC2 global interrupt	0x0000 01D8
103	111	Settable	FPU	Floating point interrupt	0x0000 01DC
104	112	Settable	ICACHE	Instruction cache global interrupt	0x0000 01E0
105	113	Settable	DCACHE	Data cache global interrupt	0x0000 01E4
106	114	Settable	ETH ⁽²⁾	Ethernet interrupt	0x0000 01E8
107	115	Settable	ETH_WKUP	Ethernet wake-up interrupt through EXTI line	0x0000 01EC

Table 146. STM32H562/563/573xx vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
108	116	Settable	DCMI_PSSI	DCMI/PSSI global interrupt	0x0000 01F0
109	117	Settable	FDCAN2_IT0 ⁽²⁾	FDCAN2 interrupt 0	0x0000 01F4
110	118	Settable	FDCAN2_IT1 ⁽²⁾	FDCAN2 interrupt 1	0x0000 01F8
111	119	Settable	CORDIC	CORDIC interrupt	0x0000 01FC
112	120	Settable	FMAC	FMAC interrupt	0x0000 0200
113	121	Settable	DTS or DTS_WKUP	DTS interrupt or AIT through EXTI line	0x0000 0204
114	122	Settable	RNG	RNG global interrupt	0x0000 0208
115	123	Settable	OTFDEC1	OTFDEC1 secure global interrupt	0x0000 020C
116	124	Settable	AES ⁽¹⁾	AES global interrupt	0x0000 0210
117	125	Settable	HASH	HASH interrupt	0x0000 0214
118	126	Settable	PKA	PKA global interrupt	0x0000 0218
119	127	Settable	CEC	HDMI-CEC global interrupt	0x0000 021C
120	128	Settable	TIM12	TIM12 global interrupt	0x0000 0220
121	129	Settable	TIM13	TIM13 global interrupt	0x0000 0224
122	130	Settable	TIM14	TIM14 global interrupt	0x0000 0228
123	131	Settable	I3C1_EV	I3C1 event interrupt	0x0000 022C
124	132	Settable	I3C1_ER	I3C1 error interrupt	0x0000 0230
125	133	Settable	I2C4_EV	I2C4 event interrupt	0x0000 0234
126	134	Settable	I2C4_ER	I2C4 error interrupt	0x0000 0238
127	135	Settable	LPTIM3 or LPTIM3_AIT	LPTIM3 global interrupt or AIT through EXTI line	0x0000 023C
128	136	Settable	LPTIM4 or LPTIM4_AIT	LPTIM4 global interrupt or AIT through EXTI line	0x0000 0240
129	137	Settable	LPTIM5 or LPTIM5_AIT	LPTIM5 global interrupt or AIT through EXTI line	0x0000 0244
130	138	Settable	LPTIM6 or LPTIM6_AIT	LPTIM6 global interrupt or AIT through EXTI line	0x0000 0248

1. Not available on STM32H562/563 devices.

2. Not available on STM32H562 devices.

Table 147. STM32H523/533xx vector table

Position	Priority	Type of priority	Acronym	Description	Address
-	-	-	-	Reserved	0x0000 0000
-	-4	Fixed	SettableReset	Reset	0x0000 0004
-	-2	Fixed	NMI	Non maskable interrupt. The RCC clock security system (CSS) is linked to the NMI vector.	0x0000 0008
-	-3 or -1	Fixed	Secure HardFault	Secure Hard fault	0x0000 000C
-	-1	Fixed	Non-secure HardFault	Non-secure Hard fault. All classes of fault.	0x0000 000C
-	0	Settable	MemManage	Memory management	0x0000 0010
-	1	Settable	BusFault	Pre-fetch fault, memory access fault	0x0000 0014
-	2	Settable	UsageFault	Undefined instruction or illegal state	0x0000 0018
-	3	Settable	SecureFault	Secure fault	0x0000 001C
-	-	-	-	Reserved	0x0000 0020 - 0x0000 0028
-	4	-	SVC	System service call via SWI instruction	0x0000 002C
-	5	-	Debug Monitor	Monitor	0x0000 0030
-	-	-	-	Reserved	0x0000 0034
-	6	Settable	PendSV	Pendable request for system service	0x0000 0038
-	7	Settable	SysTick	System tick timer	0x0000 003C
0	8	Settable	WWDG	Window watchdog interrupt	0x0000 0040
1	9	Settable	PVD_AVD	Power voltage monitor/ Analog voltage monitor	0x0000 0044
2	10	Settable	RTC	RTC global non-secure interrupts	0x0000 0048
3	11	Settable	RTC_S	RTC global secure interrupts	0x0000 004C
4	12	Settable	TAMP	Tamper global interrupts	0x0000 0050
5	13	Settable	RAMCFG	RAM configuration global interrupt	0x0000 0054
6	14	Settable	FLASH	Flash non-secure global interrupt	0x0000 0058
7	15	Settable	FLASH_S	Flash secure global interrupt	0x0000 005C
8	16	Settable	GTZC	GTZC global interrupt	0x0000 0060
9	17	Settable	RCC	RCC non-secure global interrupt	0x0000 0064
10	18	Settable	RCC_S	RCC secure global interrupt	0x0000 0068
11	19	Settable	EXTI0	EXTI Line0 interrupt	0x0000 006C
12	20	Settable	EXTI1	EXTI Line1 interrupt	0x0000 0070
13	21	Settable	EXTI2	EXTI Line2 interrupt	0x0000 0074
14	22	Settable	EXTI3	EXTI Line3 interrupt	0x0000 0078

Table 147. STM32H523/533xx vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
15	23	Settable	EXTI4	EXTI Line4 interrupt	0x0000 007C
16	24	Settable	EXTI5	EXTI Line5 interrupt	0x0000 0080
17	25	Settable	EXTI6	EXTI Line6 interrupt	0x0000 0084
18	26	Settable	EXTI7	EXTI Line7 interrupt	0x0000 0088
19	27	Settable	EXTI8	EXTI Line8 interrupt	0x0000 008C
20	28	Settable	EXTI9	EXTI Line9 interrupt	0x0000 0090
21	29	Settable	EXTI10	EXTI Line10 interrupt	0x0000 0094
22	30	Settable	EXTI11	EXTI Line11 interrupt	0x0000 0098
23	31	Settable	EXTI12	EXTI Line12 interrupt	0x0000 009C
24	32	Settable	EXTI13	EXTI Line13 interrupt	0x0000 00A0
25	33	Settable	EXTI14	EXTI Line14 interrupt	0x0000 00A4
26	34	Settable	EXTI15	EXTI Line15 interrupt	0x0000 00A8
27	35	Settable	GPDMA1_CH0	GPDMA1 channel 0 global interrupt	0x0000 00AC
28	36	Settable	GPDMA1_CH1	GPDMA1 channel 1 global interrupt	0x0000 00B0
29	37	Settable	GPDMA1_CH2	GPDMA1 channel 2 global interrupt	0x0000 00B4
30	38	Settable	GPDMA1_CH3	GPDMA1 channel 3 global interrupt	0x0000 00B8
31	39	Settable	GPDMA1_CH4	GPDMA1 channel 4 global interrupt	0x0000 00BC
32	40	Settable	GPDMA1_CH5	GPDMA1 channel 5 global interrupt	0x0000 00C0
33	41	Settable	GPDMA1_CH6	GPDMA1 channel 6 global interrupt	0x0000 00C4
34	42	Settable	GPDMA1_CH7	GPDMA1 channel 7 global interrupt	0x0000 00C8
35	43	Settable	IWDG	Independent watchdog interrupt	0x0000 00CC
36	44	Settable	SAES ⁽¹⁾	Secure AES	0x0000 00D0
37	45	Settable	ADC1	ADC1 global interrupt	0x0000 00D4
38	46	Settable	DAC1	DAC1 global interrupt	0x0000 00D8
39	47	Settable	FDCAN1_IT0	FDCAN1 Interrupt 0	0x0000 00DC
40	48	Settable	FDCAN1_IT1	FDCAN1 Interrupt 1	0x0000 00E0
41	49	Settable	TIM1_BRK/TIM1_TERR/ TIM1_IERR	TIM1 break/transition error/ index error	0x0000 00E4
42	50	Settable	TIM1_UP	TIM1 update	0x0000 00E8
43	51	Settable	TIM1_TRG_COM/ TIM1_DIR/TIM1_IDX	TIM1 trigger and commutation/direction change interrupt/index	0x0000 00EC
44	52	Settable	TIM1_CC	TIM1 capture compare interrupt	0x0000 00F0
45	53	Settable	TIM2	TIM2 global interrupt	0x0000 00F4
46	54	Settable	TIM3	TIM3 global interrupt	0x0000 00F8

Table 147. STM32H523/533xx vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
47	55	Settable	TIM4	TIM4 global interrupt	0x0000 00FC
48	56	Settable	TIM5	TIM5 global interrupt	0x0000 0100
49	57	Settable	TIM6	TIM6 global interrupt	0x0000 0104
50	58	Settable	TIM7	TIM7 global interrupt	0x0000 0108
51	59	Settable	I2C1_EV	I2C1 event interrupt	0x0000 010C
52	60	Settable	I2C1_ER	I2C1 error interrupt	0x0000 0110
53	61	Settable	I2C2_EV	I2C2 event interrupt	0x0000 0114
54	62	Settable	I2C2_ER	I2C2 error interrupt	0x0000 0118
55	63	Settable	SPI1	SPI1 global interrupt	0x0000 011C
56	64	Settable	SPI2	SPI2 global interrupt	0x0000 0120
57	65	Settable	SPI3	SPI3 global interrupt	0x0000 0124
58	66	Settable	USART1	USART1 global interrupt	0x0000 0128
59	67	Settable	USART2	USART2 global interrupt	0x0000 012C
60	68	Settable	USART3	USART3 global interrupt	0x0000 0130
61	69	Settable	UART4	UART4 global interrupt	0x0000 0134
62	70	Settable	UART5	UART5 global interrupt	0x0000 0138
63	71	Settable	LPUART1	LPUART1 global interrupt or R wake-up or T wake-up through EXTI line	0x0000 013C
64	72	Settable	LPTIM1 or LPTIM1_AIT	LPTIM1 global interrupt or AIT through EXTI line	0x0000 0140
65	73	Settable	TIM8_BRK/TIM8_TERR/ TIM8_IERR	TIM8 break interrupt/transition error/index error	0x0000 0144
66	74	Settable	TIM8_UP	TIM8 update interrupt	0x0000 0148
67	75	Settable	TIM8_TRG_COM/ TIM8_DIR/TIM8_IDX	TIM8 trigger and commutation interrupt/ direction change interrupt/index	0x0000 014C
68	76	Settable	TIM8_CC	TIM8 capture compare interrupt	0x0000 0150
69	77	Settable	ADC2	ADC2 global interrupt	0x0000 0154
70	78	Settable	LPTIM2 or LPTIM2_AIT	LPTIM2 global interrupt or AIT through EXTI line	0x0000 0158
71	79	Settable	TIM15	TIM15 global interrupt	0x0000 015C
72	80	-	-	Reserved	0x0000 0164
73	81	-	-	Reserved	0x0000 0168
74	82	Settable	USB_FS	USB FS global interrupt	0x0000 0168
75	83	Settable	CRS	Clock recovery system global interrupt	0x0000 016C
76	84	Settable	UCPD1	UCPD1 global interrupt	0x0000 0170

Table 147. STM32H523/533xx vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
77	85	Settable	FMC	FMC global interrupt	0x0000 0174
78	86	Settable	OCTOSPI1	OCTOSPI1 global interrupt	0x0000 0178
79	87	Settable	SDMMC1	SDMMC1 global interrupt	0x0000 017C
80	88	Settable	I2C3_EV	I2C3 event interrupt	0x0000 0180
81	89	Settable	I2C3_ER	I2C3 error interrupt	0x0000 0184
82	90	Settable	SPI4	SPI4 global interrupt	0x0000 0188
83	91	-	-	Reserved	0x0000 018C
84	92	-	-	Reserved	0x0000 0190
85	93	Settable	USART6	USART6 global interrupt	0x0000 0194
86	94	-	-	Reserved	0x0000 0198
87	95	-	-	Reserved	0x0000 019C
88	96	-	-	Reserved	0x0000 01A0
89	97	-	-	Reserved	0x0000 01A4
90	98	Settable	GPDMA2_CH0	GPDMA2 channel0 global interrupt	0x0000 01A8
91	99	Settable	GPDMA2_CH1	GPDMA2 channel1 global interrupt	0x0000 01AC
92	100	Settable	GPDMA2_CH2	GPDMA2 channel2 global interrupt	0x0000 01B0
93	101	Settable	GPDMA2_CH3	GPDMA2 channel3 global interrupt	0x0000 01B4
94	102	Settable	GPDMA2_CH4	GPDMA2 channel4 global interrupt	0x0000 01B8
95	103	Settable	GPDMA2_CH5	GPDMA2 channel5 global interrupt	0x0000 01BC
96	104	Settable	GPDMA2_CH6	GPDMA2 channel6 global interrupt	0x0000 01C0
97	105	Settable	GPDMA2_CH7	GPDMA2 channel7 global interrupt	0x0000 01C4
98	106	-	-	Reserved	0x0000 01C8
99	107	-	-	Reserved	0x0000 01CC
100	108	-	-	Reserved	0x0000 01D0
101	109	-	-	Reserved	0x0000 01D4
102	110	-	-	Reserved	0x0000 01D8
103	111	Settable	FPU	Floating point interrupt	0x0000 01DC
104	112	Settable	ICACHE	Instruction cache global interrupt	0x0000 01E0
105	113	Settable	DCACHE	Data cache global interrupt	0x0000 01E4
106	114	-	-	Reserved	0x0000 01E8
107	115	-	-	Reserved	0x0000 01EC
108	116	Settable	DCMI_PSSI	DCMI/PSSI global interrupt	0x0000 01F0
109	117	Settable	FDCAN2_IT0	FDCAN2 interrupt 0	0x0000 01F4

Table 147. STM32H523/533xx vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
110	118	Settable	FDCAN2_IT1	FDCAN2 interrupt 1	0x0000 01F8
111	119	-	-	Reserved	0x0000 01FC
112	120	-	-	Reserved	0x0000 0200
113	121	Settable	DTS or DTS_WKUP	DTS interrupt or AIT through EXTI line	0x0000 0204
114	122	Settable	RNG	RNG global interrupt	0x0000 0208
115	123	Settable	OTFDEC1	OTFDEC1 secure global interrupt	0x0000 020C
116	124	Settable	AES ⁽¹⁾	AES global interrupt	0x0000 0210
117	125	Settable	HASH	HASH interrupt	0x0000 0214
118	126	Settable	PKA ⁽¹⁾	PKA global interrupt	0x0000 0218
119	127	Settable	CEC	HDMI-CEC global interrupt	0x0000 021C
120	128	Settable	TIM12	TIM12 global interrupt	0x0000 0220
121	129	-	-	Reserved	0x0000 0224
122	130	-	-	Reserved	0x0000 0228
123	131	Settable	I3C1_EV	I3C1 event interrupt	0x0000 022C
124	132	Settable	I3C1_ER	I3C1 error interrupt	0x0000 0230
125	133	-	-	Reserved	0x0000 0234
126	134	-	-	Reserved	0x0000 0238
127	135	-	-	Reserved	0x0000 023C
128	136	-	-	Reserved	0x 0000 0240
129	137	-	-	Reserved	0x0000 0244
130	138	-	-	Reserved	0x0000 0248
131	139	Settable	I3C2_EV	I3C2 event interrupt	0x0000 024C
132	140	Settable	I3C2_ER	I3C2 error interrupt	0x0000 0250
133	141	-	-	Reserved	0x0000 0254

1. Not available on STM32H523 devices.

18 Extended interrupts and event controller (EXTI)

The extended interrupts and event controller (EXTI) manages the individual CPU and system wake-up through configurable and direct event inputs. It provides wake-up requests to the power control and generates an interrupt request to the CPU NVIC and events to the CPU event input. For the CPU, an additional event generation block (EVG) is needed to generate the CPU event signal.

The EXTI wake-up requests allow the system to be woken up from Stop modes.

The interrupt request and event request generation can be used also in Run modes.

The EXTI also includes the EXTI mux IO port selection.

18.1 EXTI main features

The EXTI main features are the following:

- Up to 59 input events supported
- All event inputs allow the possibility to wake up the system.
- Events that do not have an associated wake-up flag in the peripheral, have a flag in the EXTI and generate an interrupt to the CPU from the EXTI.
- Events can be used to generate a CPU wake-up event.

The asynchronous event inputs are classified in two groups:

- Configurable events (signals from I/Os or peripherals able to generate a pulse), with the following features
 - Selectable active trigger edge
 - Interrupt pending status register bits independent for the rising and falling edge
 - Individual interrupt and event generation mask, used for conditioning the CPU wake-up, interrupt and event generation
 - Software trigger possibility
 - Secure events: The access to control and configuration bits of secure input events can be made secure and or privilege.
 - EXTI IO port selection
- Direct events (interrupt and wake-up sources from peripherals having an associated flag which requires to be cleared in the peripheral), with the following features:
 - Fixed rising edge active trigger
 - No interrupt pending status register bit in the EXTI (the interrupt pending status flag is provided by the peripheral generating the event)
 - Individual interrupt and event generation mask, used to condition the CPU wake-up and event generation
 - No software trigger possibility

18.2 EXTI block diagram

The EXTI consists of a register block accessed via an AHB interface, the event input trigger block, the masking block and EXTI mux as shown in [Figure 91](#).